

● INSIDOR

Engineering Architecture

The repository, the database, auth, the pipeline service, contracts, deployment and testing — with the real migrations, configs and types inline.

ARCHITECTURE.MD · 27 JULY 2026
40,416 WORDS · GENERATED FROM THE REPOSITORY, NOT MAINTAINED SEPARATELY

Contents

1. THE SHAPE

What runs where
The shared packages

2. THE REPOSITORY

The tree
The feature-folder contract
The dependency graph
Enforcement, in the order a developer hits it
What a reach-past looks like, in firing order
The shared/ graduation rule

3. THE DATABASE

3.1 The five product rules the schema enforces
3.2 A note on where Postgres enforces immutability, and where it does not
3.3 The enums
3.4 story — where coinability, display eligibility and promote immutability live
3.5 post and the heat tables
3.6 coin, coin_snapshot, coin_mcap_series, coin_safety
3.7 story_ticker and coin_match — the two tables that cannot touch
3.8 The clocks — ct_mention, sensor_*, story_clock, story_outcome
3.9 The board — ranking is not a query
3.10 User, social, trading and launch
3.11 Search and ops
3.12 The views — one row for every Create affordance
3.13 The four hot queries

4. MIGRATIONS

4.1 The files, in order
4.2 The framework
4.3 The applier
4.4 Authoring rules, enforced by scripts/check-migrations.mjs
4.5 Forward-only, expand → backfill → contract
4.6 Out-of-band DDL cannot hide

5. AUTH AND SECURITY

5.1 The decision: server-side writes on the service role
5.2 The flow end to end
5.3 The verification code
5.4 RLS — the read path, and the correction to 0011
5.5 The service-role boundary
5.6 Realtime auth
5.7 Rate limiting
5.8 Wallet-signed actions
5.9 Secrets
5.10 Abuse

6. THE PIPELINE SERVICE

6.1 The execution model
6.2 The stage contract
6.3 Scheduling and concurrency
6.4 Budgets and the error taxonomy
6.5 The websocket clock
6.6 Observability
6.7 Local development
6.8 Deployment

7. CONTRACTS

7.1 Row types are generated, never hand-written
7.2 An unconfirmed Buy is a compile error
7.3 The one rule, once
7.4 Missing data cannot render as zero
7.5 What the brand does not do, stated plainly
7.6 The confidence stamp
7.7 Agreed functions
7.8 Ownership — is any table written by both apps?

8. ENVIRONMENTS, CI AND DEPLOYMENT

8.1 Four environments over three Postgres instances
8.2 Secrets
8.3 CI
8.4 The deploy-order rule for schema changes
8.5 Rollback

9. TESTING AND CORRECTNESS

9.1	The strategy, and what is deliberately not tested
9.2	The correctness-critical set
9.3	The frozen regression set
9.4	Testing the LLM stages
9.5	Integration tests
9.6	What replaces end-to-end
9.7	The first ten tests

10. OPEN ITEMS

This is the document you open to set up the repository and start building. It covers the shape of the system, the schema, the boundaries, and the machinery that keeps them honest. Its companion, `docs/DESIGN.md`, covers the product and the screens; nothing here repeats it.

Two rules govern every decision below.

A wrong state should be unrepresentable, not merely discouraged. Every defect the last build shipped — an LLM-invented ticker resolved against a live market, a safety checker that reported every token safe because it read fields the endpoint did not have, an age of zero for a coin whose mint time was unknown, a lead time nobody could reproduce — passed code review. So the controls live in CHECK constraints, generated columns, column-level grants, type brands, and failing tests. Not in conventions.

Absent is not zero. A missing number renders as an em dash with a reason, at every layer, from the column type to the React component. `?? 0` is a lint error.

1. THE SHAPE

Two deployables, four shared packages, one Postgres.

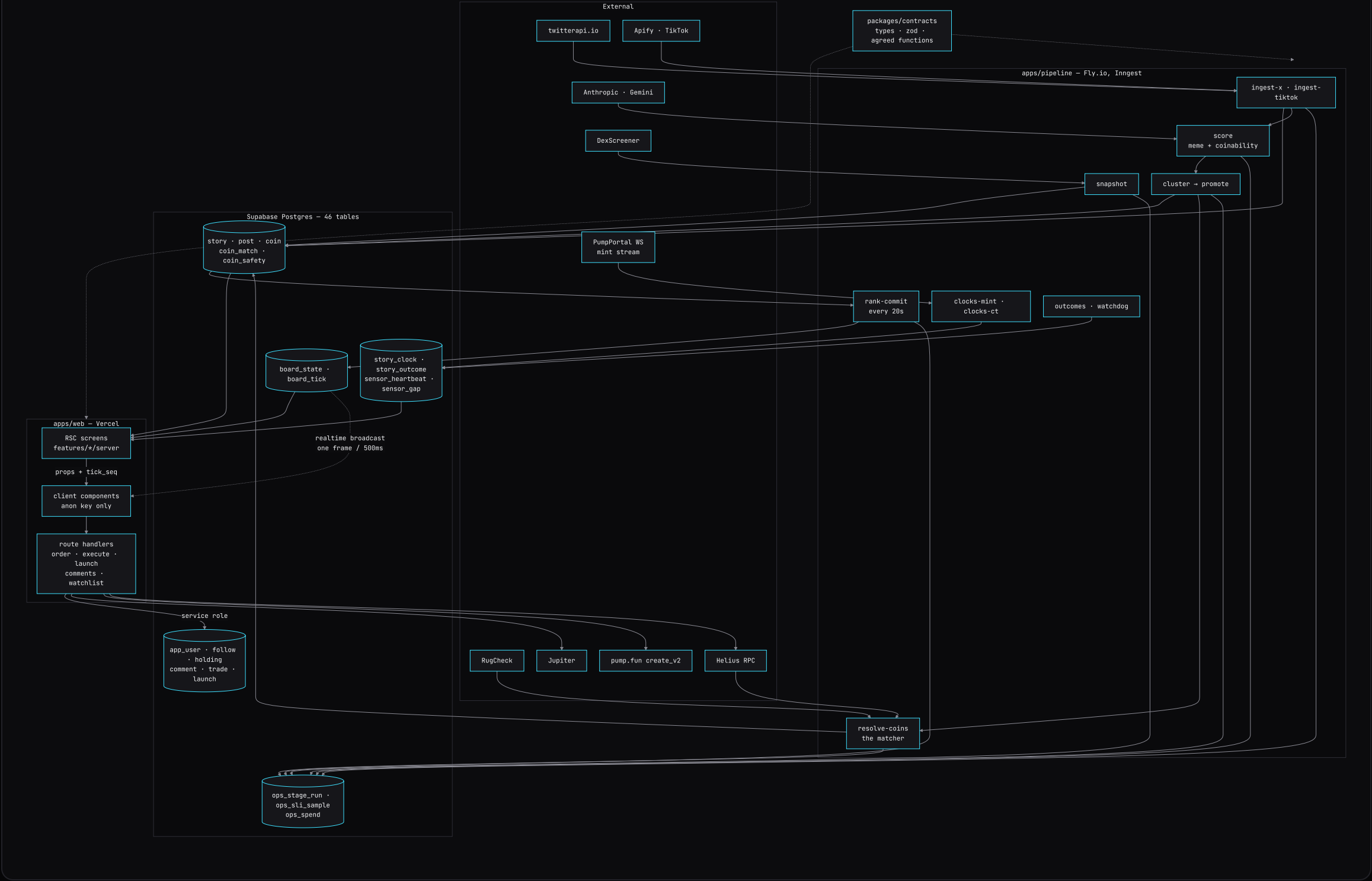
`apps/web` — Next.js 16 on Vercel. Renders every surface, holds every route handler, and is the only thing Vercel builds. It reads through RLS-scoped policies and writes only through server routes holding the service role. It never imports `apps/pipeline`.

`apps/pipeline` — a long-lived Node service on Fly.io, scheduled by Inngest. It holds the service-role key, a PumpPortal websocket, and every metered vendor credential. It is the sole writer of story, post, coin, match, clock, board and ops state. It never imports React and never imports `apps/web`.

They share a database and nothing else. All common code goes through `packages/contracts`, which is a leaf with one runtime dependency (`zod`).

What runs where

CONCERN	HOME	WHY NOT THE OTHER PLACE
Board ranking (20s commit)	pipeline	A board that ranks inside the request gets slower exactly when it gets popular, and rank stability (dwell, hysteresis, MAX_MOVE) cannot be expressed in a request-scoped <code>ORDER BY</code> .
Board read	web, 40-row index scan on <code>board_state</code>	—
Coin matching	pipeline	Needs image embeddings, mint-stream history and a matcher version. Nothing about it is request-shaped.
Quote / order / execute	web route handlers	Needs the user's wallet, an edge rate limit keyed on that wallet, and a <code>request_id</code> uniqueness check at submit.
Launch (pump.fun <code>create_v2</code>)	web route handlers	Same. The mint keypair is generated and persisted encrypted before the first signature.
Comment insert	web route handler	The honesty stamp needs a server-side chain read; a client-supplied stamp is a fabricated stamp.
Follow / watchlist	web route handler	The highest-volume follower class is anonymous device follows, which have no principal a policy could scope.
Notifications fan-out	pipeline	Triggered by a $0 \rightarrow \geq 1$ confirmed-coin transition, which only the matcher observes.
Scheduling	Inngest, inside the pipeline	Vercel Cron has no singleton, no backpressure, no retries, and no concept of "ran but produced nothing" — which is the exact failure that went unnoticed for two days.



The shared packages

PACKAGE	CONTENTS	DEPENDENCIES
packages/contracts	branded ids, enums mirrored from the database, generated row types, <code>Pending<T></code> , <code>BuyableCoin</code> , <code>primaryAction()</code> , the lead-time formatter, every wire and input zod schema	<code>zod</code> only
packages/db	three client factories — <code>browser.ts</code> (anon), <code>server.ts</code> (request-scoped), <code>service.ts</code> (service role, <code>import 'server-only'</code> , firewalled) — plus <code>types.generated.ts</code>	<code>@supabase/supabase-js</code> , <code>contracts</code>
packages/env	the only module in the repository permitted to read <code>process.env</code> ; zod schemas with no defaults and no fallbacks	<code>zod</code>
packages/config	shared tsconfig, eslint and vitest bases	—

Packages are **source-only**: no build step, no `dist/` , no `.d.ts` emit. `apps/web` compiles them via `transpilePackages` ; `apps/pipeline` compiles them in one esbuild call. That is deliberate — it removes the build graph, and with it the case for Turborepo. Build order is `nothing → nothing → (next build | esbuild)` . Adopt Turbo when CI wall time exceeds five minutes or a third deployable appears; record the reversal in `docs/adr/0001-no-turborepo.md` so it is a decision and not a drift.

2. THE REPOSITORY

The tree

```
insidor/
├─ package.json                # npm workspaces: apps/*, packages/*. Root holds no source.
├─ tsconfig.base.json
├─ eslint.config.mjs           # flat config; feature zones GENERATED from the graph
├─ .dependency-cruiser.cjs      # authoritative boundary gate; same graph
├─ .nvmrc
├─ CONTRIBUTING.md             # the shared/ graduation rule, verbatim
├─ .github/
│  └─ actions/setup/action.yml
│  └─ workflows/{ci,db,release,preview,nightly}.yaml
│  └─ CODEOWNERS               # supabase/migrations/** requires a founder review
│  └─ pull_request_template.md
├─ docs/
│  └─ DESIGN.md ARCHITECTURE.md environments.md schema.md
│  └─ architecture.svg         # GENERATED + COMMITTED. CI fails if stale.
│  └─ runbooks/rollback.md
│  └─ adr/{0001-no-turborepo,0002-feature-dag,0003-stamp-is-structural}.md
├─ scripts/
│  └─ migrate.mjs              # the ONLY path to a database
│  └─ feature-graph.json       # THE module boundary, as data
│  └─ feature-graph.mjs        # ESM view + cycle assertion
│  └─ check-model-tests.mjs     # every model/*.ts needs a sibling test
│  └─ check-app-thin.mjs        # app/** exports only Next's contract names
│  └─ check-migrations.mjs      # immutability, ordering, phase, lock safety
│  └─ check-env-parity.mjs
│  └─ check-vercel-env-scopes.mjs # no secret scoped to two environments
│  └─ release-plan.mjs         # reads @phase → emits db_first
│  └─ render-architecture.mjs
│  └─ smoke.mjs
├─ supabase/
│  └─ config.toml              # [db.seed] sql_paths → seed/*.sql
│  └─ migrations/0000..0015_*.sql # timestamps are ordinals; immutable once merged
│  └─ seed/
│     └─ 00_fixtures.sql       # captured from staging, redacted, frozen
│     └─ scenarios/01..09_*.sql # hand-written, never captured, never deleted
│  └─ queries/hot-queries.sql   # the four hot queries + the index each drives
│  └─ tests/{invariants,plans}.sql
│  └─ schema.sql               # pg_dump snapshot; CI diffs against it
│
├─ apps/
│  └─ pipeline/                # Node. Never built by Vercel. Never imported by web.
│     └─ Dockerfile docker-entrypoint.sh build.mjs
│     └─ fly.staging.toml fly.production.toml
│     └─ src/
│        └─ main.ts            # Inngest serve + /health/live + /health/ready
│        └─ preflight.ts        # env, ops_environment, schema version, vendor ping
│        └─ inngest/{client.ts,functions.ts}
│        └─ stages/
│           └─ ingest-x/ ingest-tiktok/ snapshot/
│           └─ score/           # meme_score + coinability – the ONLY LLM tier
│           └─ cluster/ promote/ # promote writes promoted_at exactly once
│           └─ resolve-coins/    # the matcher. 0.97 precision, abstain band.
│           └─ clocks-mint/ clocks-ct/
│           └─ rank-commit/      # board_state + board_tick, every 20s
│           └─ outcomes/ watchdog/
│              └─ (each: index.ts · model/ · io/ · __tests__/)
│        └─ adapters/           # ONE file per vendor; the only place a base URL exists
│           └─ twitterapi.ts apify.ts pumpportal.ts dexscreeners.ts
│           └─ rugcheck.ts helius.ts jupiter.ts pumfun.ts anthropic.ts gemini.ts
│           └─ __tests__/        # recorded-fixture contract test per adapter
│        └─ lib/{budget.ts,retry.ts,gaps.ts,logger.ts,sls.ts,stage.ts}
│
└─ web/                        # THE ONLY THING VERCEL BUILDS
   └─ next.config.ts vercel.json vitest.config.ts instrumentation.ts
   └─ src/
      └─ app/                  # ROUTING MANIFEST. 40 lines max per file. No logic.
         └─ layout.tsx globals.css
         └─ (app)/layout.tsx (app)/page.tsx
         └─ (app)/feed/page.tsx (app)/trending/page.tsx
         └─ (app)/search/page.tsx (app)/you/page.tsx
         └─ (app)/story/[id]/page.tsx (app)/story/[id]/opengraph-image.tsx
         └─ (app)/coin/[mint]/page.tsx (app)/create/page.tsx
         └─ c/[mint]/page.tsx      # public share card, revalidate 300
         └─ (ops)/ops/page.tsx     (ops)/ops/earliness/page.tsx
         └─ api/
            └─ order/route.ts execute/route.ts txstatus/route.ts
            └─ launch/prepare/route.ts launch/confirm/route.ts
            └─ comments/route.ts watchlist/route.ts push/subscribe/route.ts
            └─ board/[lane]/route.ts # hydration + disconnect fallback
            └─ health/route.ts
            └─ telemetry/{search,resync}/route.ts
```



```
| | └ ops/                # /ops, SLIs, earliness ledger. Graph root; imports nothing.
| | |
| | └ shared/
| |   └ ui/                # design system. NO domain vocabulary in props.
| |   └ db/                # re-export of packages/db factories
| |   └ realtime/          # ONE ref-counted socket + the 50-minute setAuth loop
| |   └ rpc/               # Helius client factory. No domain.
| |   └ format/            # numbers, time, addresses
| |     └ pending.tsx      # ALLOWLISTED domain: the - / --title / .. contract
| |   └ stamp/             # ALLOWLISTED domain: the confidence stamp
| |   └ lead/              # ALLOWLISTED domain: the lead-time claim
| |   └ staleness/         # ALLOWLISTED domain: tier() + the band
| |   └ fee/               # ALLOWLISTED domain: the fee line
| |   └ routes.ts          # every href in the product. Pure strings.
| |   └ hooks/             # useInterval, useMediaQuery, useLocalStorage
|
└ packages/
  └ contracts/src/
    └ index.ts ids.ts enums.ts naming.ts
    └ generated/database.types.ts    # GENERATED. NEVER HAND-EDITED.
    └ rows.ts                        # the ONLY file that imports generated/
    └ pending.ts match.ts action.ts lead.ts safety.ts
    └ wire/{envelope,board,coin,comment}.ts
    └ input/{follow,comment,order,execute,launch,prefs,report,telemetry}.ts
    └ __tests__/{action,pending,lead}.test.ts
    └ __typetests__/{brand,pending,drift}.test-d.ts
  └ db/src/{index,browser,server,service,types.generated}.ts
  └ env/src/index.ts
  └ config/{tsconfig.base.json,eslint.base.mjs,vitest.base.ts}
```

The feature-folder contract

Eleven features. Each exposes **exactly two** entry points and nothing else:

- `index.ts` — client-safe. View-models, pure model functions, client components, hooks. Nothing exported here may transitively reach a database client, a `features/*/server` module, or a Node builtin.
- `server.ts` — first line `import 'server-only'`. RSC screens, queries, route handlers, repositories.

Inside a feature there are exactly four segments, and the rules differ per segment:

Flat segments are what makes "which file is this bug in" answerable by reading the tree. `ui/` gets one level of nesting because the two lanes genuinely are two component families.

`app/` is a routing manifest. Files there are capped at 40 lines and may import **only** `@features/<name>`, `@server/<name>`, `@shared/ui` and `@shared/routes`. The cap alone would just produce a 39-line page with a query inlined; the import restriction is what removes the cause — a page with no database client, no formatter and no model import has nothing in scope to write business logic with.

The dependency graph

One data file, read by both lint configs. Adding an edge is a one-line diff a reviewer can argue with.

```
{
  "$comment": "THE module boundary. Read by eslint.config.mjs and .dependency-cruiser.cjs. Edit in its own PR.",

  "features": {
    "wallet": { "deps": [], "owns": "Privy provider, session, address, balance, signing, token refresh" },
    "coins": { "deps": [], "owns": "mint identity, market data, safety facts, coin_match, primaryAction, stamp derivation" },
    "ops": { "deps": [], "owns": "/ops, SLI evaluation, earliness ledger, public record" },

    "discussion": { "deps": ["wallet", "coins"], "owns": "comments, snap columns, address gate, moderation ladder, position badge" },
    "stories": { "deps": ["coins", "discussion"], "owns": "story rows, promoted_at, clocks, momentum, outcome, /story/[id]" },

    "trading": { "deps": ["wallet", "coins", "stories"], "owns": "quote/order/execute, trade box, positions, /coin/[mint]" },
    "launch": { "deps": ["wallet", "coins", "stories"], "owns": "create sheet, namer, anti-dupe, create_v2, share card" },
    "watchlist": { "deps": ["wallet", "coins", "stories"], "owns": "the star, /you, mint notification, push" },

    "feed": { "deps": ["coins", "stories", "watchlist"], "owns": "both lanes, control strip, rail, board stability" },
    "trending": { "deps": ["coins", "stories", "watchlist"], "owns": "/trending, the 24h weights" },
    "search": { "deps": ["coins", "stories", "watchlist"], "owns": "the box, four modes, anti-duplicate verdict, family table" }
  },

  "$comment_feed": "feed does NOT depend on trading. The feed's Buy is navigation to the coin page, where the quote is fresh and the fee is disclosed. The board therefore",
  "$comment_stories": "stories → discussion, never the reverse. The caller supplies the snap-column values, so 'written once at insert' is a function signature rather than a habit.",

  "sharedDomainAllowList": [
    "shared/stamp",
    "shared/lead",
    "shared/staleness",
    "shared/fee",
    "shared/format/pending.tsx"
  ],
  "$comment_allowList": "The ONLY shared modules permitted domain vocabulary. A sixth entry requires editing this array, which is visible in the diff.",

  "appMayImport": ["@features/*", "@server/*", "@shared/ui", "@shared/routes"],

  "serviceKeyReachableFrom": [
    "apps/web/src/features/[^/]+/server/",
    "apps/pipeline/src/"
  ]
}
```

Three structural facts fall out of that graph and are worth stating plainly. `coins` is a root because mint identity, safety facts and `primaryAction()` are depended on by everything and depend on nothing. `feed → trading` is absent, so the board physically cannot execute a swap. And `stories → discussion` runs that direction because the comment stamp is written by the caller, which makes "written once at insert" a type signature rather than a habit.

Enforcement, in the order a developer hits it

Layer 1 — TypeScript path mapping. Fires in the editor, before any lint runs.

There is no `@/*` wildcard. That single alias is what made every reach-in import in the old build compile cleanly.

```
// tsconfig.base.json
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2023"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "allowJs": false,
    "checkJs": false,
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitOverride": true,
    "noFallthroughCasesInSwitch": true,
    "noPropertyAccessFromIndexSignature": true,
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "noEmit": true
  }
}
```

```
// apps/web/tsconfig.json
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "lib": ["DOM", "DOM.Iterable", "ES2023"],
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],

    // THE BOUNDARY.
    // @features/feed      → src/features/feed/index.ts    (client-safe entry)
    // @server/feed        → src/features/feed/server.ts    (server-only entry)
    // @features/feed/ui/Row → DOES NOT RESOLVE. ts(2307), in the editor.
    //
    // The two entry points use DIFFERENT ALIAS PREFIXES, and that is not
    // cosmetic. TypeScript ranks path patterns by the length of the text
    // BEFORE the wildcard and nothing else (findBestPatternMatch), so a
    // second pattern "@features/*/server" would tie with "@features/*" on a
    // 10-character prefix, lose the strict-greater comparison, and be
    // silently ignored — binding @features/x/server to
    // src/features/x/server/index.ts, the INTERNAL segment barrel, which has
    // no `import 'server-only'`. That resolves, typechecks clean, and quietly
    // routes around the service-key firewall. A distinct prefix cannot tie.
    "paths": {
      "@features/*":    ["/src/features/*/index.ts"],
      "@server/*":      ["/src/features/*/server.ts"],
      "@shared/*":      ["/src/shared/*"],
      "@insidor/contracts": ["/../../packages/contracts/src/index.ts"],
      "@insidor/db":      ["/../../packages/db/src/index.ts"],
      "@insidor/env":     ["/../../packages/env/src/index.ts"]
    }
  },
  "include": ["next-env.d.ts", "src/**/*.ts", "src/**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules", ".next"]
}
```

Do **not** try `"@features/*": ["/src/features/*.ts", "/src/features/*/index.ts"]` as a way to make both entry points resolve under one prefix. It works, and it opens a strictly wider hole than the one it closes: every deep `.ts` file becomes importable, re-exposing `model/` and `server/` wholesale.

TypeScript project references are deliberately absent. They govern build ordering, not import boundaries — a file importing a package missing from `references` still resolves via source paths — and at this size a full typecheck is seconds.

Layer 2 — ESLint. Fires on save. Zones are generated from the graph; there is no hand-maintained list to drift.

Syntax error in text
mermaid version 11.16.0

```
// eslint.config.mjs
import js from '@eslint/js';
import tseslint from 'typescript-eslint';
import { FlatCompat } from '@eslint/eslintrc';

import {
  FEATURES, forbiddenSiblings, allowedSentence, SHARED_DOMAIN_ALLOWLIST,
} from './scripts/feature-graph.mjs';

const compat = new FlatCompat({ baseDirectory: import.meta.dirname });
const F = 'apps/web/src/features';
const BRANDS = 'BuyableCoin|ConfirmedCoin|MintAddress|Ticker|Instant|Minutes|Usd|Lamports|StoryId';

const featureZones = FEATURES.map((name) => {
  const denied = forbiddenSiblings(name).flatMap((f) => [`@features/${f}`, `@server/${f}`]);
  return {
    files: [`${F}/${name}/**/*.{ts,tsx}`],
    rules: {
      'no-restricted-imports': ['error', {
        patterns: [
          ...(denied.length ? [{
            group: denied,
            message:
              `${allowedSentence(name)}. Either add the edge to scripts/feature-graph.json ` +
              `in this PR and defend it, or move the shared part down the graph.`,
          }] : []),
          {
            group: [`@features/*/*`, `@server/*/*`],
            message:
              'Reach a feature through index.ts or server.ts. Internals are not public API – ' +
              'if you need it, export it from the entry point deliberately.',
          },
          {
            group: [`../../*`, `../../../*`],
            message:
              'This escapes the feature folder. Import the other feature through @features/<name>. ' +
              'dependency-cruiser is authoritative here and catches shallower escapes too.',
          },
          {
            group: [`@insidor/pipeline`, `@insidor/pipeline/*`],
            message: 'apps/web never imports apps/pipeline. Share through packages/contracts.',
          },
        ],
      },
    ],
  },
});

export default tseslint.config(
  { ignores: ['**/.next/**', '**/dist/**', '**/node_modules/**', '**/*.generated.ts'] },

  js.configs.recommended,
  ...tseslint.configs.strictTypeChecked,
  ...compat.extends('next/core-web-vitals'),

  {
    languageOptions: {
      parserOptions: { projectService: true, tsconfigRootDir: import.meta.dirname },
    },
    rules: {
      '@typescript-eslint/consistent-type-imports': 'error',
      '@typescript-eslint/no-floating-promises': 'error',
      '@typescript-eslint/no-non-null-assertion': 'error',
      '@typescript-eslint/no-unnecessary-condition': 'error',
      '@typescript-eslint/switch-exhaustiveness-check': 'error',
      // The three that actually catch a laundered brand. Type-aware, so they
      // require projectService above.
      '@typescript-eslint/no-explicit-any': 'error',
      '@typescript-eslint/no-unsafe-assignment': 'error',
      '@typescript-eslint/no-unsafe-return': 'error',
      '@typescript-eslint/ban-ts-comment': ['error', { 'ts-expect-error': 'allow-with-description' }],
      '@typescript-eslint/strict-boolean-expressions': ['error', {
        allowNullableObject: false, allowNullableString: false, allowNullableNumber: false,
      }],
      eqeqeq: ['error', 'always'],

      'no-restricted-syntax': ['error',
        // Absent is not zero. This is the token-lookup `return 0` bug as a lint rule.
        { selector: 'LogicalExpression[operator="??"][right.value=0]',
          message: 'Absent is not zero. Return Pending<T>; see @insidor/contracts/pending.' },
        { selector: 'LogicalExpression[operator="||"]',
          message: 'Absent is not zero. Return Pending<T>.' },
      ],
    },
  },
);
```

```

    { selector: 'CallExpression[callee.name="Number"][arguments.0.type="Identifier"]',
      message: 'Number(x) coerces null to 0. Parse into Pending<T>.' },
    // Brands are constructed only inside packages/contracts. BOTH assertion
    // forms — `x as Brand` and `<Brand>x` are different AST nodes.
    { selector: `TSAsExpression[typeAnnotation.typeName.name=/^(${BRANDS})$/]`,
      message: 'Branded types are constructed only in packages/contracts. ' +
        'Casting here is how a Buy button reaches an unconfirmed coin.' },
    { selector: `TSTypeAssertion[typeAnnotation.typeName.name=/^(${BRANDS})$/]`,
      message: 'Same rule, angle-bracket form.' },
  ],
},
},

...featureZones,

// model/ is PURE. This is what makes it testable with five-line tests and no mocks.
{
  files: [`${F}/*/model/**/*.ts`, 'apps/pipeline/src/stages/*/model/**/*.ts'],
  rules: {
    'no-restricted-imports': ['error', {
      patterns: [{
        group: [
          'react', 'react-dom', 'react/*', 'next', 'next/*',
          '@supabase/*', '@privy-io/*', '@solana/*', '@insidor/db',
          '@shared/db', '@shared/db/*', '@shared/realtime', '@shared/rpc',
          '**/server/**', '../server/*', './server/*', '@server/*',
        ],
      }],
      message:
        'model/ is pure: no React, no I/O, no async, no clients. ' +
        'If you need data, take it as an argument — that is the whole point of the segment.',
    }],
  },
  'no-restricted-globals': ['error',
    { name: 'fetch', message: 'model/ does no I/O. Pass the data in.' },
    { name: 'window', message: 'model/ is environment-free.' },
    { name: 'document', message: 'model/ is environment-free.' },
    { name: 'localStorage', message: 'model/ is environment-free.' },
  ],
  'no-restricted-properties': ['error',
    { object: 'Math', property: 'random', message: 'model/ must be deterministic. Inject the value.' },
  ],
  'no-restricted-syntax': ['error',
    { selector: "NewExpression[callee.name='Date']",
      message: 'Take `now: number` as a parameter. A model that reads the clock cannot be tested, ' +
        'and age-of-coin is the one axis this product sells.' },
    { selector: "MemberExpression[object.name='Date'][property.name='now']",
      message: 'Take `now: number` as a parameter. Same reason.' },
  ],
  'max-lines-per-function': ['error', { max: 60, skipBlankLines: true, skipComments: true }],
},
},

// THE SERVICE-KEY FIREWALL. Client code cannot name server code.
{
  files: [`${F}/*/ui/**/*.{ts,tsx}`, `${F}/*/hooks/**/*.ts`],
  rules: {
    'no-restricted-imports': ['error', {
      patterns: [{
        group: [
          '**/server/**', '../server', '../server/*', './server/*',
          '@server/*', '@shared/db', '@shared/db/*',
          '@insidor/db', '@insidor/env', 'server-only', 'next/headers',
        ],
      }],
      message:
        'ui/ and hooks/ ship to the browser. They may not name server modules. ' +
        'Pass the data down as props from the RSC in server/, or call a route handler.',
    }],
  },
},
},

// app/ is a routing manifest.
{
  files: ['apps/web/src/app/**/*.{ts,tsx}'],
  rules: {
    'max-lines': ['error', { max: 40, skipBlankLines: true, skipComments: true }],
    'no-restricted-imports': ['error', {
      patterns: [{
        group: [
          '@shared/*', '!@shared/ui', '!@shared/routes',
          '@insidor/db', '@insidor/env', '@supabase/*', '@features/*/*', '@server/*/*',
        ],
      }],
    }],
  },
},

```

```
      message:
        'app/ delegates and does nothing else. It may import @features/<name>, @server/<name>, ' +
        '@shared/ui and @shared/routes. If a page needs a query, the query belongs to a feature ' +
        'and the page renders that feature’s Screen component.',
    }],
  }],
},
},

// shared/ is one-way, and only five modules may speak domain.
{
  files: ['apps/web/src/shared/**/*.{ts,tsx}'],
  rules: {
    'no-restricted-imports': ['error', {
      patterns: [{
        group: ['@features/*', '@server/*'],
        message:
          'shared/ never imports a feature. If shared needs it, it is not shared – ' +
          'it belongs in the feature that owns the concept.',
      }],
    }],
  },
},
{
  files: ['apps/web/src/shared/**/*.{ts,tsx}'],
  ignores: SHARED_DOMAIN_ALLOWLIST.map((p) => `apps/web/src/${p}${p.includes('.') ? '' : '/*'}`),
  rules: {
    'no-restricted-syntax': ['error', {
      selector: 'Identifier[name=/(?:(?:coin|mint|ticker|story|narrative|cashtag|rugcheck|jupiter)/i)]',
      message:
        'shared/ carries no domain vocabulary. <Table columns rows/> is shared; <CoinRow coin/> is a feature. ' +
        'The five exceptions are listed in scripts/feature-graph.json#sharedDomainAllowlist.',
    }],
  },
},
},

// ONE VOCABULARY. The tables say story; nothing says narrative.
{
  files: ['apps/web/src/**/*.{ts,tsx}', 'apps/pipeline/src/**/*.ts'],
  ignores: ['packages/contracts/src/naming.ts'],
  rules: {
    'no-restricted-syntax': ['error', {
      selector: 'Identifier[name=/narrative/i]',
      message:
        'The product word and the table name are both “story”. `narrative` is vocabulary from a ' +
        'codebase being deleted. Two words for one concept is how a codebase grows two ' +
        'implementations of the same rule – which is literally what happened to ct_pickup ' +
        '(a boolean and a regex, two definitions of one event).',
    }],
  },
},
},

// THE TWO CORRECTNESS PROPERTIES, AS LINT RULES.
{
  files: ['apps/web/src/**/*.tsx'],
  ignores: ['apps/web/src/shared/stamp/**'],
  rules: {
    'no-restricted-syntax': ['error', {
      selector: "JSXExpressionContainer > MemberExpression[property.name=/^(mint|mint_ca|ca|contract_address)$/]",
      message:
        'A mint may not be rendered as bare text. Use <MintRef value={stampedMint} /> from @shared/stamp. ' +
        'The stamp is a property of the (story, mint) pair and travels with it everywhere.',
    }],
  },
},
},
{
  files: ['apps/web/src/**/*.{ts,tsx}'],
  ignores: [`${F}/coins/ui/PrimaryAction.tsx`, `${F}/coins/model/action.ts`, `${F}/coins/__tests__/**`],
  rules: {
    'no-restricted-syntax': ['error', {
      selector: 'Literal[value=/^(Buy \\$|See the \\d+ coins|Create coin)/]',
      message:
        'Primary-action labels are produced by primaryAction(), never written at a call site. ' +
        'Render <PrimaryAction action={...} /> from @features/coins.',
    }],
  },
},
},
},

// apps/pipeline is a Node service and never imports React.
{
  files: ['apps/pipeline/**/*.ts'],
  rules: {
```

```

    'no-restricted-imports': ['error', {
      patterns: [{
        group: ['react', 'react-dom', 'next', 'next/*', '@insidor/web', '@insidor/web/*'],
        message: 'apps/pipeline is a Node service and never imports the web app or React.',
      }],
    }],
  },
},
},

// packages/contracts is a leaf: zero runtime deps except zod.
{
  files: ['packages/contracts/**/*.ts'],
  rules: {
    'no-restricted-imports': ['error', {
      patterns: [{
        group: ['@insidor/*', '@shared/*', '@features/*', '@server/*', 'react', 'next', 'next/*', '@supabase/*'],
        message: 'packages/contracts is a leaf with zero runtime deps. Only zod is permitted.',
      }],
    }],
  },
},
},
},
);

```

Layer 3 – dependency-cruiser. The authoritative gate, in CI. It catches what `no-restricted-imports` patterns cannot see: dynamic `import()` , relative-path escapes, and transitive reachability.

```
// .dependency-cruiser.cjs
const graph = require('./scripts/feature-graph.json');

const FEATURES = Object.keys(graph.features);
const W = '^apps/web/src';

// One generated rule per feature. The CI output names the edge you added.
const dagRules = FEATURES.map((f) => {
  const allowed = [f, ...graph.features[f].deps].join('|');
  return {
    name: `dag-${f}`,
    severity: 'error',
    comment:
      `features/${f} may import [${graph.features[f].deps.join(', ') || 'nothing'}].` +
      `Source: scripts/feature-graph.json.`,
    from: { path: `${W}/features/${f}/` },
    to: { path: `${W}/features/([^/]+)/`, pathNot: `${W}/features/(${allowed})/` },
  };
});

// no-deep-feature-import is generated per feature too, rather than relying on a
// $1 back-reference in to.pathNot – the back-reference form is version-sensitive
// and this costs eleven objects and zero coverage.
const deepImportRules = FEATURES.map((f) => ({
  name: `no-deep-import-${f}`,
  severity: 'error',
  comment: 'A feature is reached through index.ts (client-safe) or server.ts (server-only). Nothing else is public.',
  from: { path: `${W}/(features|app|shared)/`, pathNot: `${W}/features/${f}/` },
  to: {
    path: `${W}/features/${f}/.+`,
    pathNot: `${W}/features/${f}/(index|server)\\.ts$`,
  },
}));

module.exports = {
  forbidden: [
    ...dagRules,
    ...deepImportRules,

    {
      name: 'client-never-touches-server',
      severity: 'error',
      comment: 'ui/ and hooks/ ship to the browser.',
      from: { path: `${W}/features/[^/]+/(ui|hooks)/` },
      to: { path: [`${W}/features/[^/]+/server`, `${W}/shared/db`, `^packages/db/src/(server|service)`] },
    },

    // =====
    // THE SERVICE-KEY FIREWALL. `reachable: true` is the whole point:
    // the leak that matters is TRANSITIVE. A client component importing a
    // feature barrel that re-exports a repo that constructs the service
    // client is invisible to a direct-edge rule.
    // =====
    {
      name: 'service-key-firewall',
      severity: 'error',
      comment: 'The service role is reachable from exactly two places: a feature server/ folder, and the pipeline.',
      from: { pathNot: graph.serviceKeyReachableFrom.join('|') },
      to: { path: `^packages/db/src/service\\.ts$`, reachable: true },
    },
    {
      name: 'client-entry-never-reaches-service-key',
      severity: 'error',
      comment: 'Transitive reachability from anything that ships to a browser.',
      from: { path: [`${W}/app/.+\\.tsx$`, `${W}/features/[^/]+/(ui|hooks)/`] },
      to: { path: `^packages/db/src/service\\.ts$`, reachable: true },
    },

    {
      name: 'model-is-pure',
      severity: 'error',
      from: { path: '(apps/web/src/features|apps/pipeline/src/stages)/[^/]+/model/' },
      to: {
        path: [
          `^node_modules/(react|react-dom|next|@supabase|@privy-io|@solana)`,
          '/server/', '/shared/db', '/shared/realtime', '/shared/rpc',
        ],
      },
    },
    {
      name: 'app-has-no-logic',
```

Two operational notes on that config, both learned the expensive way.

Exit 0 is not evidence. dependency-cruiser declares support for `typescript ≥2 <7`. With a TypeScript major it does not support, it prints an advisory to stderr, reports *"no dependency violations found (0 modules, 0 dependencies cruised)"*, and **exits 0**. The gate goes green having inspected nothing. So the CI step asserts a floor on the module count, not just the exit code:

```
npx depcruise apps/web/src packages --config .dependency-cruiser.cjs --output-type json \
| tee /tmp/dc.json \
| jq -e '.summary.totalCruised > 60' > /dev/null \
|| { echo '::error::dependency-cruiser cruised almost nothing — check the TypeScript version'; exit 1; }
```

The **required rule is a convention check, not the firewall**. It asserts a property of files literally named `features/*/server.ts` and says nothing about `features/x/repo.ts` or any other new file that touches the key. The firewall is `service-key-firewall` with `reachable: true`, which is why both exist.

Layer 4 — server-only, at build time. A `'use client'` file that imports `@server/feed` fails the Vercel build outright. Three characters that turn a lint bypass into a failed deployment rather than a service-role key in a JS chunk.

Layer 5 — a test, because the server-only import must actually be there. No custom ESLint plugin: the repository already ships vitest, and the position of the import does not matter (ES modules hoist, so any top-level import throws).

```
// apps/web/src/__tests__/server-entries.test.ts
import { readFileSync } from 'node:fs';
import { globSync } from 'node:fs';
import { expect, test } from 'vitest';

test('INVARIANT every feature server entry imports server-only', () => {
  const entries = globSync('src/features/*/server.ts', { cwd: process.cwd() });
  expect(entries.length).toBeGreaterThan(8);
  for (const f of entries) {
    expect(readFileSync(f, 'utf8'), f).toMatch(/^\\s*import \["\"server-only\["\"\\s*$/m);
  }
});
```

What a reach-past looks like, in firing order

1. IN THE EDITOR (TypeScript path mapping)
`import { PostRow } from '@features/feed/ui/posts/PostRow';`
→ `ts(2307) Cannot find module '@features/feed/ui/posts/PostRow'.`
No alias resolves a feature internal and there is no `@/*` to fall back to.
This is the layer that changes behaviour, because it fires while you type.
2. ON SAVE (ESLint), for the relative-path workaround
`import { PostRow } from '../../../feed/ui/posts/PostRow';`
→ `error This escapes the feature folder. Import the other feature through @features/<name>.` `no-restricted-imports`
3. IN CI (dependency-cruiser), for everything else including dynamic `import()`
`error no-deep-import-feed: features/search/ui/Row.tsx`
→ `features/feed/ui/posts/PostRow.tsx`
`error dag-search: features/search may import [coins, stories, watchlist].`
Source: `scripts/feature-graph.json`.
and `docs/architecture.svg` changes, so the PR shows a new arrow between two boxes.
4. AT BUILD (server-only), for the one that costs real money
`import { getPostsBoard } from '@server/feed';` // inside a `'use client'` file
→ `Error: You're importing a component that needs "server-only".`
A lint bypass becomes a failed build instead of a service key in a JS chunk.

The shared/ graduation rule

Something moves from a feature into `shared/` only when **all three** hold:

1. **Three call sites in three different features.** Not three call sites in one.
2. **Zero domain vocabulary.** If a prop is named `coin`, `mint`, `story` or `ticker`, it is not shared.
3. **You would publish it to npm** without embarrassment and without explaining Insidor.

Until all three hold, duplicate the code. Two copies of a twenty-line component are cheaper than one wrong abstraction in `shared/` that four features bend around.

Promotion is a PR titled `promote: <thing> → shared/<module>` that **deletes both originals in the same commit**. Demotion is the half people forget: anything in `shared/` with one importer for thirty days moves back into that importer. `npm run depcruise:graph` makes single-importer shared modules visible.

The five domain exceptions — `stamp`, `lead`, `staleness`, `fee`, `format/pending.tsx` — are exempt *because* they are the contracts. Where a component renders differently in two places, the product contradicts itself, and on this product

every contradiction is about money or about a claim. Putting them in a feature would mean six features importing a seventh to render a stamp.

Wallet context lives in `features/wallet/`, **not** `shared/`. A `WalletContext` in `shared/` means `shared/` imports Privy, which means `shared/` has business rules, which means the next thing goes in there too.

3. THE DATABASE

One Postgres, one schema (`public`), 46 tables, plus an `insidor` schema holding the migration framework and the trigger functions. Everything the browser can see is governed by RLS and column-level grants; there is no second schema and no PostgREST exposure trick, because the grants already do that job with one fewer moving part.

Naming, decided once so nobody has to remember a list: tables are singular, enums are singular, columns are snake_case , every foreign key to a story is story_id . The route is `/story/[id]` , every surface's copy says story, and the design document calls the story the spine. `narrative` is vocabulary from a codebase being deleted; carrying it forward means every developer translates between the URL and the schema forever. A lint rule bans the identifier in both apps.

3.1 The five product rules the schema enforces

These are the rules that cost money when they are wrong. Each is enforced by a constraint, a generated column, a type, or a foreign key — never by convention.

#	RULE	MECHANISM	FILE
1	A CONFIRMED match cannot exist without the evidence that justifies it	<code>CHECK coin_match_confirmed_requires_evidence</code> — <code>score ≥ 0.80</code> , <code>ticker_source <> 'llm'</code> , <code>mint_time NOT NULL</code> , <code>relation <> 'mentioned'</code> , and either near-proof or two strong channels	0004
2	A ticker without provenance cannot be inserted at all	<code>story_ticker.source ticker_source NOT NULL</code> with no default , plus <code>CHECK (source <> 'llm' OR resolved_mint IS NULL)</code> . The table carries no market columns at all	0004
3	A tier- <code>never</code> story is unreadable, and a launch against a non- <code>normal</code> story is a foreign-key violation	<code>story.display_eligible</code> GENERATED + the RLS policy reading it; <code>launch.story_coinability_tier</code> with a composite FK to <code>story(id, coinability_tier)</code> <code>ON UPDATE RESTRICT</code> and <code>CHECK (= 'normal')</code>	0002, 0008, 0011
4	<code>promoted_at</code> is set once and never moves	<code>insidor.freeze_columns('promoted_at')</code> BEFORE UPDATE trigger	0002
5	A lead time cannot exist without both clock readings that produced it	<code>story_clock.lead_time_min</code> is GENERATED; both sensor watermarks are <code>NOT NULL</code> and must be ≥ <code>promoted_at</code> ; <code>promoted_at</code> is overwritten at INSERT by a trigger reading <code>story.promoted_at</code>	0005

Four supporting rules are enforced the same way and belong in the same list, because each of them corresponds to a defect that shipped:

- **Unknown authority fails closed.** `coin_safety` has *no boolean anywhere* for an authority. `authority_state NOT NULL DEFAULT 'unknown'` with only the literal `'revoked'` passing the generated `intrinsic_gate_pass` . The old `api/safety.js` wrote `x == null ? true` against an endpoint with no authority fields and reported every token safe. That write now has nowhere to land.
- **Unknown age is not zero.** `coin.minted_at` is nullable with `CHECK ((minted_at IS NULL) = (minted_at_source = 'unknown'))` — if we claim to know when it was minted, we must say how we know — and it is write-once.
- **A top-10 figure that includes the bonding-curve PDA is not stored.** `CHECK (top10_pct IS NULL OR curve_pda_excluded)` . That figure reads 80–95% on every healthy pre-graduation token.
- **A "peak" from one observation is a last-observed value with a flattering label.** `story_outcome` refuses `hit / dud` below six samples of `coin_mcap_series` .

3.2 A note on where Postgres enforces immutability, and where it does not

This matters because the schema leans on it and the enforcement is **asymmetric**.

- **Generated columns are checked at DDL time.** `cookDefault()` calls `contain_mutable_functions_after_planning()` and raises `generation expression is not immutable` . A STABLE function in a generated column is a migration that fails on first apply.
- **CHECK constraints are not checked at all.** Postgres *assumes* immutability and documents the consequence: *"PostgreSQL does not disallow that, but it will not notice if there are rows in the table that now violate the CHECK constraint. That would cause a subsequent database dump and restore to fail."* `CHECK (x ≤ now())` applies green and breaks a restore months later.

So every CHECK in this schema is written immutable-safe as a **convention**, and the convention is backed by two mechanical checks rather than by discipline: a **dump→restore round trip** in CI (see §8), and an assertion in

supabase/tests/invariants.sql scanning pg_constraint for functions whose pg_proc.provolatile < 'i'.

Three concrete traps that follow from this, all of which the DDL below already avoids:

1. `extract(field FROM timestampz)` is **STABLE** and is rejected in a generated column. `extract(field FROM interval)` is immutable. The subtraction must stay *inside* the extract: `extract(epoch FROM (a - b))` works because `timestampz - timestampz` yields `interval`. The obvious-looking simplification `extract(epoch FROM t_crypto)` fails at migration time.
2. `array_to_string` is **STABLE**, permanently and deliberately — it invokes the element type's output function, which may be timezone-dependent. It cannot appear in a generated column. `0009` therefore declares a typed immutable wrapper first:

```
sql -- text elements use textout, which IS immutable, so this wrapper is sound -- for text[] and
only for text[]. `entity_keys::text` is NOT a shortcut -- that CoerceViaIO resolves to array_out,
also STABLE, and fails identically. CREATE OR REPLACE FUNCTION insidor.text_array_to_string(text[],
text) RETURNS text LANGUAGE sql IMMUTABLE PARALLEL SAFE STRICT AS $$ SELECT array_to_string($1, $2)
$$;
```

Once a generated column depends on it, the wrapper is frozen: redefining it later will not re-validate stored rows. 3.

`to_tsvector(text, text)` is **STABLE**; `to_tsvector(regconfig, text)` is **immutable**. The `'english'::regconfig` cast in `0009` is load-bearing.

One arithmetic detail with a product consequence. `lead_time_min` is `round(extract(epoch FROM (...)) / 60.0)::int`. Since PG14, `extract` returns `numeric`, and `round(numeric)` breaks ties away from zero, while `round(double precision)` is round-half-to-even. A −30-second lead rounds to `-1` under the numeric path and `0` under the float path — and `lead_time_min < 0` is the LATE predicate on the public record. Pin local development to the deployed Postgres major, or the same clock readings publish two different verdicts.

3.3 The enums

Declaration order is load-bearing where an enum is ordered — but note that no generated column in this schema uses an ordered enum comparison. Ordered gates encode a product rule in declaration order, where `ALTER TYPE ... ADD VALUE ... BEFORE` changes which future labels pass, silently and with no recompute. Gates use explicit equality and fail closed.

```

-- 0001_extensions_and_types.sql

CREATE EXTENSION IF NOT EXISTS pgcrypto WITH SCHEMA public;
CREATE EXTENSION IF NOT EXISTS pg_trgm WITH SCHEMA public;
CREATE EXTENSION IF NOT EXISTS vector WITH SCHEMA public; -- halfvec + HNSW
CREATE EXTENSION IF NOT EXISTS btree_gin WITH SCHEMA public;


-- pg_cron and pg_net need shared_preload_libraries. Optional here, asserted
-- present by the pipeline's boot check, so a missing one is loud in production
-- and quiet in CI rather than the reverse.
DO $$
BEGIN
    BEGIN CREATE EXTENSION IF NOT EXISTS pg_cron; EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'pg_cron unavailable: scheduled jobs must be installed separately'; END;
    BEGIN CREATE EXTENSION IF NOT EXISTS pg_net; EXCEPTION WHEN OTHERS THEN
        RAISE NOTICE 'pg_net unavailable: the in-database health observer is not installed'; END;
END $$;


SELECT insidor.create_enum('platform', ARRAY['x','tiktok']);
SELECT insidor.create_enum('story_status', ARRAY['provisional','open','dormant','closed','merged']);
SELECT insidor.create_enum('lifecycle', ARRAY['heating','peaking','cooling','dead']);
SELECT insidor.create_enum('coinability_tier', ARRAY['never','no_create','normal']);
-- The provenance column whose absence produced the $KANG Buy button.
SELECT insidor.create_enum('ticker_source', ARRAY['cashtag','mint_in_post','llm','insidor_launch']);
-- verdict is CONFIDENCE. relation is PROVENANCE. Conflating them is how an
-- adopted coin got sold as derived; an adopted coin can legitimately be unsure.
SELECT insidor.create_enum('match_verdict', ARRAY['confirmed','unsure','rejected']);
SELECT insidor.create_enum('match_relation', ARRAY['derived','adopted','mentioned']);
-- Three-valued, never boolean. There is no `mint_revoked boolean` in this
-- schema, so the api/safety.js defect is unwritable.
SELECT insidor.create_enum('authority_state', ARRAY['unknown','revoked','active']);
SELECT insidor.create_enum('route_state', ARRAY['unknown','ok','absent']);
SELECT insidor.create_enum('coin_band', ARRAY['fresh','graduating','live']);
SELECT insidor.create_enum('board_lane', ARRAY['posts','coins']);
SELECT insidor.create_enum('comment_status', ARRAY['visible','held','blocked','removed']);
SELECT insidor.create_enum('position_state', ARRAY['holds','sold','none','unknown']);
SELECT insidor.create_enum('story_outcome_kind', ARRAY['hit','dud','no_coin','late','unmeasurable']);
-- ONE table, ONE enum, so shipping gap detection for the mint stream and
-- forgetting it for the CT poller is impossible.
SELECT insidor.create_enum('sensor', ARRAY['mint_stream','ct_poll','ingest_x','ingest_tiktok','snapshotter']);
SELECT insidor.create_enum('notification_kind', ARRAY['mint','window_closing','accelerating','correction']);
SELECT insidor.create_enum('trade_side', ARRAY['buy','sell']);
SELECT insidor.create_enum('trade_status', ARRAY['ordering','signed','submitted','confirmed','failed','expired','cancelled']);
SELECT insidor.create_enum('media_kind', ARRAY['image','video','gif']);
SELECT insidor.create_enum('wallet_kind', ARRAY['embedded','external']);

```

Four trigger functions carry the write-once semantics the whole schema depends on:

```

-- Once a listed column is non-NULL it can never change. Convention beats
-- nothing; a trigger beats convention. Dynamic EXECUTE per column per row, so
-- it is used only on low-rate tables (story, coin, story_clock, comment,
-- holding, trade, launch, fee_share, app_user) and never on post or
-- coin_snapshot.

CREATE OR REPLACE FUNCTION insidor.freeze_columns() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE col text; oldv text; newv text;
BEGIN
    FOREACH col IN ARRAY TG_ARGV LOOP
        EXECUTE format('SELECT ($1).%I::text', col) INTO oldv USING OLD;
        EXECUTE format('SELECT ($1).%I::text', col) INTO newv USING NEW;
        IF oldv IS NOT NULL AND newv IS DISTINCT FROM oldv THEN
            RAISE EXCEPTION '%.% is write-once: it was set to % and cannot be changed to %',
                TG_TABLE_NAME, col, oldv, COALESCE(newv, 'NULL');
        END IF;
    END LOOP;
    RETURN NEW;
END $$;

-- Rejects the known posted_at double-conversion bug at the door, rather than
-- letting one future-dated row render "+31m early" on a board dead for two days.

CREATE OR REPLACE FUNCTION insidor.reject_future_timestamp() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE col text; v timestamptz;
BEGIN
    FOREACH col IN ARRAY TG_ARGV LOOP
        EXECUTE format('SELECT ($1).%I', col) INTO v USING NEW;
        IF v IS NOT NULL AND v > now() + interval '5 minutes' THEN
            RAISE EXCEPTION '%.% is % which is in the future; refusing the row', TG_TABLE_NAME, col, v;
        END IF;
    END LOOP;
    RETURN NEW;
END $$;

CREATE OR REPLACE FUNCTION insidor.touch_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$ BEGIN NEW.updated_at := now(); RETURN NEW; END $$;

-- base58 run detector for the comment address gate. Deliberately in the
-- database: the client check is a courtesy, the server route is the policy,
-- this is the enforcement that survives both being wrong.

CREATE OR REPLACE FUNCTION insidor.base58_runs(p_text text)
RETURNS text[] LANGUAGE sql IMMUTABLE AS $$
    SELECT COALESCE(array_agg(m[1]), ARRAY[]::text[])
    FROM regexp_matches(COALESCE(p_text, ''), '([1-9A-HJ-NP-Za-km-z]{32,44})', 'g') m
$$;

```

3.4 story — where coinability, display eligibility and promote immutability live

```
CREATE TABLE IF NOT EXISTS public.story (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  status story_status NOT NULL DEFAULT 'provisional',
  title text, title_version integer NOT NULL DEFAULT 0, titled_at timestamptz,
  title_centroid halfvec(768), blurb text, subject text, subject_type text,
  entity_keys text[] NOT NULL DEFAULT '{}',
  cashtags text[] NOT NULL DEFAULT '{}',
  platforms platform[] NOT NULL DEFAULT '{}',

  first_post_at timestamptz, -- the author's clock
  first_seen_at timestamptz NOT NULL DEFAULT now(), -- ours
  created_at timestamptz NOT NULL DEFAULT now(),

  -- THE lead-time clock. Set once at PROMOTE, frozen by the trigger below.
  promoted_at timestamptz,
  promoted_at_backfilled boolean NOT NULL DEFAULT false,
  censored_entry boolean NOT NULL DEFAULT false,

  -- The LEAST permissive tier is the default: an unclassified story is
  -- invisible until the classifier explicitly relaxes it. Fail closed.
  coinability_tier coinability_tier NOT NULL DEFAULT 'never',
  coinability_reason text, coinability_model text, coinability_confidence real,

  needs_review boolean NOT NULL DEFAULT false, coherence real,
  centroid halfvec(768), centroid_n real NOT NULL DEFAULT 0,
  exemplar_post_ids uuid[] NOT NULL DEFAULT '{}',

  n_posts integer NOT NULL DEFAULT 0, distinct_authors integer NOT NULL DEFAULT 0,
  combined_views bigint NOT NULL DEFAULT 0,
  lifecycle lifecycle, lifecycle_reason text,
  heat double precision, presented_heat smallint, heat_rank integer,
  top_post_id uuid,

  merged_into_id uuid REFERENCES public.story(id) ON DELETE SET NULL,
  merged_at timestamptz, merge_count_hour smallint NOT NULL DEFAULT 0,

  -- Maintained by trigger from coin_match. The primary button is a pure
  -- function of confirmed_coin_count and nothing else.
  confirmed_coin_count smallint NOT NULL DEFAULT 0,
  unsure_coin_count smallint NOT NULL DEFAULT 0,
  updated_at timestamptz NOT NULL DEFAULT now(),

  earliness_eligible boolean GENERATED ALWAYS AS (
    promoted_at IS NOT NULL AND NOT promoted_at_backfilled AND NOT censored_entry) STORED,

  -- A tier-`never` story is not "hidden by a filter". It is unreadable: the
  -- RLS policy in 0011 reads this column.
  display_eligible boolean GENERATED ALWAYS AS (
    promoted_at IS NOT NULL AND coinability_tier <> 'never'
    AND status IN ('open','dormant','closed')) STORED,

  can_create boolean GENERATED ALWAYS AS (
    coinability_tier = 'normal' AND status IN ('open','dormant','closed')
    AND needs_review = false AND promoted_at IS NOT NULL) STORED
);

-- The composite key that lets child tables carry the tier and be constrained on
-- it. This is how "a story surfacing when its coinability forbids it" becomes a
-- foreign-key violation instead of a code review comment.
SELECT insidor.add_constraint('public.story','story_id_tier_key','UNIQUE (id, coinability_tier)');

SELECT insidor.add_constraint('public.story','story_merged_needs_target',
  $$CHECK (status <> 'merged' OR (merged_into_id IS NOT NULL AND merged_at IS NOT NULL))$$);
SELECT insidor.add_constraint('public.story','story_no_self_merge',
  $$CHECK (merged_into_id IS DISTINCT FROM id)$$);
SELECT insidor.add_constraint('public.story','story_promote_after_first_post',
  $$CHECK (promoted_at IS NULL OR first_post_at IS NULL
    OR promoted_at ≥ first_post_at - interval '2 minutes')$$);
SELECT insidor.add_constraint('public.story','story_promoted_not_provisional',
  $$CHECK (promoted_at IS NULL OR status <> 'provisional')$$);
SELECT insidor.add_constraint('public.story','story_tier_needs_reason',
  $$CHECK (coinability_tier = 'never' OR coinability_reason IS NOT NULL)$$);
SELECT insidor.add_constraint('public.story','story_counts_nonneg',
  $$CHECK (confirmed_coin_count ≥ 0 AND unsure_coin_count ≥ 0)$$);

-- RULE 4. The single most load-bearing trigger in the schema.
DROP TRIGGER IF EXISTS story_freeze_promoted_at ON public.story;
CREATE TRIGGER story_freeze_promoted_at BEFORE UPDATE ON public.story
  FOR EACH ROW EXECUTE FUNCTION insidor.freeze_columns('promoted_at');
DROP TRIGGER IF EXISTS story_touch ON public.story;
CREATE TRIGGER story_touch BEFORE UPDATE ON public.story
```

```
FOR EACH ROW EXECUTE FUNCTION insidor.touch_updated_at();

CREATE INDEX IF NOT EXISTS story_board_idx ON public.story (heat DESC NULLS LAST) WHERE display_eligible;
CREATE INDEX IF NOT EXISTS story_no_coin_idx ON public.story (heat DESC NULLS LAST)
    WHERE display_eligible AND confirmed_coin_count = 0 AND can_create; -- "No coin yet"
CREATE INDEX IF NOT EXISTS story_promoted_at_idx ON public.story (promoted_at) WHERE promoted_at IS NOT NULL;
CREATE INDEX IF NOT EXISTS story_merged_into_idx ON public.story (merged_into_id) WHERE merged_into_id IS NOT NULL;
CREATE INDEX IF NOT EXISTS story_status_idx ON public.story (status);

-- Deliberately absent, and why:
-- search_series a fabricated sine wave that drove search ordering
-- gain_24h      combined_views * 0.14, an invention
-- lead_time_min derived, and only from story_clock where both clocks exist
-- age_min, accel, trend_*, img_seed, narr_idx, bought_reach
```

3.5 post and the heat tables

```
CREATE TABLE IF NOT EXISTS public.post (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  story_id    uuid REFERENCES public.story(id) ON DELETE SET NULL,
  platform    platform NOT NULL,
  platform_post_id  text NOT NULL,

  author_handle  text NOT NULL,
  author_name    text,
  author_id      text,
  author_followers  integer,  -- current, display only
  -- The plausibility denominator, kept SEPARATE: `followers` grew BECAUSE of
  -- the post, so scoring reach against it is circular.
  followers_at_post  integer,

  posted_at      timestampz NOT NULL,  -- timestampz, not bigint ms
  first_seen_at  timestampz NOT NULL DEFAULT now(),

  text           text NOT NULL DEFAULT '',
  lang           text,
  sample_replies jsonb NOT NULL DEFAULT '[]'::jsonb,

  media_url text, media_kind media_kind, media_sha256 bytea, media_phash bit(64),
  media_local_url text, ocr_text text,
  derived_subject  text,  -- rendered under a DERIVED label only

  entity_keys      text[] NOT NULL DEFAULT '{}',
  cashtags         text[] NOT NULL DEFAULT '{}',
  mints_in_text    text[] NOT NULL DEFAULT '{}',  -- S_mint_in_post, the strongest channel

  -- Latest observed counters. Every one is NULLABLE: absent is not zero.
  views bigint, likes integer, reposts integer, replies integer,
  quotes integer, bookmarks integer,
  unavailable boolean NOT NULL DEFAULT false,
  second_wave boolean NOT NULL DEFAULT false,
  censored_entry boolean NOT NULL DEFAULT false,
  updated_at timestampz NOT NULL DEFAULT now()
);
SELECT insidor.add_constraint('public.post','post_platform_key','UNIQUE (platform, platform_post_id)');

DROP TRIGGER IF EXISTS post_reject_future ON public.post;
CREATE TRIGGER post_reject_future BEFORE INSERT OR UPDATE ON public.post
  FOR EACH ROW EXECUTE FUNCTION insidor.reject_future_timestamp('posted_at');

CREATE INDEX IF NOT EXISTS post_story_views_idx ON public.post (story_id, views DESC NULLS LAST);
CREATE INDEX IF NOT EXISTS post_recent_idx      ON public.post (posted_at DESC);

-- post_snapshot: PK (post_id, captured_at), plus the DESC index the old build
-- never had.
CREATE INDEX IF NOT EXISTS post_snapshot_post_time_idx
  ON public.post_snapshot (post_id, captured_at DESC);

-- post_heat. ACCEL renders an em dash on a single snapshot, never a
-- fabricated 1.0x.
SELECT insidor.add_constraint('public.post_heat','post_heat_burst_needs_ticks',
  $$CHECK (burst IS NULL OR ticks_qualified ≥ 2)$$);

-- story_momentum is a TABLE in five-minute buckets carrying member_count, NOT a
-- matview over raw snapshots: posts sample on a per-post geometric grid, so
-- summing by minute makes a story's total DROP when fewer posts happened to
-- snapshot, and COOLING then fires on a sampling artefact.
SELECT insidor.add_constraint('public.story_momentum','story_momentum_aligned',
  $$CHECK ((extract(epoch FROM (bucket_at - timestampz 'epoch'))::bigint % 300) = 0)$$);
```

post_meme_score , post_embedding (halfvec(768) + HNSW), platform_norm and author_roster complete 0002 . post_embedding.model is load-bearing: gemini-embedding-001 and -2 spaces are documented as incompatible. There is no constraint enforcing a single model across the corpus, because a re-embed necessarily has both present transiently — so the pipeline asserts it instead, at the start of every retrieval run.

3.6 coin, coin_snapshot, coin_mcap_series, coin_safety

```
CREATE TABLE IF NOT EXISTS public.coin (
  mint text PRIMARY KEY,
  symbol text, name text, decimals smallint, token_program text,
  logo_url text, logo_sha256 bytea, logo_phash bit(64), creator_wallet text,
  -- NULL is legal and meaningful: unknown age. Never 0.
  minted_at timestamptz,
  minted_at_source text NOT NULL DEFAULT 'unknown',
  launchpad text, curve_pct real, migrated boolean NOT NULL DEFAULT false,
  pool_address text, socials jsonb NOT NULL DEFAULT '{} '::jsonb,
  metadata_twitter text,          -- attacker-controlled; $social caps at 0.85
  first_seen_at timestamptz NOT NULL DEFAULT now(),
  is_insidor_launch boolean NOT NULL DEFAULT false,
  is_major boolean NOT NULL DEFAULT false,  -- excluded from DERIVED
  updated_at timestamptz NOT NULL DEFAULT now(),

  symbol_norm text GENERATED ALWAYS AS (
    upper(regexp_replace(COALESCE(symbol,''), '[^A-Za-z0-9]', '', 'g')))) STORED,

  -- `band` is generated from row-local facts only. The Fresh 6h ceiling depends
  -- on now() and therefore CANNOT live here — it is an eligibility filter in the
  -- board query, not part of the band. A non-pump.fun launchpad has NULL
  -- curve_pct and lands in 'fresh' until a pool exists, never nowhere.
  band coin_band GENERATED ALWAYS AS (
    CASE WHEN migrated OR pool_address IS NOT NULL      THEN 'live'::coin_band
         WHEN curve_pct IS NOT NULL AND curve_pct ≥ 50 THEN 'graduating'::coin_band
         ELSE 'fresh'::coin_band END) STORED
);

SELECT insidor.add_constraint('public.coin','coin_mint_base58',
  $$CHECK (mint ~ '^[1-9A-HJ-NP-Za-km-z]{32,44}$')$$);
SELECT insidor.add_constraint('public.coin','coin_symbol_len',
  $$CHECK (symbol IS NULL OR length(symbol) ≤ 13)$$);  -- create_v2 cap is 13
SELECT insidor.add_constraint('public.coin','coin_minted_at_source_enum',
  $$CHECK (minted_at_source IN ('pumpportal_create','pair_min','launch','unknown'))$$);
-- If we claim to know when it was minted, we must say how we know.
SELECT insidor.add_constraint('public.coin','coin_minted_at_needs_source',
  $$CHECK ((minted_at IS NULL) = (minted_at_source = 'unknown'))$$);
SELECT insidor.add_constraint('public.coin','coin_curve_range',
  $$CHECK (curve_pct IS NULL OR curve_pct BETWEEN 0 AND 100)$$);

-- minted_at is write-once. Precedence is decided at write time: PumpPortal create
-- event > min(pairCreatedAt) across ALL Solana pairs > NULL. Letting a later
-- best-pair read overwrite it reintroduces the 418-vs-647-day defect.
DROP TRIGGER IF EXISTS coin_freeze_mint_time ON public.coin;
CREATE TRIGGER coin_freeze_mint_time BEFORE UPDATE ON public.coin
  FOR EACH ROW EXECUTE FUNCTION insidor.freeze_columns('minted_at','creator_wallet');

CREATE INDEX IF NOT EXISTS coin_symbol_norm_idx ON public.coin (symbol_norm);
CREATE INDEX IF NOT EXISTS coin_symbol_trgm_idx ON public.coin USING gin (symbol_norm gin_trgm_ops);
CREATE INDEX IF NOT EXISTS coin_band_minted_idx ON public.coin (band, minted_at DESC NULLS LAST);
CREATE INDEX IF NOT EXISTS coin_creator_idx ON public.coin (creator_wallet) WHERE creator_wallet IS NOT NULL;
CREATE INDEX IF NOT EXISTS coin_logo_sha_idx ON public.coin (logo_sha256) WHERE logo_sha256 IS NOT NULL;

COMMENT ON TABLE public.coin IS
  'There is no age_min column. A stored age scalar is wrong the second after it is written.';
```

`coin_snapshot` is the time series — PK `(mint, captured_at)`, index `(mint, captured_at DESC)`. There was no price column anywhere in the old schema, so every window and the whole CoinHeat formula were uncomputable. **Every numeric is nullable: absent is not zero.**

```
SELECT insidor.add_constraint('public.coin_snapshot','coin_snapshot_price_positive',
  $$CHECK (price_usd IS NULL OR price_usd > 0)$$);  -- $0 claims worthlessness
SELECT insidor.add_constraint('public.coin_snapshot','coin_snapshot_mcap_positive',
  $$CHECK (mcap_usd IS NULL OR mcap_usd > 0)$$);
SELECT insidor.add_constraint('public.coin_snapshot','coin_snapshot_top10_needs_exclusion',
  $$CHECK (top10_pct IS NULL OR curve_pda_excluded)$$);

-- coin_mcap_series: five-minute mcap for 24h, the ONLY source of peak_mcap.
SELECT insidor.add_constraint('public.coin_mcap_series','coin_mcap_series_aligned',
  $$CHECK ((extract(epoch FROM (bucket_at - timestamptz 'epoch'))::bigint % 300) = 0)$$);
```

```
-- =====
-- coin_safety. There is NO boolean anywhere in this table for an authority.
-- =====

CREATE TABLE IF NOT EXISTS public.coin_safety (
  mint text PRIMARY KEY REFERENCES public.coin(mint) ON DELETE CASCADE,
  mint_authority authority_state NOT NULL DEFAULT 'unknown',
  freeze_authority authority_state NOT NULL DEFAULT 'unknown',
  authority_source text, transfer_fee_bps integer, has_transfer_hook boolean,
  sell_route route_state NOT NULL DEFAULT 'unknown', sell_route_checked_at timestamptz,
  top1_pct real, top10_pct real, holder_count integer,
  curve_pda_excluded boolean NOT NULL DEFAULT false,
  lp_locked_pct real, honeypot_proxy boolean,
  signals_checked smallint NOT NULL DEFAULT 0, signals_total smallint NOT NULL DEFAULT 9,
  raw jsonb, checked_at timestamptz NOT NULL DEFAULT now(),

  -- Unknown fails closed BY CONSTRUCTION: only the literal 'revoked'/'ok'
  -- passes. Note `has_transfer_hook = false` and not `IS NOT TRUE` – the latter
  -- would let NULL through, which is precisely the old bug's shape. Liquidity
  -- and band-dependent concentration are applied in the board query, because
  -- they are market data and change every tick.
  intrinsic_gate_pass boolean GENERATED ALWAYS AS (
    mint_authority = 'revoked'
    AND freeze_authority = 'revoked'
    AND sell_route = 'ok'
    AND COALESCE(transfer_fee_bps, 0) = 0
    AND COALESCE(has_transfer_hook, true) = false
    AND top10_pct IS NOT NULL
    AND curve_pda_excluded) STORED
);

SELECT insidor.add_constraint('public.coin_safety','coin_safety_signals',
  $$CHECK (signals_checked BETWEEN 0 AND signals_total)$$);
SELECT insidor.add_constraint('public.coin_safety','coin_safety_top10_needs_exclusion',
  $$CHECK (top10_pct IS NULL OR curve_pda_excluded)$$);
SELECT insidor.add_constraint('public.coin_safety','coin_safety_route_needs_check',
  $$CHECK (sell_route = 'unknown' OR sell_route_checked_at IS NOT NULL)$$);
```

`coin_image_embedding` is `halfvec(512)` + HNSW — a **separate space** from `post_embedding` 's `halfvec(768)` . Mixing them in one column is a silent recall bug. `creator_stat` carries `CHECK (has_verdict = false OR (mint_count ≥ 1 AND first_mint_at IS NOT NULL))` , so a clean tick can never be manufactured by absence of data.

3.7 story_ticker and coin_match — the two tables that cannot touch

`narrative_tickers` held five concepts in one row: a ticker, a mint, market data, a verdict, and a provenance it did not have. That is *the mechanism* by which an LLM-invented ticker got resolved against a live market and rendered a Buy button. It becomes two tables that cannot reach each other.

```
-- =====
-- story_ticker. Candidates and create-path prefills. Deliberately carries NO
-- mcap, liquidity, holders, vol24h, safety, canonical or first_deployed column.
-- Their absence is the fix: removing the columns makes the defect
-- unrepresentable rather than merely discouraged.
-- =====

CREATE TABLE IF NOT EXISTS public.story_ticker (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    story_id    uuid NOT NULL REFERENCES public.story(id) ON DELETE CASCADE,
    ticker      text NOT NULL,
    name        text,
    -- NOT NULL, NO DEFAULT. An INSERT that forgets provenance fails. There is no
    -- value this column can hold that means "we don't know where this came from".
    source      ticker_source NOT NULL,
    source_span text,
    post_id     uuid REFERENCES public.post(id) ON DELETE SET NULL,
    resolved_mint text REFERENCES public.coin(mint) ON DELETE SET NULL,
    score       real,
    created_at  timestampz NOT NULL DEFAULT now()
);

SELECT insidor.add_constraint('public.story_ticker', 'story_ticker_shape',
    $$CHECK (ticker ~ '^[A-Z][A-Z0-9]{1,12}$')$$);
-- THE constraint. The $KANG defect, expressed as an impossibility.
SELECT insidor.add_constraint('public.story_ticker', 'story_ticker_llm_never_resolves',
    $$CHECK (source <> 'llm' OR resolved_mint IS NULL)$$);
-- An ungrounded LLM suggestion cannot even be stored.
SELECT insidor.add_constraint('public.story_ticker', 'story_ticker_llm_needs_span',
    $$CHECK (source <> 'llm' OR source_span IS NOT NULL)$$);
-- A cashtag or mint-in-post claim must name the post it came from.
SELECT insidor.add_constraint('public.story_ticker', 'story_ticker_evidence_needs_post',
    $$CHECK (source NOT IN ('cashtag','mint_in_post') OR post_id IS NOT NULL)$$);
SELECT insidor.add_constraint('public.story_ticker', 'story_ticker_launch_has_mint',
    $$CHECK (source <> 'insidor_launch' OR resolved_mint IS NOT NULL)$$);
SELECT insidor.add_constraint('public.story_ticker', 'story_ticker_unique',
    'UNIQUE (story_id, ticker, source)');

CREATE INDEX IF NOT EXISTS story_ticker_story_idx ON public.story_ticker (story_id);
-- The create-form prefill reads only non-'llm' sources.
CREATE INDEX IF NOT EXISTS story_ticker_prefill_idx
    ON public.story_ticker (ticker) WHERE source IN ('cashtag','mint_in_post');
CREATE INDEX IF NOT EXISTS story_ticker_trgm_idx
    ON public.story_ticker USING gin (ticker gin_trgm_ops);
```

```
-- =====
-- coin_match. One row per (story, mint). The only input to the primary button.
-- =====

CREATE TABLE IF NOT EXISTS public.coin_match (
    id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    story_id          uuid NOT NULL REFERENCES public.story(id) ON DELETE CASCADE,
    mint              text NOT NULL REFERENCES public.coin(mint) ON DELETE CASCADE,
    verdict           match_verdict NOT NULL,
    relation          match_relation NOT NULL,
    -- Denormalised at match time: the pipeline truncates and re-inserts ticker
    -- rows every cycle, so a page load racing a cycle would render
    -- "Buy $undefined". The button never reads a table that can vanish.
    ticker            text NOT NULL,
    name              text,
    ticker_source     ticker_source NOT NULL,
    score             real NOT NULL,
    -- NULL means NOT CHECKED, rendered "not checked", never "failed", never 0.
    s_mint_in_post    real,
    s_img             real,
    s_text            real,
    s_tick            real,
    s_social          real,
    time_prior        real NOT NULL,
    market_plausibility real,
    mint_time         timestampz,
    earliest_post_at  timestampz NOT NULL,
    matcher_version   text NOT NULL,
    evidence          jsonb NOT NULL DEFAULT '{}':jsonb, -- server-only
    evidence_public   jsonb NOT NULL DEFAULT '{}':jsonb, -- categorical prose
    matched_at        timestampz NOT NULL DEFAULT now(),
    retracted_at      timestampz,

    channels_ran smallint GENERATED ALWAYS AS (
        (s_mint_in_post IS NOT NULL)::int + (s_img IS NOT NULL)::int
        + (s_text IS NOT NULL)::int + (s_tick IS NOT NULL)::int
        + (s_social IS NOT NULL)::int
    ) STORED,
    strong_channels smallint GENERATED ALWAYS AS (
        (COALESCE(s_img, 0) ≥ 0.55)::int
        + (COALESCE(s_text, 0) ≥ 0.55)::int
        + (COALESCE(s_tick, 0) ≥ 0.60)::int
    ) STORED,
    -- The subtraction stays INSIDE the extract: timestampz - timestampz yields
    -- interval, and extract(text, interval) is immutable. extract from a bare
    -- timestampz is STABLE and would be rejected here.
    delta_min integer GENERATED ALWAYS AS (
        CASE WHEN mint_time IS NULL THEN NULL
        ELSE round(extract(epoch FROM (mint_time - earliest_post_at)) / 60.0)::int END
    ) STORED
);

SELECT insidor.add_constraint('public.coin_match', 'coin_match_pair_key',
    'UNIQUE (story_id, mint)');
SELECT insidor.add_constraint('public.coin_match', 'coin_match_score_range',
    '$$CHECK (score BETWEEN 0 AND 1)$$');
SELECT insidor.add_constraint('public.coin_match', 'coin_match_channel_ranges',
    '$$CHECK ((s_mint_in_post IS NULL OR s_mint_in_post BETWEEN 0 AND 1)
        AND (s_img IS NULL OR s_img BETWEEN 0 AND 1)
        AND (s_text IS NULL OR s_text BETWEEN 0 AND 1)
        AND (s_tick IS NULL OR s_tick BETWEEN 0 AND 1)
        -- capped: the metadata twitter field is attacker-controlled and must
        -- never be able to confirm alone.
        AND (s_social IS NULL OR s_social BETWEEN 0 AND 0.85))$$');
SELECT insidor.add_constraint('public.coin_match', 'coin_match_ticker_shape',
    '$$CHECK (ticker ~ '^[A-Z][A-Z0-9]{1,12}$')$$');

-- RULE 1. Written out in full rather than calling a function, because a CHECK
-- depending on a user-defined function is a pg_dump/restore ordering hazard and
-- this is the one constraint that must never fail to restore.
SELECT insidor.add_constraint('public.coin_match', 'coin_match_confirmed_requires_evidence',
    '$$CHECK (
        verdict <> 'confirmed' OR (
            score ≥ 0.80
            AND ticker_source <> 'llm' -- an invented ticker can never confirm
            AND mint_time IS NOT NULL -- unknown age fails closed, never "brand new"
            AND relation <> 'mentioned' -- a typed cashtag is candidate generation
            AND (
                COALESCE(s_mint_in_post, 0) ≥ 0.90
                OR ( (COALESCE(s_img, 0) ≥ 0.55)::int
                    + (COALESCE(s_text, 0) ≥ 0.55)::int
                    + (COALESCE(s_tick, 0) ≥ 0.60)::int ) ≥ 2
            )
        )
    )$$');
```

```

    )
)$$);

-- The identical predicate, exported for the matcher's tests and the eval
-- harness. If the two ever disagree, CI catches it.
CREATE OR REPLACE FUNCTION public.confirmable(
    p_mint_in_post real, p_img real, p_text real, p_tick real
) RETURNS boolean LANGUAGE sql IMMUTABLE AS $$
    SELECT COALESCE(p_mint_in_post, 0) ≥ 0.90
        OR ( (COALESCE(p_img, 0) ≥ 0.55)::int
            + (COALESCE(p_text, 0) ≥ 0.55)::int
            + (COALESCE(p_tick, 0) ≥ 0.60)::int ) ≥ 2
$$;

-- Kills the GYATT class: a 604-day-old shell can never be DERIVED. The 10-minute
-- negative tolerance is clock skew; roughly 2.5% of tokens are minted before
-- their referenced post.
SELECT insidor.add_constraint('public.coin_match', 'coin_match_derived_temporal_gate',
$$CHECK (
    relation <> 'derived' OR (
        mint_time IS NOT NULL
        AND mint_time ≥ earliest_post_at - interval '10 minutes'
        AND mint_time ≤ earliest_post_at + interval '48 hours'
    )
)$$);

SELECT insidor.add_constraint('public.coin_match', 'coin_match_adopted_predates',
$$CHECK (relation <> 'adopted'
    OR (mint_time IS NOT NULL AND mint_time < earliest_post_at - interval '10 minutes'))$$);
SELECT insidor.add_constraint('public.coin_match', 'coin_match_mentioned_not_actionable',
    $$CHECK (relation <> 'mentioned' OR verdict = 'unsure')$$);
SELECT insidor.add_constraint('public.coin_match', 'coin_match_retraction',
    $$CHECK (retracted_at IS NULL OR verdict = 'rejected')$$);
SELECT insidor.add_constraint('public.coin_match', 'coin_match_public_evidence_present',
    $$CHECK (verdict = 'rejected'
        OR (jsonb_typeof(evidence_public) = 'object' AND evidence_public <> '{} '::jsonb))$$);

CREATE INDEX IF NOT EXISTS coin_match_story_idx
    ON public.coin_match (story_id, verdict, score DESC) WHERE retracted_at IS NULL;
CREATE INDEX IF NOT EXISTS coin_match_mint_idx
    ON public.coin_match (mint, verdict, score DESC) WHERE retracted_at IS NULL;
CREATE INDEX IF NOT EXISTS coin_match_confirmed_mint_time_idx
    ON public.coin_match (story_id, mint_time)
    WHERE verdict = 'confirmed' AND retracted_at IS NULL;

-- The counters behind the primary button. IDENTITY ONLY: safety is not a term.
-- If it were, a four-minute-old mint with a null mcap would flip the button back
-- to Create and drive a duplicate launch in the exact window this product exists
-- to serve.
CREATE OR REPLACE FUNCTION insidor.recount_story_coins() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE v_story uuid;
BEGIN
    v_story := COALESCE(NEW.story_id, OLD.story_id);
    UPDATE public.story s SET
        confirmed_coin_count = (
            SELECT count(*) FROM public.coin_match m
            WHERE m.story_id = v_story AND m.retracted_at IS NULL
                AND m.verdict = 'confirmed' AND m.mint_time IS NOT NULL AND m.ticker IS NOT NULL),
        unsure_coin_count = (
            SELECT count(*) FROM public.coin_match m
            WHERE m.story_id = v_story AND m.retracted_at IS NULL AND m.verdict = 'unsure')
    WHERE s.id = v_story;
    RETURN NULL;
END $$;

DROP TRIGGER IF EXISTS coin_match_recount ON public.coin_match;
CREATE TRIGGER coin_match_recount
    AFTER INSERT OR UPDATE OR DELETE ON public.coin_match
    FOR EACH ROW EXECUTE FUNCTION insidor.recount_story_coins();

```

`match_label` is the gold set and the eval set growing itself: `UNIQUE (story_id, mint, matcher_version)`, `CHECK ((label IS NULL) = (labelled_at IS NULL))`, and a `stratum` column so precision can be reported per band.

3.8 The clocks — `ct_mention`, `sensor_*`, `story_clock`, `story_outcome`

Clock A — every Solana mint — is `coin itself` (`coin.minted_at` with `minted_at_source = 'pumpportal_create'`). There is no separate `coin_mints` table: two tables holding a mint time is two tables that can disagree.

```
-- Clock B.

CREATE TABLE IF NOT EXISTS public.ct_mention (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  story_id uuid REFERENCES public.story(id) ON DELETE CASCADE,
  mint text REFERENCES public.coin(mint) ON DELETE CASCADE,
  handle text NOT NULL, platform_post_id text NOT NULL,
  posted_at timestampz NOT NULL, observed_at timestampz NOT NULL DEFAULT now(),
  permalink text NOT NULL, matched_on text NOT NULL,
  created_at timestampz NOT NULL DEFAULT now());

SELECT insidor.add_constraint('public.ct_mention','ct_mention_key','UNIQUE (handle, platform_post_id)');
SELECT insidor.add_constraint('public.ct_mention','ct_mention_targets_something',
  $$CHECK (story_id IS NOT NULL OR mint IS NOT NULL)$$);
SELECT insidor.add_constraint('public.ct_mention','ct_mention_permalink_present',
  $$CHECK (length(permalink) > 0)$$);


-- Symmetric by construction: ONE table, ONE enum.
CREATE TABLE IF NOT EXISTS public.sensor_heartbeat (
  sensor sensor PRIMARY KEY,
  last_beat_at timestampz NOT NULL,
  observed_through timestampz NOT NULL,      -- the watermark
  connected_at timestampz,
  detail jsonb NOT NULL DEFAULT '{} '::jsonb);

SELECT insidor.add_constraint('public.sensor_heartbeat','sensor_watermark_not_future',
  $$CHECK (observed_through ≤ last_beat_at + interval '1 minute')$$);


CREATE TABLE IF NOT EXISTS public.sensor_gap (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sensor sensor NOT NULL, started_at timestampz NOT NULL, ended_at timestampz,
  detected_by text NOT NULL, note text, created_at timestampz NOT NULL DEFAULT now());
SELECT insidor.add_constraint('public.sensor_gap','sensor_gap_ordered',
  $$CHECK (ended_at IS NULL OR ended_at > started_at)$$);
```

```
-- =====
-- story_clock. RULE 5. Four mechanisms make "a lead time without both clock
-- readings" a row Postgres will not accept:
--   1. promoted_at is copied by trigger from story.promoted_at, itself
--      write-once; the row cannot exist before PROMOTE.
--   2. Both sensor watermarks are NOT NULL: you cannot record a clock without
--      recording how far that sensor had actually scanned.
--   3. Both watermarks must reach past promoted_at.
--   4. t_crypto and lead_time_min are GENERATED. There is no INSERT or UPDATE
--      that can write a lead time by hand at all.
-- =====

CREATE TABLE IF NOT EXISTS public.story_clock (
    story_id uuid PRIMARY KEY REFERENCES public.story(id) ON DELETE CASCADE,
    promoted_at timestamptz NOT NULL,
    t_mint timestamptz,
    t_mint_mint text REFERENCES public.coin(mint) ON DELETE SET NULL,
    t_ct timestamptz, t_ct_handle text, t_ct_url text,
    t_mint_observed_through timestamptz NOT NULL,
    t_ct_observed_through timestamptz NOT NULL,
    first_post_at timestamptz,
    unmeasurable boolean NOT NULL DEFAULT false,
    unmeasurable_sensor sensor,
    unmeasurable_gap_id bigint REFERENCES public.sensor_gap(id) ON DELETE SET NULL,
    measured_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),

    -- LEAST() is avoided: an explicit CASE is unambiguously immutable.
    t_crypto timestamptz GENERATED ALWAYS AS (
        CASE WHEN t_mint IS NULL THEN t_ct
             WHEN t_ct IS NULL THEN t_mint
             WHEN t_mint < t_ct THEN t_mint ELSE t_ct END) STORED,

    -- The CASE is re-inlined rather than referencing t_crypto, because a
    -- generation expression cannot reference another generated column.
    lead_time_min integer GENERATED ALWAYS AS (
        CASE WHEN t_mint IS NULL AND t_ct IS NULL THEN NULL
             ELSE round(extract(epoch FROM (
                 (CASE WHEN t_mint IS NULL THEN t_ct
                      WHEN t_ct IS NULL THEN t_mint
                      WHEN t_mint < t_ct THEN t_mint ELSE t_ct END) - promoted_at
                )) / 60.0)::int END) STORED
);

SELECT insidor.add_constraint('public.story_clock','story_clock_mint_within_watermark',
    $$CHECK (t_mint IS NULL OR t_mint ≤ t_mint_observed_through)$$);
SELECT insidor.add_constraint('public.story_clock','story_clock_ct_within_watermark',
    $$CHECK (t_ct IS NULL OR t_ct ≤ t_ct_observed_through)$$);
-- THE constraint: both sensors must have scanned past the promote moment. A
-- lead measured against a poller that had not caught up is rejected here, not
-- caught in review.
SELECT insidor.add_constraint('public.story_clock','story_clock_both_sensors_past_promote',
    $$CHECK (t_mint_observed_through ≥ promoted_at AND t_ct_observed_through ≥ promoted_at)$$);
SELECT insidor.add_constraint('public.story_clock','story_clock_unmeasurable_names_sensor',
    $$CHECK (unmeasurable = false OR unmeasurable_sensor IS NOT NULL)$$);
-- The proof travels with the claim.
SELECT insidor.add_constraint('public.story_clock','story_clock_ct_needs_proof',
    $$CHECK (t_ct IS NULL OR (t_ct_handle IS NOT NULL AND t_ct_url IS NOT NULL))$$);
SELECT insidor.add_constraint('public.story_clock','story_clock_mint_needs_mint',
    $$CHECK (t_mint IS NULL OR t_mint_mint IS NOT NULL)$$);

-- A later sighting must never move an already-recorded first sighting.
DROP TRIGGER IF EXISTS story_clock_freeze ON public.story_clock;
CREATE TRIGGER story_clock_freeze BEFORE UPDATE ON public.story_clock
    FOR EACH ROW EXECUTE FUNCTION insidor.freeze_columns('promoted_at','t_mint','t_ct');

-- lead_time_min needs promoted_at in the same row, so it must be duplicated,
-- and duplication invites drift. The trigger makes the story the sole source
-- and refuses the insert if the story has not promoted.
CREATE OR REPLACE FUNCTION insidor.story_clock_copy_promoted_at() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE v_promoted timestamptz;
BEGIN
    SELECT promoted_at INTO v_promoted FROM public.story WHERE id = NEW.story_id;
    IF v_promoted IS NULL THEN
        RAISE EXCEPTION 'story % has no promoted_at; a clock cannot exist before PROMOTE', NEW.story_id;
    END IF;
    NEW.promoted_at := v_promoted;    -- not the caller's value. The story is the source.
    RETURN NEW;
END $$;
DROP TRIGGER IF EXISTS story_clock_promoted_at ON public.story_clock;
CREATE TRIGGER story_clock_promoted_at BEFORE INSERT ON public.story_clock
```

```

FOR EACH ROW EXECUTE FUNCTION insidor.story_clock_copy_promoted_at();

CREATE INDEX IF NOT EXISTS story_clock_lead_idx
  ON public.story_clock (lead_time_min) WHERE unmeasurable = false;
-- "+2h14m and counting": no t_crypto yet.
CREATE INDEX IF NOT EXISTS story_clock_open_idx
  ON public.story_clock (promoted_at DESC) WHERE t_mint IS NULL AND t_ct IS NULL;

```

A note for whoever has to change this later: **if a generated column ever has to become writable, the replacement is a BEFORE INSERT OR UPDATE trigger writing a plain integer column guarded by freeze_columns , not a view.** A view would keep the single-implementation guarantee but break story_clock_lead_idx , which is a partial index on the generated column and the reason lead is sortable everywhere it appears.

```

-- story_outcome. Misses are visible by default.
CREATE TABLE IF NOT EXISTS public.story_outcome (
  story_id uuid PRIMARY KEY REFERENCES public.story(id) ON DELETE CASCADE,
  outcome story_outcome_kind NOT NULL,
  lead_time_min integer,
  first_coin_mint text REFERENCES public.coin(mint) ON DELETE SET NULL,
  peak_mcap numeric(20,2),
  peak_mcap_samples integer NOT NULL DEFAULT 0,
  hit_threshold numeric(20,2),
  lifecycle_reason text,
  classified_at timestamptz NOT NULL DEFAULT now(),
  classifier_version text NOT NULL);

SELECT insidor.add_constraint('public.story_outcome','story_outcome_late_is_negative',
  $$CHECK (outcome <> 'late' OR (lead_time_min IS NOT NULL AND lead_time_min < 0))$$);
SELECT insidor.add_constraint('public.story_outcome','story_outcome_no_coin_has_no_coin',
  $$CHECK (outcome <> 'no_coin' OR first_coin_mint IS NULL)$$);
SELECT insidor.add_constraint('public.story_outcome','story_outcome_hit_dud_need_coin',
  $$CHECK (outcome NOT IN ('hit','dud') OR first_coin_mint IS NOT NULL)$$);
-- A "peak" from one observation is a last-observed value with a flattering
-- label. Six samples is thirty minutes of coin_mcap_series.
SELECT insidor.add_constraint('public.story_outcome','story_outcome_peak_needs_series',
  $$CHECK (outcome NOT IN ('hit','dud')
    OR (peak_mcap IS NOT NULL AND peak_mcap_samples >= 6 AND hit_threshold IS NOT NULL))$$);
SELECT insidor.add_constraint('public.story_outcome','story_outcome_unmeasurable_has_no_lead',
  $$CHECK (outcome <> 'unmeasurable' OR lead_time_min IS NULL)$$);

```

That six-sample floor has a consequence the nightly classifier must handle explicitly: a story whose coin dies inside thirty minutes cannot be classified hit or dud . It is classified unmeasurable with a lifecycle_reason , never left NULL — the watchdog holds count(outcome IS NULL AND promoted_at < now() - 72h) at zero.

3.9 The board — ranking is not a query

```

CREATE TABLE IF NOT EXISTS public.board_state (
  lane          board_lane NOT NULL,
  band          coin_band,           -- NULL on the posts lane
  post_id       uuid REFERENCES public.post(id) ON DELETE CASCADE,
  mint          text REFERENCES public.coin(mint) ON DELETE CASCADE,
  rank          smallint NOT NULL,
  prev_rank     smallint,
  heat          double precision NOT NULL,
  presented_heat smallint,
  entered_at    timestamptz NOT NULL DEFAULT now(),
  pinned_until  timestamptz NOT NULL DEFAULT now(),  -- 90s dwell
  enter_ticks   smallint NOT NULL DEFAULT 0,
  exit_ticks    smallint NOT NULL DEFAULT 0,
  escape_hatch  boolean NOT NULL DEFAULT false,      -- skipped the dwell, NEW badge
  tick_seq      bigint NOT NULL,
  committed_at timestamptz NOT NULL DEFAULT now()
);

SELECT insidor.add_constraint('public.board_state', 'board_state_entity_xor',
  $$CHECK ((post_id IS NOT NULL)::int + (mint IS NOT NULL)::int = 1)$$);
SELECT insidor.add_constraint('public.board_state', 'board_state_lane_entity_agrees',
  $$CHECK ((lane = 'posts' AND post_id IS NOT NULL AND band IS NULL)
    OR (lane = 'coins' AND mint IS NOT NULL AND band IS NOT NULL))$$);
SELECT insidor.add_constraint('public.board_state', 'board_state_rank_range',
  $$CHECK (rank BETWEEN 1 AND 200)$$);

-- Q1 and Q2 both open here: a 40-row ordered index scan, no Sort node.
CREATE UNIQUE INDEX IF NOT EXISTS board_state_rank_key
  ON public.board_state (lane, COALESCE(band, 'fresh'::coin_band), rank);

```

`board_tick` is RANGE-partitioned by day with 48-hour retention: ~173k rows per day per lane at a 20-second commit, and the Δ10m lookback needs only ten minutes. Dropping a partition is a DETACH, not a DELETE storm against a table being written three times a minute. Partitions are created a week ahead by `pg_cron`; two are created in the migration so a fresh database is immediately writable.

`board_gate_count` carries `eligible`, `shown` and `by_reason jsonb` per commit, so the line under the board — “40 of 63 eligible · 12 hidden by gate” — can distinguish a filtered window from a slow night, including a bare coinability count.

3.10 User, social, trading and launch

`app_user` maps a Privy DID to an internal uuid: `id uuid PRIMARY KEY`, `privy_did text UNIQUE NOT NULL`, `wallet_address text UNIQUE`, both frozen by trigger. An address is identity on Solana; it must never be reassigned to a new DID.

`follow` targets a story **XOR** a coin, and carries a `user_id XOR` a `device_id`, with four partial unique indexes. A story follow never silently becomes a coin follow: when a watched story mints, the story row stays and an explicit second row is written for the coin.

`holding` is unique on `(user_id, wallet, mint)` — **not** `(user_id, mint)`, or a second wallet holding the same coin corrupts both cost basis and the frozen earliness. `minutes_early` is a *signed*, nullable integer, and it cannot exist without the flag that says it was measurable:

```

SELECT insidor.add_constraint('public.holding', 'holding_minutes_early_requires_measurable',
  $$CHECK (minutes_early IS NULL OR (minutes_early_measurable AND story_id IS NOT NULL))$$);

```

The flag itself is set by a `BEFORE INSERT OR UPDATE` trigger that refuses unless a `story_clock` row exists with `unmeasurable = false` and a computed lead — a trigger and not a composite FK, because `ON UPDATE CASCADE` would make it impossible to later mark a clock unmeasurable while anyone held the coin, and *discovering a sensor gap must never be blocked by a user’s position*.

`comment` carries the honesty stamp: `snap_views`, `snap_coin_count`, `snap_unsure_count`, `snap_age_min`, `snap_top_mcap`, `snap_early`, `snap_position_state`, `snap_position_lamports`, `snap_position_mint`, `snap_position_decimals` — all frozen by `freeze_columns`, plus `created_at`, `author_wallet` and `body`. Three triggers guard it: `comment_assert_tier` (refuses a tier-`never` story), `comment_address_gate` (blocks every base58 32–44 run **including this story’s own mints**), and `comment_speed_limit` (three per wallet per sixty seconds, because edge limits are per-region and therefore not a global cap). Two constraints matter:

```

SELECT insidor.add_constraint('public.comment', 'comment_position_holds_needs_amount
  $$CHECK (snap_position_state <> 'holds'
    OR (snap_position_lamports > 0 AND snap_position_mint IS NOT NULL
      AND snap_position_decimals IS NOT NULL))$$);
-- "$0 mcap" reads as a bug, not as "no coin yet".
SELECT insidor.add_constraint('public.comment', 'comment_zero_coin_has_no_mcap',
  $$CHECK (snap_coin_count > 0 OR snap_top_mcap IS NULL))$$);

-- "Earliest" means written when the story was SMALLEST, not oldest by clock.
CREATE INDEX IF NOT EXISTS comment_story_earliest_idx
  ON public.comment (story_id, snap_views, created_at) WHERE status = 'visible';

```

`trade` puts single-submission in the database rather than in a UI disabled state:

```

SELECT insidor.add_constraint('public.trade', 'trade_request_id_key', 'UNIQUE (request_id)');
SELECT insidor.add_constraint('public.trade', 'trade_signature_key', 'UNIQUE (signature)');
-- Deliberately wide. Jupiter documents a 50 bps referral floor but a 10 bps
-- platform fee was measured live on fresh mints, so this number is READ from
-- the order response and displayed, never hardcoded.
SELECT insidor.add_constraint('public.trade', 'trade_fee_bps_range',
  $$CHECK (fee_bps_quoted BETWEEN 0 AND 255
    AND (fee_bps_charged IS NULL OR fee_bps_charged BETWEEN 0 AND 255))$$);
-- A fill is confirmed by signature status, never inferred from a balance change.
SELECT insidor.add_constraint('public.trade', 'trade_confirmed_needs_signature',
  $$CHECK (status <> 'confirmed'
    OR (signature IS NOT NULL AND out_amount_raw IS NOT NULL AND confirmed_at IS NOT NULL))$$);

CREATE TRIGGER trade_freeze BEFORE UPDATE ON public.trade
  FOR EACH ROW EXECUTE FUNCTION insidor.freeze_columns(
    'request_id', 'signature', 'fee_bps_quoted', 'sol_usd_at_trade',
    'in_amount_raw', 'min_out_raw');

```

`sol_usd_at_trade` is `NOT NULL` and captured server-side. Every PnL number in the product derives from that column, never from a browser price global.

`launch` is where Rule 3 becomes a foreign key:

```

SELECT insidor.add_constraint('public.launch', 'launch_story_tier_fk',
  $$FOREIGN KEY (story_id, story_coinability_tier)
    REFERENCES public.story (id, coinability_tier) ON UPDATE RESTRICT$$);
SELECT insidor.add_constraint('public.launch', 'launch_requires_normal_tier',
  $$CHECK (story_coinability_tier = 'normal')$$);
SELECT insidor.add_constraint('public.launch', 'launch_landed_has_mint',
  $$CHECK (status <> 'landed' OR (mint IS NOT NULL AND tx_signature IS NOT NULL AND landed_at IS NOT NULL))$$);

CREATE TRIGGER launch_freeze BEFORE UPDATE ON public.launch
  FOR EACH ROW EXECUTE FUNCTION insidor.freeze_columns(
    'mint_keypair_enc', 'mint', 'tx_signature', 'creator_wallet', 'ticker');

```

`mint_keypair_enc` is `NOT NULL` and persisted encrypted **before the first signature**, so a retry reuses it and a late confirmation fails with *"account already in use"* rather than minting twice.

`ON UPDATE RESTRICT` has a deliberate and slightly startling consequence: **reclassifying a story that has already launched a coin fails with a foreign-key error**. That is intended — it surfaces the question rather than resolving it with a silent cascade — but it needs a runbook entry before creation ships, or it lands as a confusing production error.

`namer_run` is written at **form open**, not at submit, with an outcome in `accepted | edited | abandoned | typed_over` and `time_to_first_keystroke_ms`. Abandonment is the strongest negative signal the namer produces, and a table that only records submissions fits its weights on a hits-only sample.

`fee_share` is `UNIQUE (mint, role)` with `freeze_columns('written_at', 'recipient', 'bps')`, because `updateFeeSharesV2` is one-shot on chain.

3.11 Search and ops

```
-- 0009_search.sql — the immutable wrapper comes FIRST, before any column that
-- depends on it.

CREATE OR REPLACE FUNCTION insidor.text_array_to_string(text[], text)
RETURNS text LANGUAGE sql IMMUTABLE PARALLEL SAFE STRICT AS
$$ SELECT array_to_string($1, $2) $$;

SELECT insidor.add_column('public.story', 'search_doc',
  $$tsvector GENERATED ALWAYS AS (
    setweight(to_tsvector('english'::regconfig, coalesce(title, '')), 'A')
  || setweight(to_tsvector('english'::regconfig, coalesce(subject, '')), 'A')
  || setweight(to_tsvector('english'::regconfig,
    insidor.text_array_to_string(entity_keys, ' ')), 'B')
  || setweight(to_tsvector('english'::regconfig, coalesce(blurb, '')), 'C')
  ) STORED$$);
```

Three retrieval legs are fused with RRF: FTS (GIN on `search_doc`), trigram (`gin_trgm_ops` on `story.title`, `story_ticker.ticker`, `coin.symbol_norm`, at 0.30 on titles and 0.18 on tickers because ticker drift is one or two characters), and vector (HNSW on `story.centroid` and `post_embedding.embedding`, `halfvec_cosine_ops`, `m=16`, `ef_construction=64`). **All three legs join `eligible_story`**, the single eligibility view, so a tier- `never` story cannot be reached through the vector leg because somebody paraphrased its title.

The seven `ops_*` tables are the observability substrate. Three of their constraints encode lessons rather than schema:

```
-- rows_out is the column that makes the output-freshness SLI work. `ingested`
-- does not mean "output" everywhere, and two of the four workers that logged at
-- all would otherwise page from day one.
SELECT insidor.add_constraint('public.ops_stage_run', 'ops_stage_run_ok_finished',
  $$CHECK (ok = false OR finished_at IS NOT NULL)$$);

-- ops_stage_expected is the FIXED list the SLI query LEFT JOINS, so a worker
-- that dies before it can log anything emits a 999 streak on ABSENCE. Absence
-- and zero must both breach.
INSERT INTO public.ops_stage_expected(stage, interval_seconds) VALUES
  ('ingest', 180), ('ingest-tiktok', 600), ('snapshotter', 60), ('score', 120),
  ('cluster', 45), ('resolve-coins', 60), ('trends', 300), ('rank-commit', 20)
ON CONFLICT (stage) DO NOTHING;

-- A green board painted by an SLI querying a nonexistent column is exactly how
-- the original outage stayed invisible.
SELECT insidor.add_constraint('public.ops_sli_sample', 'ops_sli_null_is_unknown',
  $$CHECK ((value IS NULL) = (state = 'unknown'))$$);
```

`ops_spend` deserves one line of its own: **absence of a row means no data, not zero spend.**

3.12 The views — one row for every Create affordance

```
-- Identity is not tradeability. `verdict = 'confirmed'` never expires — a rugged
-- coin would otherwise render Buy forever — so the resolver takes two inputs and
-- this is the second. Concentration is band-dependent on purpose: a flat 60%
-- gate empties Fresh on day one, when a three-minute-old token has nine buyers
-- and is legitimately concentrated.
CREATE OR REPLACE FUNCTION public.coin_tradeable(
  p_band coin_band, p_intrinsic_gate boolean, p_liquidity_usd numeric,
  p_top10_pct real, p_buys_5m integer, p_sells_5m integer,
  p_price_captured_at timestamptz, p_minted_at timestamptz
) RETURNS boolean LANGUAGE sql STABLE AS $$
  SELECT COALESCE(p_intrinsic_gate, false)           -- unknown fails closed
         AND p_minted_at IS NOT NULL                 -- unknown age fails closed
         AND p_top10_pct IS NOT NULL
         AND p_top10_pct ≤ CASE WHEN p_band = 'fresh' THEN 85 ELSE 60 END
         AND (p_band <> 'live' OR COALESCE(p_liquidity_usd, 0) ≥ 3000)
         AND NOT (COALESCE(p_sells_5m, 0) = 0 AND COALESCE(p_buys_5m, 0) > 30)
         AND p_price_captured_at IS NOT NULL
         AND p_price_captured_at > now() - interval '5 minutes'

$$;

-- ONE row read, and no client-side coinability logic anywhere:
--   select can_create, buy_count, unsure_count from story_cta_v where id = $1
CREATE OR REPLACE VIEW public.story_cta_v AS
SELECT s.id, s.can_create, s.coinability_tier,
       s.confirmed_coin_count AS buy_count,
       s.unsure_coin_count    AS unsure_count,
       s.needs_review,
       (SELECT count(*) FROM public.launch l WHERE l.story_id = s.id
        AND l.created_at > now() - interval '1 hour') AS launches_last_1h,
       (SELECT count(*) FROM public.launch l WHERE l.story_id = s.id
        AND l.created_at > now() - interval '24 hours') AS launches_last_24h,
       (SELECT m.mint    FROM public.coin_match m WHERE m.story_id = s.id
        AND m.verdict = 'confirmed' AND m.retracted_at IS NULL
        ORDER BY m.mint_time NULLS LAST LIMIT 1) AS first_confirmed_mint,
       (SELECT m.ticker  FROM public.coin_match m WHERE m.story_id = s.id
        AND m.verdict = 'confirmed' AND m.retracted_at IS NULL
        ORDER BY m.mint_time NULLS LAST LIMIT 1) AS first_confirmed_ticker
FROM public.story s
WHERE s.display_eligible;

COMMENT ON VIEW public.story_cta_v IS
  'The gate is in the query. A no_create story returns can_create = false, and a tier `never` story returns NO ROW because it is not display_eligible.';
```

`record_summary_mv` is the public record's denominators, computed once so `/ops/earliness` and the public page read the same numbers from the same place. `late_n` is published in the same weight as `early_n`; a record with no left tail is marketing. Below `n = 30` every percentile renders as an em dash with *"not enough data (n=12)"*.

3.13 The four hot queries

`supabase/queries/hot-queries.sql` holds them in full, with the index each one drives. `supabase/tests/plans.sql` asserts the plan shape in CI — no `Seq Scan` on `board_state`, `coin_snapshot`, `coin_match`, `comment` or `story_outcome`, and no `Sort` on Q1, Q3-discussion or Q4.

QUERY	SURFACE	TARGET	OPENS ON
Q1 posts board	<code>/feed?lane=posts</code>	< 8 ms	<code>board_state_rank_key</code> , ordered scan, no Sort; then 40 primary-key joins
Q2 coins board	<code>/feed?lane=coins&band=fresh</code>	< 15 ms	same, plus two <code>LATERAL</code> s — a coin with no trades yet must still appear (pre-rank zone, every numeric <code>..</code>), and a coin with no story is 85–95% of that band. An inner join to either would delete the modal row.
Q3 story page	<code>/story/[id]</code>	< 25 ms, one round trip	<code>story_pkey</code> , <code>coin_match_story_idx</code> , <code>post_story_views_idx</code> , <code>story_momentum_recent_idx</code> . Discussion is a second query on its own Suspense boundary so a slow comment scan never blocks the evidence.
Q4 the record	<code>/search?chip=finished</code> , <code>/ops/earliness</code>	< 30 ms	<code>story_outcome_recent_idx</code> . Default filter is All, asserted in CI.

Two details in Q2 and Q3 are product decisions living in SQL. Q2 applies the Fresh six-hour ceiling as a `WHERE` clause — `c.minted_at IS NULL OR c.minted_at > now() - interval '6 hours'` — because it depends on `now()` and a generated column cannot. Q3 sorts a story's matches by `verdict` then `mint_time ASC`, *"the first one made from this story"*; sorting by market cap systematically demotes the freshly minted derived coin, which is the coin the pipeline exists to find.

Q3 also carries the staleness readout, and its shape matters:

```
(SELECT jsonb_object_agg(e.stage, jsonb_build_object(
    'last_output_at', r.last_output_at,
    'zero_streak', COALESCE(r.zero_streak, 999)))
FROM public.ops_stage_expected e
LEFT JOIN LATERAL (
    SELECT max(sr.finished_at) FILTER (WHERE sr.rows_out > 0) AS last_output_at,
           count(*) FILTER (WHERE sr.rows_out = 0) AS zero_streak
    FROM public.ops_stage_run sr
    WHERE sr.stage = e.stage AND sr.started_at > now() - interval '2 hours') r ON true
WHERE e.enabled) AS pipeline_health
```

`max(ran_at)` alone reads `live` during the exact outage that has actually happened. The `FILTER (WHERE rows_out > 0)` is the work-done check, and the `LEFT JOIN` against a fixed expected-stage list is what makes absence breach as loudly as zero.

4. MIGRATIONS

The old build had 24 loose `schema-*.sql` files applied by 12 ad-hoc scripts with no record of what was live. Everything below exists to make that state unreachable.

4.1 The files, in order

#	FILE	CONTENTS
0000	migration_framework.sql	insidor.schema_migrations , migration_begin (advisory lock + ordering + drift), migration_end , add_constraint , add_column , create_enum
0001	extensions_and_types.sql	pgcrypto, pg_trgm, vector, btree_gin; pg_cron/pg_net optional; 21 enums; freeze_columns , reject_future_timestamp , touch_updated_at , base58_runs
0002	story_and_post.sql	story , post , post_snapshot , post_heat , post_meme_score , post_embedding , story_momentum , platform_norm , author_roster
0003	coin.sql	coin , coin_snapshot , coin_mcap_series , coin_safety , coin_image_embedding , creator_stat
0004	matching.sql	story_ticker , coin_match , match_label , confirmable() , the recount trigger
0005	clocks_and_outcome.sql	ct_mention , sensor_heartbeat , sensor_gap , story_clock , story_outcome
0006	board.sql	board_state , board_tick (RANGE-partitioned by day, 48h retention), board_gate_count
0007	user_and_social.sql	app_user , follow , holding , comment , comment_report , notification , push_subscription , notification_prefs
0008	trading_and_launch.sql	trade , launch , namer_run , fee_share
0009	search.sql	insidor.text_array_to_string() , generated search_doc tsvectors, GIN, trgm, HNSW, search_log , eligible_story
0010	ops.sql	ops_stage_run , ops_stage_expected , ops_event , ops_sli_sample , ops_funnel , ops_budget_cap , ops_spend
0011	rls.sql	RLS on all 46 tables, policies, column-level grants
0012	views.sql	coin_tradeable() , story_cta_v , record_summary_mv
0013	realtime.sql	replica identity DEFAULT, comment and coin_match broadcast triggers
0014	environment_and_budget.sql	ops_environment singleton, reserve_units() , audit_ddl() event trigger, the migrator role
0015	roles_bootstrap.sql	anon / authenticated / service_role creation guards for non-Supabase Postgres

Version numbers are **ordinals, not timestamps**. Concurrent PRs adding `0016` collide in git rather than silently interleaving, which is the correct place for that argument to happen.

4.2 The framework

Every file from `0001` on follows the same contract:

```
BEGIN;
SELECT insidor.migration_begin('0007','user_and_social','@@CHECKSUM_0007@@');
-- ... DDL, every statement idempotent ...
SELECT insidor.migration_end('0007','user_and_social','@@CHECKSUM_0007@@');
COMMIT;
```

```
-- 0000_migration_framework.sql

CREATE SCHEMA IF NOT EXISTS insidor;

CREATE TABLE IF NOT EXISTS insidor.schema_migrations (
    version    text          PRIMARY KEY,
    name       text          NOT NULL,
    checksum   text          NOT NULL,
    applied_at timestamptz NOT NULL DEFAULT now(),
    applied_by text          NOT NULL DEFAULT current_user,
    duration_ms integer,
    CONSTRAINT schema_migrations_version_fmt CHECK (version ~ '^[0-9]{4}$')
);
COMMENT ON TABLE insidor.schema_migrations IS
    'The record of what is applied to this database. If a version is not here, it is not applied.';

CREATE TABLE IF NOT EXISTS insidor.migration_run (
    version text PRIMARY KEY,
    started_at timestamptz NOT NULL DEFAULT clock_timestamp()
);

CREATE OR REPLACE FUNCTION insidor.migration_begin(p_version text, p_name text, p_checksum text)
RETURNS boolean LANGUAGE plpgsql AS $$
DECLARE v_existing insidor.schema_migrations%ROWTYPE; v_ahead text;
BEGIN
    -- SINGLETON: released automatically at COMMIT/ROLLBACK. Two deploys racing
    -- serialise instead of interleaving DDL.
    PERFORM pg_advisory_xact_lock(hashtext('insidor.schema_migrations'));
    SELECT * INTO v_existing FROM insidor.schema_migrations WHERE version = p_version;
    IF FOUND THEN
        -- DRIFT
        IF v_existing.checksum <> p_checksum THEN
            RAISE EXCEPTION
                'migration % (%) was applied at % with checksum %, but the file on disk now hashes to %. '
                'Migrations are immutable once applied. Write a new migration.',
                p_version, v_existing.name, v_existing.applied_at, v_existing.checksum, p_checksum;
        END IF;
        INSERT INTO insidor.migration_run(version) VALUES (p_version)
            ON CONFLICT (version) DO UPDATE SET started_at = clock_timestamp();
        RETURN true; -- already applied; the DDL below must be a no-op
    END IF;
    -- ORDERING
    SELECT version INTO v_ahead FROM insidor.schema_migrations
        WHERE version > p_version ORDER BY version LIMIT 1;
    IF v_ahead IS NOT NULL THEN
        RAISE EXCEPTION 'refusing to apply % after %: migrations must be applied in ascending order',
            p_version, v_ahead;
    END IF;
    INSERT INTO insidor.migration_run(version) VALUES (p_version)
        ON CONFLICT (version) DO UPDATE SET started_at = clock_timestamp();
    RETURN false;
END $$;

CREATE OR REPLACE FUNCTION insidor.migration_end(p_version text, p_name text, p_checksum text)
RETURNS void LANGUAGE plpgsql AS $$
DECLARE v_started timestamptz;
BEGIN
    SELECT started_at INTO v_started FROM insidor.migration_run WHERE version = p_version;
    INSERT INTO insidor.schema_migrations(version, name, checksum, duration_ms)
    VALUES (p_version, p_name, p_checksum,
        GREATEST(0, (EXTRACT(epoch FROM clock_timestamp()) - COALESCE(v_started, clock_timestamp())) * 1000)::int))
    ON CONFLICT (version) DO NOTHING;
    DELETE FROM insidor.migration_run WHERE version = p_version;
END $$;

-- ALTER TABLE ... ADD CONSTRAINT has no IF NOT EXISTS, and a schema this
-- constraint-heavy needs one or every file is 40% DO-block noise.
CREATE OR REPLACE FUNCTION insidor.add_constraint(p_table regclass, p_name text, p_def text)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conrelid = p_table AND conname = p_name) THEN
        EXECUTE format('ALTER TABLE %s ADD CONSTRAINT %I %s', p_table::text, p_name, p_def);
    END IF;
END $$;

CREATE OR REPLACE FUNCTION insidor.add_column(p_table regclass, p_name text, p_def text)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
    EXECUTE format('ALTER TABLE %s ADD COLUMN IF NOT EXISTS %I %s', p_table::text, p_name, p_def);
END $$;

CREATE OR REPLACE FUNCTION insidor.create_enum(p_name text, p_values text[])

```

```
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = p_name) THEN
        EXECUTE format('CREATE TYPE public.%I AS ENUM (%s)',
            p_name, (SELECT string_agg(quote_literal(v), ', ' ) FROM unnest(p_values) v));
    END IF;
END $$;

INSERT INTO insidor.schema_migrations(version, name, checksum)
VALUES ('0000', 'migration_framework', 'bootstrap') ON CONFLICT (version) DO NOTHING;
```

4.3 The applier

`scripts/migrate.mjs` is the **only** way migrations reach any database — local, preview, staging or production. Node ESM, `pg` only, no CLI, no dashboard.

```
#!/usr/bin/env node
// node scripts/migrate.mjs status | up | verify
//
// Checksums: each file contains a literal `@@CHECKSUM_NNNN@@` placeholder. The
// hash is taken over the file WITH the placeholder intact, then substituted in
// before execution. So the recorded checksum is a stable hash of the source
// text, and editing one character of an applied migration makes
// insidor.migration_begin() refuse it.

import { readFile, readdir } from 'node:fs/promises';
import { createHash } from 'node:crypto';
import path from 'node:path';
import process from 'node:process';
import pg from 'pg';

const DIR = path.resolve('supabase/migrations');
const RE = /^(\\d{4})_([a-z0-9_]+)\\.sql$/;

async function load() {
  const files = (await readdir(DIR)).filter((f) => RE.test(f)).sort();
  return Promise.all(files.map(async (file) => {
    const [, version, name] = file.match(RE);
    const raw = await readFile(path.join(DIR, file), 'utf8');
    const checksum = createHash('sha256').update(raw).digest('hex').slice(0, 32);
    const sql = raw.replaceAll(`@@CHECKSUM_${version}@@`, checksum);
    if (version !== '0000' && !raw.includes(`@@CHECKSUM_${version}@@`)) {
      throw new Error(`${file}: missing @@CHECKSUM_${version}@@ placeholder`);
    }
    return { file, version, name, checksum, sql };
  }));
}

async function main() {
  const cmd = process.argv[2] ?? 'status';
  const url = process.env.DATABASE_URL;
  if (!url) throw new Error('DATABASE_URL is not set');

  const migrations = await load();
  const client = new pg.Client({ connectionString: url });
  await client.connect();

  const has = await client.query(
    `select to_regclass('insidor.schema_migrations') is not null as ok`);
  const applied = has.rows[0].ok
    ? (await client.query('select version, checksum from insidor.schema_migrations')).rows
    : [];
  const appliedBy = new Map(applied.map((r) => [r.version, r.checksum]));

  if (cmd === 'status') {
    for (const m of migrations) {
      const a = appliedBy.get(m.version);
      const state = !a ? 'PENDING' : a === m.checksum || a === 'bootstrap' ? 'applied' : 'DRIFT';
      console.log(`${m.version}  ${state.padEnd(8)}  ${m.name}`);
    }
    await client.end();
    return;
  }

  if (cmd !== 'up' && cmd !== 'verify') throw new Error(`unknown command: ${cmd}`);

  for (const m of migrations) {
    const already = appliedBy.has(m.version);
    if (already && cmd === 'up') { console.log(`${m.version} skip  ${m.name}`); continue; }
    // `verify` deliberately re-runs applied migrations: every statement is
    // idempotent, so a second pass must be a no-op. If it is not, the DDL is
    // wrong and CI fails here rather than during a production deploy.
    process.stdout.write(`${m.version} apply  ${m.name} ... `);
    const t0 = Date.now();
    await client.query(m.sql);
    console.log(`${Date.now() - t0}ms`);
  }
  await client.end();
}

main().catch((e) => { console.error(`\\nmigration failed: ${e.message}`); process.exit(1); });
```

Do not let the Supabase CLI apply these files, anywhere, ever. Two independent incompatibilities make it silently wrong. First, the CLI executes the raw bytes, so it records `checksum = '@@CHECKSUM_0013@@'` in its own ledger — after which `scripts/migrate.mjs` computes the real sha256, mismatches, and the drift guard bricks the database

permanently against the project's own tool. Second, the CLI's batch apply is implicitly transactional and appends its ledger INSERT to the same batch; our inner `COMMIT;` closes that transaction early, so a failure after it leaves committed DDL with **no row in either ledger** — a half-migrated database that neither `supabase migration list` nor `migrate.mjs status` reports as broken.

Consequently Supabase Branching's automatic Migrate step is **disabled**. Branch databases are migrated by the workflow, calling `migrate.mjs` against the branch's `DATABASE_URL` . One applier, one ledger.

(The CLI's filename regex is `^([0-9]+)(.*)\.sql$` , so `0001_...` is accepted. Do not rename these files to timestamps; that is not the incompatibility.)

4.4 Authoring rules, enforced by `scripts/check-migrations.mjs`

Blocking in CI. Every rule corresponds to something the old build broke.

1. **Filename** is `NNNN_snake_case.sql` , four digits.
2. **Immutable once merged**. The script diffs each file against `origin/main` ; a changed byte in an applied migration fails the PR. Deleting one fails too.
3. **Strictly increasing**. A new version must exceed every merged version.
4. **Required header**: `@phase` , `@lock` , `@ticket` , `@summary` .
5. **Every statement idempotent** — `CREATE TABLE IF NOT EXISTS` , `add_constraint` , `add_column` , `create_enum` . `migrate.mjs verify` re-runs the whole set and it must be a no-op.
6. `ADD COLUMN ... NOT NULL without DEFAULT` is **rejected** outside a `contract` phase. It breaks every INSERT from the code currently deployed the instant it lands.
7. `ALTER TABLE ... ADD CONSTRAINT` **must be NOT VALID** , with `VALIDATE CONSTRAINT` in a later migration, unless `@lock: takes-access-exclusive` is set deliberately. Inline CHECKs inside `CREATE TABLE` are unaffected.
8. `CREATE INDEX on a populated table` lives in `supabase/migrations-concurrent/` and uses `CREATE INDEX CONCURRENTLY IF NOT EXISTS` . Those files run in autocommit and carry no `BEGIN / COMMIT` .
9. **Every transactional file opens with the three guards**:

```
set local lock_timeout = '3s';           -- an unbounded ALTER queued behind a long
set local statement_timeout = '60s';     -- read stalls every writer
set local app.migration = 'on';          -- or ops.audit_ddl() pages the on-call
```

10. **A `@phase: contract` PR may contain nothing outside `supabase/` and `docs/`** . This is what makes "drop the column one release after the code stopped reading it" mechanical rather than remembered.

```
-- supabase/migrations/_TEMPLATE.sql
-- @phase: expand           # expand | backfill | contract
-- @lock: safe              # safe | takes-access-exclusive
-- @ticket: INS-000
-- @summary: one line, present tense, what this adds.
--
-- Reviewer checklist (the linter enforces 1-5; a human must do 6-8):
-- 1. Version is greater than every merged migration.
-- 2. Every statement is idempotent; re-running is a no-op.
-- 3. New columns nullable or defaulted.
-- 4. ADD CONSTRAINT is NOT VALID; VALIDATE ships later.
-- 5. Indexes on populated tables live in migrations-concurrent/.
-- 6. Does the code currently deployed to production still work after this?
-- 7. Can this be a CHECK constraint or a generated column instead of a rule
--    someone remembers? Product rules belong in the schema.
-- 8. If this touches coin_match, coinability_tier, promoted_at or anything the
--    primary-action resolver reads: has the frozen regression set been re-run?

BEGIN;
SELECT insidor.migration_begin('00NN', 'name', '@@CHECKSUM_00NN@@');

set local lock_timeout = '3s';
set local statement_timeout = '60s';
set local app.migration = 'on';

-- your DDL here

SELECT insidor.migration_end('00NN', 'name', '@@CHECKSUM_00NN@@');
COMMIT;
```

4.5 Forward-only, expand → backfill → contract

There are **no down migrations**. A down migration is written once, never tested, and does not fix the failure that actually matters — a migration that already ran and corrupted data. What exists instead: expand-only phases enforced by the linter, a pre-migration dump retained ninety days, and PITR marked in the runbook as founder-authorised, because restoring it deletes real trades.

A breaking change — say `coin_match.ticker` moving to `coin.symbol` — takes five releases:

RELEASE	MIGRATION	APPS
R1	<code>@phase: expand</code> — add the new column nullable; add an <code>AFTER INSERT/UPDATE</code> trigger mirroring writes from the old column	unchanged
R2	none	both apps dual-write ; readers prefer new, fall back to old
R3	<code>@phase: backfill</code> — fill history in batches, resumable and idempotent; add <code>CHECK ... NOT VALID</code>	unchanged
R4	<code>@phase: expand</code> — <code>VALIDATE CONSTRAINT</code>	both apps read new only ; the fallback is deleted
R5	<code>@phase: contract</code> — drop the trigger, drop the old column	unchanged

Never reuse a name with a new meaning. That is the one change whose failure mode is silent wrongness rather than an error.

4.6 Out-of-band DDL cannot hide

Four mechanisms, in the order they fire:

- `ops.audit_ddl()` , an event trigger on `ddl_command_end` , records every DDL statement and flags any not carrying `app.migration = 'on'` . A `pg_cron` job posts flagged rows to Slack through `pg_net` every five minutes. It survives total loss of the app platform.
- Only migrator can do DDL.** `CREATE` on `public` and `insidor` is revoked from `public` , `anon` , `authenticated` and `service_role` .
- The credential does not exist outside the gate.** `MIGRATOR_DATABASE_URL` for production is a GitHub *Environment* secret on `production` , which has required reviewers and a wait timer. No human holds it.
- And the next PR fails.** CI builds a database from scratch and diffs it against `supabase/schema.sql` — the committed dump of what production actually runs, refreshed by the release workflow after every production migration. Hand-run DDL makes that diff non-empty and blocks everyone until it is reconciled by a real migration.

5. AUTH AND SECURITY

Identity is a Solana wallet, held by Privy. It is not a Supabase Auth user, and it never will be. Everything below follows from that one fact and from a second one that is easy to state and expensive to forget: **Supabase chooses a PostgreSQL role by reading the literal `role` claim out of the presented JWT, and falls back to `anon` when it is absent.** Privy's access token has a closed claim set — `sid`, `sub`, `iss`, `aud`, `iat`, `exp` — and no documented claim-injection hook. A Privy token handed to PostgREST does not fail; it succeeds *as* `anon`, returns HTTP 200 with zero rows, and renders a portfolio page reading `$0` for a user who is holding. That is the silent-wrongness shape this whole document exists to prevent, so it is designed against rather than discovered.

5.1 The decision: server-side writes on the service role

v1: every write goes through a Next.js Route Handler that verifies the Privy token itself and then writes with the service role. Supabase third-party auth via `jwtks_url` is phase 2, not v1.

Four reasons, in order of weight.

1. **There is no `authenticated` principal to authorise against.** Privy meets Supabase's stated third-party requirement (asymmetric keys with a `kid`; `https://auth.privy.io/api/v1/apps/<app_id>/jwtks.json` returns EC P-256 / `ES256`), and Supabase's `POST /v1/projects/{ref}/config/auth/third-party-auth` accepts a generic `jwtks_url` — the five named providers are guides, not an allow-list. Note `jwtks_url`, **not** `oidc_issuer_url`: `auth.privy.io` serves no `/.well-known/openid-configuration` (404). None of that helps, because the token still carries no `role`.
2. **The `sub` is the wrong type.** Privy's `sub` is a DID (`did:privy:cl...`); `app_user.id` is a `uuid`. Even with claim injection, a policy written `user_id::text = auth.jwt() -> 'sub'` compiles, runs, and evaluates FALSE forever. `0011_rls.sql` **as merged contains exactly that expression in six places.** It is corrected in 5.4 below, and it is the single most important correction in this section.
3. **The highest-volume follower class is anonymous device follows,** which have no principal at all. No policy can distinguish the owner of a `device_id` from anyone who guesses one, so that path is server-routed regardless of what the authenticated path does.
4. **Every write has an invariant that requires a server-side READ.** The comment stamp needs a Helius balance call. The launch needs a fresh `count(confirmed) = 0` at submit. The trade needs a SOL price captured server-side. A direct client insert cannot satisfy any of them, so a client insert path would have to be forbidden anyway — which means building it buys nothing and adds a second way in.

What we give up is one hop of latency and the use of PostgREST as a write API. Neither is load-bearing: no write in this product sits on the board's 2-second path.

The migration path, and it is not the obvious one. Phase 2 is *not* persuading Privy to inject `role`. It is **minting our own Supabase JWT** in the route that already verifies the Privy token: import a signing key into Supabase, mint `{ sub: <app_user.id>, role: 'authenticated', exp: now + 15m }`, hand it to the browser, and let PostgREST enforce the owner policies directly. The DID→uuid problem disappears because we choose the `sub`. The precondition is that `current_app_user()` already accepts **both** shapes, which is why it is written the way it is below — phase 2 then becomes a config change plus a token endpoint, with zero policy edits and zero schema change. Two things to settle first: whether a self-minted JWT counts against Supabase's third-party MAU meter at \$0.00325 (ask billing), and a 15-minute TTL with a silent refresh so a stale tab does not 401 mid-scroll.

5.2 The flow end to end

A user lands, connects a wallet, and forty minutes later posts a comment.

```
1  GET /story/<id>      RSC renders with the ANON key through RLS. No session.
                          Browser holds: nothing.

2  Connect wallet      Privy modal → embedded or external wallet.
                          Privy issues an ACCESS TOKEN (~1h, JWT, ES256) and an
                          IDENTITY TOKEN. Both live in Privy's own cookie/memory.
                          We store neither ourselves.

3  First authed call   client: await getAccessToken() ← ALWAYS re-read, never
                          cached in our own state; the SDK refreshes near expiry.
                          POST /api/comments
                          Authorization: Bearer <privy access token>
                          body: { storyId, body } ← NO wallet address.

4  Route: verify       jose.jwtVerify(token, JWKS, { issuer, audience })
                          JWKS cached in module scope, remote-refetched on unknown
                          kid. Failure of ANY kind → 401. Never a fallback path.

5  Route: resolve      did → app_user (upsert on first sight).
                          wallet address read from PRIVY'S API using the app secret,
                          never from the request body. Bound once, frozen forever.

6  Route: establish facts
                          Helius: does this wallet hold this story's top coin?
                          coin_match: confirmed count, unsure count.
                          story: age, views, top mcap.
                          → the eleven snap_* columns, computed SERVER-SIDE.

7  Route: write        SERVICE-ROLE client constructed from env in-process.
                          insert into comment (... , user_id = <resolved uuid>, ...)
                          DB triggers: tier gate, address gate, 3/60s speed limit.

8  Database: broadcast AFTER INSERT trigger, status='visible' only →
                          realtime.send(sanitised row, 'insert', 'story:<id>', true)

9  Every browser on    one ref-counted socket receives the frame. Token refresh
    that story         loop keeps the socket alive past the hour (5.6).
```

Step 5 is where forged input dies. The wallet address is **never** a request field. It is fetched from Privy server-side with `PRIVY_APP_SECRET`, written once into `app_user.wallet_address`, and frozen by the `app_user_freeze` trigger already in `0007` (`freeze_columns('privy_did', 'wallet_address')`) with a `UNIQUE` constraint on top. One address, one DID, permanently. A client that posts `{ wallet: <someone else's> }` is posting a field nothing reads.

Step 7 has a trap worth naming because it is the mistake that will be made: **never construct the Supabase client from a forwarded `Authorization` header**. Supabase's documentation is unambiguous that it adheres to the RLS policy of the signed-in user even when the client was initialised with the service key. Since the route has already verified a Privy token, forwarding it is the natural thing to type — and it silently demotes the route off the service role, at which point every write fails closed and every read returns empty. The factories in 5.5 make it impossible to type.

Token lifetime. Privy access tokens live about an hour. The client calls `getAccessToken()` immediately before every request and never stores the string; the SDK handles refresh. The server treats `exp` as absolute — no clock skew allowance beyond jose's default, no "recently expired is fine" branch.

5.3 The verification code

We verify with `jose` and `createRemoteJWKSet` rather than through `@privy-io/node`'s helper. That is a deliberate choice against the SDK: we need to assert `iss` and `aud` explicitly and to control the failure taxonomy, and this project has already been burned once by a helper whose error handling was assumed rather than read (the retry that only retried 429). `jose` is also the library that mints the phase-2 Supabase JWT, so it is one dependency, not two.

```
// apps/web/src/features/wallet/server/verify.ts
import 'server-only';
import { createRemoteJWKSet, jwtVerify, type JWPayload } from 'jose';
import { serverEnv } from '@insidor/env';

/** Privy's per-app JWKS. NO /.well-known/openid-configuration exists (404), so the
 * URL is constructed, not discovered.
 * Δ VERIFY: that this path is a supported public contract (open item U8), and that
 * `iss` is literally "privy.io" on a live token. Read a real token before trusting
 * either. Both are asserted below, so a change breaks loudly, not silently. */
const JWKS = createRemoteJWKSet(
  new URL(`https://auth.privy.io/api/v1/apps/${serverEnv.NEXT_PUBLIC_PRIVY_APP_ID}/jwks.json`),
  { cacheMaxAge: 10 * 60_000, timeoutDuration: 4_000 },
);

export type PrivyClaims = JWPayload & { sub: string; sid: string };

export class AuthError extends Error {
  constructor(readonly reason: string) { super(reason); }
}

export async function verifyPrivyToken(authorization: string | null): Promise<PrivyClaims> {
  const raw = authorization?.startsWith('Bearer ') === true ? authorization.slice(7) : null;
  if (raw === null || raw.length === 0) throw new AuthError('missing_bearer');

  let payload: JWPayload;
  try {
    ({ payload } = await jwtVerify(raw, JWKS, {
      issuer: 'privy.io',
      audience: serverEnv.NEXT_PUBLIC_PRIVY_APP_ID,
      algorithms: ['ES256'], // pinned. `none` and HS* are unrepresentable.
      clockTolerance: 0,
    }));
  } catch (e) {
    // EVERY failure is 401. There is no branch here that continues on error, and no
    // catch that returns a default principal. This is the shape of the safety-endpoint
    // defect: a read that could not establish a fact must not report the safe answer.
    throw new AuthError(e instanceof Error ? e.name : 'verify_failed');
  }

  if (typeof payload.sub !== 'string' || !payload.sub.startsWith('did:privy:')) {
    throw new AuthError('sub_not_a_privy_did');
  }

  if (typeof payload.sid !== 'string') throw new AuthError('missing_sid');
  return payload as PrivyClaims;
}
```

```
// apps/web/src/features/wallet/server/principal.ts
import 'server-only';
import { serviceClient } from '@insidor/db';
import { serverEnv } from '@insidor/env';
import { verifyPrivyToken, AuthError } from './verify';

export type Principal = { userId: string; did: string; wallet: string };

/** The wallet address comes from PRIVY, never from the request. Privy's ACCESS token
 * carries no linked accounts – only the identity token does, and a client-supplied
 * identity token is a client-supplied wallet. So we ask Privy directly.
 * △ VERIFY on the day this is written: endpoint path, the `privy-app-id` header, and
 * Basic auth = base64(app_id:app_secret). Documented, not yet called by us. */
async function privyWallet(did: string): Promise<string> {
  const res = await fetch(`https://auth.privy.io/api/v1/users/${encodeURIComponent(did)}`, {
    headers: {
      authorization: `Basic ${Buffer.from(
        `${serverEnv.NEXT_PUBLIC_PRIVY_APP_ID}:${serverEnv.PRIVY_APP_SECRET}`,
      ).toString('base64')}`,
      'privy-app-id': serverEnv.NEXT_PUBLIC_PRIVY_APP_ID,
    },
    signal: AbortSignal.timeout(4_000),
  });
  // Any non-2xx throws. A 402, a 403 and a 500 are all failures – the old build's
  // retry helper retried only 429 and let a 402 vanish for two days.
  if (!res.ok) throw new AuthError(`privy_user_http_${res.status}`);
  const user = (await res.json()) as { linked_accounts?: { type: string; chain_type?: string; address?: string }[] };
  const w = user.linked_accounts?.find(
    (a) => a.type === 'wallet' && a.chain_type === 'solana' && typeof a.address === 'string',
  );
  if (w?.address === undefined) throw new AuthError('no_solana_wallet_linked');
  return w.address;
}

/** The ONE function every authed route starts with. */
export async function requirePrincipal(req: Request): Promise<Principal> {
  const { sub: did } = await verifyPrivyToken(req.headers.get('authorization'));
  const db = serviceClient();

  const existing = await db.from('app_user')
    .select('id, wallet_address').eq('privy_did', did).maybeSingle();
  if (existing.error !== null) throw existing.error;
  if (existing.data !== null && existing.data.wallet_address !== null) {
    void db.from('app_user').update({ last_seen_at: new Date().toISOString() }).eq('id', existing.data.id);
    return { userId: existing.data.id, did, wallet: existing.data.wallet_address };
  }

  const wallet = await privyWallet(did);
  const up = await db.from('app_user')
    .upsert({ privy_did: did, wallet_address: wallet }, { onConflict: 'privy_did' })
    .select('id, wallet_address').single();
  // 23505 on wallet_address means this address is already bound to another DID.
  // That is a takeover attempt or a Privy account merge; it is a 409, never a rebind.
  if (up.error !== null) throw up.error;
  return { userId: up.data.id, did, wallet: up.data.wallet_address! };
}
```

```
// apps/web/src/app/api/comments/route.ts – 12 lines, inside the 40-line cap.
export { POST } from '@server/discussion';
```

```
// apps/web/src/features/discussion/server/comments.handler.ts (abridged to the auth path)
import 'server-only';
import { CommentInput } from '@insidor/contracts';
import { requirePrincipal, AuthError } from '@server/wallet';
import { pgStatus } from './pg-error';

export async function POST(req: Request): Promise<Response> {
  let p;
  try { p = await requirePrincipal(req); }
  catch (e) {
    if (e instanceof AuthError) return Response.json({ error: e.reason }, { status: 401 });
    throw e;
  }

  const parsed = CommentInput.safeParse(await req.json());
  if (!parsed.success) return Response.json({ error: 'invalid' }, { status: 422 });

  const stamp = await buildStamp(parsed.data.storyId, p.wallet); // Helius + coin_match, server-side
  const ins = await serviceClient().from('comment').insert({
    story_id: parsed.data.storyId,
    user_id: p.userId, // resolved, never from the body
    author_wallet: p.wallet, // bound, never from the body
    body: parsed.data.body,
    ...stamp,
  }).select('id').single();

  if (ins.error === null) return pgStatus(ins.error); // 23505→409, IR429→429, 23514→422
  return Response.json({ id: ins.data.id }, { status: 201 });
}
```

5.4 RLS — the read path, and the correction to 0011

The policies are almost entirely SELECT policies. They exist to make the coinability tier and the moderation state unbypassable on the half a browser can actually reach. 0011 gets them right in structure and wrong in the predicate: six policies compare `user_id::text` to the raw `sub`, which under a Privy token is a DID and never a uuid. Nothing has been applied to any database yet (open item 1), so **the fix lands in 0011 itself**; if any environment has already applied it, the identical block ships as `0016_auth_principal.sql`, since a merged-and-applied migration is immutable.

```

-- Replaces the six `user_id::text = ... → 'sub'` predicates in 0011.
--
-- Accepts BOTH principal shapes deliberately:
--   * a Privy DID → resolved through app_user.privy_did      (phase 1, inert)
--   * a uuid      → our own app_user.id                     (phase 2, live)
-- so switching to server-minted Supabase JWTs is a config change, not a policy rewrite.
--
-- nullif() is not decoration: PostgREST sets request.jwt.claims to the EMPTY STRING on
-- some paths, and '::jsonb' raises INSIDE the policy — failing the entire query rather
-- than filtering a row. The CASE guard around ::uuid is the same discipline: WHERE-clause
-- evaluation order is not guaranteed, so the regex cannot protect the cast from there.
CREATE OR REPLACE FUNCTION public.current_app_user() RETURNS uuid
LANGUAGE sql STABLE AS $$
    WITH claim AS (
        SELECT nullif(current_setting('request.jwt.claims', true), '')::jsonb → 'sub' AS sub
    ), norm AS (
        SELECT sub,
            CASE WHEN sub ~* '^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$'
                THEN sub::uuid END AS sub_uuid
        FROM claim
    )
    SELECT u.id FROM public.app_user u, norm n
    WHERE u.privy_did = n.sub OR u.id = n.sub_uuid
    LIMIT 1
$$;

-- Owner-scoped reads. Note `(select public.current_app_user())`, not a bare call: the
-- subselect form is evaluated ONCE as an InitPlan instead of per row. On `holding` that
-- is the difference between one app_user lookup and one per position.
DO $$
DECLARE t text;
BEGIN
    FOREACH t IN ARRAY ARRAY['holding','trade','notification','notification_prefs',
                            'push_subscription','follow'] LOOP
        EXECUTE format('DROP POLICY IF EXISTS %I ON public.%I', t || '_owner_read', t);
        EXECUTE format($f$
            CREATE POLICY %I ON public.%I FOR SELECT TO authenticated
            USING (user_id = (SELECT public.current_app_user()))
            $f$, t || '_owner_read', t);
    END LOOP;
END $$;

DROP POLICY IF EXISTS comment_author_read ON public.comment;
CREATE POLICY comment_author_read ON public.comment FOR SELECT TO authenticated
    USING (user_id = (SELECT public.current_app_user()));

DROP POLICY IF EXISTS follow_owner_write ON public.follow;
CREATE POLICY follow_owner_write ON public.follow FOR INSERT TO authenticated
    WITH CHECK (user_id = (SELECT public.current_app_user()) AND device_id IS NULL);
DROP POLICY IF EXISTS follow_owner_delete ON public.follow;
CREATE POLICY follow_owner_delete ON public.follow FOR DELETE TO authenticated
    USING (user_id = (SELECT public.current_app_user()));

-- FORCE, not just ENABLE. Without FORCE, a table's OWNER bypasses its own policies —
-- and the owner is the role the migrations run as.
DO $$
DECLARE t text;
BEGIN
    FOREACH t IN ARRAY ARRAY['story','post','coin','coin_match','comment','holding','trade',
                            'follow','app_user','notification','launch'] LOOP
        EXECUTE format('ALTER TABLE public.%I FORCE ROW LEVEL SECURITY', t);
    END LOOP;
END $$;

```

`launch`, `comment` and `trade` INSERT are deliberately not granted even to `authenticated`, permanently — the stamp needs a chain read, the launch needs a fresh confirmed-count, the trade needs a server-captured SOL price. The route is the only path.

An invariant test, because a policy nobody exercises is a comment.

```

-- supabase/tests/invariants.sql
SET ROLE authenticated;
SELECT set_config('request.jwt.claims', '{"sub":"did:privy:notarealuser"}', true);
DO $$ BEGIN
  ASSERT (SELECT count(*) FROM public.holding) = 0, 'unknown DID must see zero holdings';
END $$;
SELECT set_config('request.jwt.claims', format('{"sub":"%s"}', (SELECT privy_did FROM public.app_user LIMIT 1)), true);
DO $$ BEGIN
  ASSERT (SELECT count(*) FROM public.holding) > 0, 'a known DID must see its own holdings - '
    'this is the assertion that catches the DID/uuid mismatch that returns 200 with zero rows';
END $$;
RESET ROLE;

```

5.5 The service-role boundary

Three factories, three files, one of which is firewalled. The names differ so a wrong import reads wrong.

```

// packages/db/src/browser.ts — anon key. Ships to the browser. This is fine.
import { createClient } from '@supabase/supabase-js';
import type { Database } from './types.generated';
import { clientEnv } from '@insidor/env';

let singleton: ReturnType<typeof createClient<Database>> | undefined;
export function browserClient() {
  singleton ??= createClient<Database>(
    clientEnv.NEXT_PUBLIC_SUPABASE_URL,
    clientEnv.NEXT_PUBLIC_SUPABASE_ANON_KEY,
    { auth: { persistSession: false, autoRefreshToken: false }, realtime: { params: { eventsPerSecond: 4 } } },
  );
  return singleton;
}

```

```

// packages/db/src/server.ts — anon key, request-scoped, for RSC reads through RLS.
import 'server-only';
import { createClient } from '@supabase/supabase-js';
import type { Database } from './types.generated';
import { serverEnv } from '@insidor/env';

/** Reads only. If you are here to write, you want serviceClient() in a route handler. */
export function requestClient(accessToken?: string) {
  return createClient<Database>(
    serverEnv.NEXT_PUBLIC_SUPABASE_URL,
    serverEnv.NEXT_PUBLIC_SUPABASE_ANON_KEY,
    {
      auth: { persistSession: false, autoRefreshToken: false },
      global: accessToken === undefined ? {} : { headers: { Authorization: `Bearer ${accessToken}` } } },
    },
  );
}

```

```

// packages/db/src/service.ts
// THE ONLY FILE IN THE REPOSITORY THAT NAMES SUPABASE_SERVICE_ROLE_KEY.
import 'server-only';
import { createClient, type SupabaseClient } from '@supabase/supabase-js';
import type { Database } from './types.generated';
import { serverEnv } from '@insidor/env';

let singleton: SupabaseClient<Database> | undefined;

/** Bypasses RLS. Takes NO arguments — in particular it cannot be handed a request or a
 * header, because the failure mode this signature exists to prevent is a caller
 * "helpfully" forwarding the user's Authorization header, which demotes the client off
 * the service role and silently returns zero rows on every read and denies every write. */
export function serviceClient(): SupabaseClient<Database> {
  singleton ??= createClient<Database>(
    serverEnv.NEXT_PUBLIC_SUPABASE_URL,
    serverEnv.SUPABASE_SERVICE_ROLE_KEY,
    { auth: { persistSession: false, autoRefreshToken: false }, global: { headers: { 'x-insidor-role': 'service' } } },
  );
  return singleton;
}

```

What structurally prevents the key reaching the browser, in firing order:

LAYER	MECHANISM	FAILURE MODE IT REMOVES
Env schema	<code>SUPABASE_SERVICE_ROLE_KEY</code> exists only in <code>packages/env</code> 's server schema and is outside the <code>NEXT_PUBLIC_</code> namespace. Next inlines nothing else.	Even a bundled import evaluates to <code>undefined</code> , not a key.
Module	<code>import 'server-only'</code> on line 1 of <code>service.ts</code> .	A <code>'use client'</code> graph reaching it fails the build , not a lint.
Signature	<code>serviceClient()</code> takes no parameters.	The forwarded-header demotion is untypeable.
Lint	ui/ and hooks/ may not name <code>@insidor/db</code> , <code>@insidor/env</code> , <code>server-only</code> or <code>next/headers</code> (§2, layer 2).	Direct imports from client segments.
Graph	dependency-cruiser <code>service-key-firewall</code> with <code>reachable: true</code> , from anything outside <code>features/*/server/</code> and <code>apps/pipeline/src/</code> .	Transitive reach — a client component importing a barrel that re-exports a repo. This is the one that actually catches it.
Database	<code>service_role</code> is the only role holding INSERT/UPDATE/DELETE on any table.	A leaked <i>anon</i> key writes nothing.

Two additions specific to this section:

```
// eslint.config.mjs - appended. process.env has exactly one legal home.
{
  files: ['apps/**/*.{ts,tsx}', 'packages/**/*.ts'],
  ignores: ['packages/env/src/**', '**/*.config.{ts,js,mjs}'],
  rules: {
    'no-restricted-properties': ['error',
      { object: 'process', property: 'env',
        message: 'packages/env is the only module that reads process.env. No defaults, no ' +
          'fallbacks: `process.env.X ?? "..."` is the same shape as the token lookup that ' +
          'returned age 0 - failing OPEN on the axis the product sells.' }]],
    'no-restricted-syntax': ['error',
      { selector: 'Literal[value=/(eyJ|sb_secret_|service_role)/]',
        message: 'That looks like a Supabase key literal. Keys come from @insidor/env.' }]],
  },
},
```

A CI grep is the last line, and it is not redundant with the graph rule — it catches the key arriving in a bundle by a path that is not an import at all (an env var misconfigured into `NEXT_PUBLIC_` , an inlined literal, a source map):

```
# .github/workflows/ci.yml, after `next build`
if grep -rLE '(sb_secret_|"role" *: *"service_role")' apps/web/.next/static/ ; then
  echo '::error::service-role material found in a client chunk'; exit 1
fi
```

5.6 Realtime auth

The failure everyone predicts: Realtime caches channel authorisation for the connection lifetime and closes the socket when the JWT expires, Privy tokens live about an hour, so without a refresh loop every user silently drops off the live feed after sixty minutes — values freeze, nothing errors, and the board looks slow rather than broken.

The sharper version of the fix is to be deliberate about *which* token the socket carries. In v1 there is no `authenticated` role, so the browser connects with the **anon key** and never with a Privy token — handing Realtime a Privy token would both run as `anon` anyway and add an hourly disconnect for no benefit. That means `0013` 's private channels need a policy that lets `anon` read the sharded topics:

```
-- 0016_auth_principal.sql. Private channels authorise against realtime.messages.
-- The payloads are already sanitised (broadcast_comment returns early unless
-- status='visible') and carry nothing not already public through 0011, so anon read on
-- these three topic families is the same disclosure the REST path already permits.
CREATE POLICY realtime_public_topics ON realtime.messages FOR SELECT TO anon, authenticated
  USING (realtime.topic() ~ '^(story|coin|board):');
-- Nobody may WRITE a broadcast frame: every frame originates from a SECURITY DEFINER
-- trigger or from the pipeline on the service role. There is no INSERT policy, so a
-- browser cannot inject a fake `match` event and flip a Create button into a Buy.
```

The loop ships anyway, written and tested now, because phase 2 turns the token into a 15-minute Supabase JWT and the loop must already exist on that day. One socket, ref counted, one token source:

```
// apps/web/src/shared/realtime/client.ts
'use client';
import { browserClient } from '@shared/db';

/** Phase 1 returns the anon key (long-lived). Phase 2 returns a 15-minute minted
 * Supabase JWT. The loop below does not care which, and that is the point: the day the
 * token becomes short-lived, nothing else changes. */
type TokenSource = () => Promise<string>;

const REFRESH_MS = 50 * 60_000; // < Privy's ~1h and < any minted TTL we would choose.
let refs = 0;
let timer: ReturnType<typeof setInterval> | undefined;
let getToken: TokenSource | undefined;

async function pushToken(): Promise<void> {
  if (getToken === undefined) return;
  try {
    // setAuth re-sends access_token on EVERY joined channel; it does not re-subscribe,
    // so no frames are lost and no re-authorisation round trip is paid per channel.
    // Δ VERIFY: whether setAuth returns a promise in the installed supabase-js. Await
    // works either way; a non-promise resolves immediately.
    await browserClient().realtime.setAuth(await getToken());
  } catch {
    // Never leave a stale token in place silently. A failed refresh retries in 30s and
    // surfaces as the amber `polling fallback` rail state if it keeps failing.
    setTimeout(() => void pushToken(), 30_000);
  }
}

/** A pure interval is NOT sufficient. A laptop asleep for three hours fires its timers
 * late, by which time the socket is already closed by the server. These two listeners
 * are the difference between "works in a demo" and "works after lunch". */
function onWake() { void pushToken(); }

export function acquireRealtime(source: TokenSource) {
  getToken = source;
  if (refs++ === 0) {
    void pushToken();
    timer = setInterval(() => void pushToken(), REFRESH_MS);
    document.addEventListener('visibilitychange', onWake);
    window.addEventListener('online', onWake);
  }
  return () => {
    if (--refs === 0) {
      clearInterval(timer);
      document.removeEventListener('visibilitychange', onWake);
      window.removeEventListener('online', onWake);
    }
  };
}
```

```
// apps/web/src/shared/realtime/__tests__/refresh.test.ts
it('re-mints before the token can expire', async () => {
  vi.useFakeTimers();
  const source = vi.fn().mockResolvedValue('t');
  acquireRealtime(source);
  await vi.advanceTimersByTimeAsync(59 * 60_000);
  // 1 initial + 1 at 50 minutes. If this is ever 1, every user drops off at the hour.
  expect(source).toHaveBeenCalledTimes(2);
});
```

5.7 Rate limiting

Two layers, because they catch different things and neither subsumes the other.

Edge, before a function holding a service key starts. This is what bounds the metered-vendor bill: an `/api/order` that reaches the handler has already spent a Jupiter call. `checkRateLimit` needs a `request` object, which is the concrete reason every money endpoint is a Route Handler and never a Server Action.

```
// apps/web/src/features/wallet/server/limit.ts
import 'server-only';
import { checkRateLimit } from '@vercel/firewall';

/**  $\triangle$  VERIFY: the return shape of @vercel/firewall's checkRateLimit and that a rule with
 * this id exists in the project firewall config. A missing rule must not read as
 * "allowed" — hence the fail-closed default below. */
export async function edgeLimit(req: Request, id: string, key: string): Promise<Response | null> {
  try {
    const { rateLimited } = await checkRateLimit(id, { request: req, rateLimitKey: key });
    return rateLimited ? new Response(null, { status: 429, headers: { 'retry-after': '10' } }) : null;
  } catch {
    // A money endpoint whose limiter is unreachable is CLOSED, not open. This is the
    // rule the RugCheck defect broke in the other direction.
    return new Response(null, { status: 503 });
  }
}
```

Database, because edge counters are regional. Vercel's rate limit is enforced per region; a client that spreads requests across regions — trivially, by resolving the anycast address from several networks — gets N times the cap. A global cap has to live somewhere global, and the only globally consistent thing in this architecture is Postgres. It also survives two tabs, two devices and a retry spanning a deploy.

```
-- 0016_auth_principal.sql
CREATE TABLE IF NOT EXISTS insidor.rate_bucket (
  scope      text      NOT NULL,
  key        text      NOT NULL,
  window_start timestampz NOT NULL,
  n          integer    NOT NULL DEFAULT 0,
  PRIMARY KEY (scope, key, window_start)
);

-- Fixed windows via date_bin: no read-modify-write race, one atomic upsert, and the
-- count is returned by the same statement that increments it.
CREATE OR REPLACE FUNCTION insidor.rate_take(p_scope text, p_key text, p_limit integer, p_window interval)
RETURNS void LANGUAGE plpgsql AS $$
DECLARE v_n integer;
BEGIN
  INSERT INTO insidor.rate_bucket AS b (scope, key, window_start, n)
  VALUES (p_scope, p_key, date_bin(p_window, now(), timestampz 'epoch'), 1)
  ON CONFLICT (scope, key, window_start) DO UPDATE SET n = b.n + 1
  RETURNING b.n INTO v_n;
  IF v_n > p_limit THEN
    -- A DISTINCT SQLSTATE. 23514 (check_violation) would be mapped to 422 by the route,
    -- and a rate limit reported as "invalid input" is a bug report we cannot action.
    RAISE EXCEPTION 'rate limit: % per % for %', p_limit, p_window, p_scope USING ERRCODE = 'IR429';
  END IF;
END $$;

CREATE OR REPLACE FUNCTION insidor.launch_rate_limit() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  PERFORM insidor.rate_take('launch', NEW.signer_wallet, 3, interval '1 hour');
  RETURN NEW;
END $$;
DROP TRIGGER IF EXISTS launch_rate_limit ON public.launch;
CREATE TRIGGER launch_rate_limit BEFORE INSERT ON public.launch
  FOR EACH ROW EXECUTE FUNCTION insidor.launch_rate_limit();

CREATE OR REPLACE FUNCTION insidor.trade_rate_limit() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  PERFORM insidor.rate_take('order', NEW.wallet, 20, interval '1 minute');
  RETURN NEW;
END $$;
DROP TRIGGER IF EXISTS trade_rate_limit ON public.trade;
CREATE TRIGGER trade_rate_limit BEFORE INSERT ON public.trade
  FOR EACH ROW EXECUTE FUNCTION insidor.trade_rate_limit();

-- A janitor, because an unbounded counter table is a slow outage.
DELETE FROM insidor.rate_bucket WHERE window_start < now() - interval '1 day';
```

The comment limit already exists in `0007` (`comment_speed_limit` , 3 per wallet per 60s) and stays as written — it counts real rows on an existing index and therefore cannot disagree with the table it protects.

```
// apps/web/src/features/discussion/server/pg-error.ts
const MAP: Record<string, number> = {
  '23505': 409, // unique violation – a duplicate submission, never a retry
  '23514': 422, // check violation – address gate, body length, tier
  'IR429': 429, // rate_take
  '42501': 403, // insufficient privilege – a policy or grant, not a user error
};
export function pgStatus(e: { code?: string; message: string }): Response {
  const status = MAP[e.code ?? ''] ?? 500;
  return Response.json({ error: e.code ?? 'unknown', message: status === 500 ? 'server error' : e.message }, { status });
}
```

5.8 Wallet-signed actions

Buys and mints are signed in the user's wallet. The server's job is to be unable to believe a client.

The server must never trust: that a swap succeeded, the output amount, the signature, the wallet address, the fee bps, the SOL/USD price, the mint, the decimals, the story attribution, or the confirmed-coin count the page was rendered with. Every one of those is either read from a vendor response server-side or read from the chain.

Can a client claim a swap succeeded? It can POST anything it likes. It cannot make that claim load-bearing, for four structural reasons.

1. **The signature does not come from the client.** `/api/execute` receives the *signed transaction bytes* and calls Jupiter's `/execute` itself. The signature in `trade.signature` is the one Jupiter returns. There is no code path that accepts a signature as an input field.
2. **The transaction we broadcast is the transaction we issued.** `/api/order` stores `sha256(message_bytes)` on the trade row at `ORDERING`; `/api/execute` recomputes it from the submitted bytes and refuses on mismatch. Without that check we are a willing relay for any transaction a compromised page hands us, under a UI that says "Buy".
3. **holding is written from a chain read, never from a response body.** The confirm step fetches the transaction and derives the amount from the token-balance delta for the *bound wallet and the requested mint*. A missing or ambiguous delta writes nothing and leaves the trade `submitted` — absent is not zero.
4. **Nobody but service_role can insert into holding or trade** (5.4), and `trade.request_id` and `trade.signature` are both `UNIQUE`, so a replay is a `23505` and therefore a **409**, never a second fill.

```
// apps/web/src/features/trading/server/execute.handler.ts (the trust boundary only)
import { createHash } from 'node:crypto';

const bytes = Buffer.from(input.signedTransactionBase64, 'base64');
const tx = VersionedTransaction.deserialize(bytes);

// (a) It is the transaction we issued for THIS request_id.
const digest = createHash('sha256').update(tx.message.serialize()).digest('hex');
if (digest !== trade.order_message_sha256) return json({ error: 'transaction_mismatch' }, 409);

// (b) It is signed by the wallet we bound to this principal – not by whoever the
//     client says. Fee payer is staticAccountKeys[0] by construction.
const feePayer = tx.message.staticAccountKeys[0]!;
if (feePayer.toBase58() !== principal.wallet) return json({ error: 'wrong_signer' }, 403);
if (!nacl.sign.detached.verify(tx.message.serialize(), tx.signatures[0]!, feePayer.toBytes())) {
  return json({ error: 'bad_signature' }, 400);
}

// (c) WE broadcast. The signature is Jupiter's answer, not the client's assertion.
const res = await jupiter.execute({ signedTransaction: input.signedTransactionBase64, requestId: trade.request_id });
await db.from('trade').update({ status: 'submitted', signature: res.signature, submitted_at: now }).eq('id', trade.id);
```

```
// apps/web/src/features/trading/server/confirm.ts – the ONLY writer of a holding row.
const tx = await helius.getTransaction(signature, { maxSupportedTransactionVersion: 0 });
if (tx === null) return { state: 'still_confirming' }; // NOT failed. NOT zero.
if (tx.meta?.err !== null) return { state: 'reverted' };

const pre = tx.meta.preTokenBalances?.find((b) => b.mint === trade.mint && b.owner === trade.wallet);
const post = tx.meta.postTokenBalances?.find((b) => b.mint === trade.mint && b.owner === trade.wallet);
if (post === undefined) return { state: 'unreadable' }; // renders `–`, never 0
const deltaRaw = BigInt(post.uiTokenAmount.amount) - BigInt(pre?.uiTokenAmount.amount ?? '0');
if (deltaRaw ≤ 0n) return { state: 'unreadable' };
// out_amount_raw is deltaRaw. The client's number, if it sent one, was never read.
```

Launches carry the same shape plus one rule the browser cannot enforce: the mint keypair is generated and persisted encrypted **before** the first signature (`0008` , `launch.mint_keypair_enc` , frozen), so a retry reuses it and a late confirmation fails with "account already in use" instead of minting twice. `/api/launch/confirm` re-runs `count(confirmed) = 0` server-side at submit, so nobody pays gas to duplicate a coin the page was already hiding, and the tier gate is a composite foreign key (`launch_requires_normal_tier`) rather than a check anyone can skip.

5.9 Secrets

SECRET	LIVES IN	READ BY	ROTATION
<code>NEXT_PUBLIC_PRIVY_APP_ID</code>	Vercel, per-environment	browser + web server	Not secret. Changes only with a new Privy app.
<code>PRIVY_APP_SECRET</code>	Vercel (Production / Preview scoped separately)	<code>features/wallet/server</code> only	Privy dashboard rotate → set <code>_NEXT</code> → deploy → promote → delete old. Quarterly.
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Vercel + Fly	<code>packages/db/src/service.ts</code> , pipeline	Supabase issues a second key; deploy both apps; revoke. Full audit of <code>.next/static</code> after (5.5 grep).
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Vercel	browser	Rotates with the project JWT secret; forces every open socket to reconnect. Do it in a maintenance window or the whole board reloads.
<code>SUPABASE_JWT_SIGNING_KEY</code> (phase 2)	Vercel	the token endpoint only	Supabase supports two active keys; overlap is the rotation.
<code>LAUNCH_KEYPAIR_ENC_KEY</code>	Vercel	<code>features/launch/server</code>	Cannot be rotated naively — it decrypts historical <code>mint_keypair_enc</code> . Rotation is re-encrypt-then-swap, or it destroys the ability to retry any in-flight launch.
<code>DEVICE_COOKIE_SECRET</code>	Vercel	watchlist route	Rotate freely; the cost is anonymous device follows becoming unclaimable. Two-key verify.
<code>JUPITER_API_KEY</code> , <code>HELIUS_API_KEY</code>	Vercel + Fly	adapters	Quarterly, rehearsed on Helius first (open item 18).
<code>MIGRATOR_DATABASE_URL</code>	GitHub Environment <code>production</code> only	<code>release.yml</code>	With the <code>migrator</code> role password.

`packages/env` is the only module permitted to read `process.env` , with no defaults and no fallbacks, and no secret is ever scoped to "All Environments" in Vercel — `scripts/check-vercel-env-scopes.mjs` fails the PR. The rotation column is new; open item 18 recorded that key rotation was unaddressed, and the answer is the two-key overlap pattern everywhere except `LAUNCH_KEYPAIR_ENC_KEY` , which is called out because rotating it the obvious way silently orphans every prepared launch.

Anonymous device follows are a real principal with no identity, so the `device_id` is not a client-chosen string:

```
// apps/web/src/features/watchlist/server/device.ts
import { createHmac, timingSafeEqual } from 'node:crypto';
const sign = (id: string) => createHmac('sha256', serverEnv.DEVICE_COOKIE_SECRET).update(id).digest('base64url');

export function readDevice(req: Request): string | null {
  const raw = cookieFrom(req, 'idv'); // "<uuid>.<mac>"
  const [id, mac] = raw?.split('.') ?? [];
  if (id === undefined || mac === undefined) return null;
  const want = Buffer.from(sign(id)), got = Buffer.from(mac);
  return want.length === got.length && timingSafeEqual(want, got) ? id : null;
}
```

Without the MAC, `follow_device_id_shape` in `0007` accepts any 16–64 hex string and a scraper can enumerate other devices' watchlists. HttpOnly, `SameSite=Lax` , 400 days.

5.10 Abuse

A wallet costs a fraction of a cent, so wallet-gating is not sybil resistance and is not claimed as any. The defence is that the two things worth attacking are structurally expensive or structurally unreachable.

The ranking has no user-supplied input at all. `PostHeat` is arithmetic over engagement counts on X and TikTok; `CoinHeat` is arithmetic over Jupiter’s market data. Follows, watchlists, comments and clicks appear in **no term of**

either formula, and story attachment is a filter and a tiebreak rather than a multiplier — deliberately, so that gaming the matcher, the 0.97-precision-critical component, buys nothing. There is therefore no sybil attack on the board, because there is no input to sybil. Any future proposal to add an engagement term to the ranking reopens this section; that is the review trigger.

Match reports are advisory, not actuating. A report (5/day per principal) enqueues a matcher re-run and never retracts a match. Retraction is the matcher’s own verdict. A hundred wallets reporting a correct match achieve one re-run.

Comments are shaped so volume does not pay. Three defences already in the schema, one new.

- The discussion sort key is `(story_id, snap_views, created_at)` — earliest means *written when the story was smallest*. That ordering cannot be farmed by posting more; it can only be earned by being early on a story that later mattered, which requires being right. Volume moves nobody up.
- The position stamp is a server-side chain read. A `holds` badge is a claim about the author's money and `comment_position_holds_needs_amount` refuses the row without the lamports and the mint, so a credible-looking comment costs a real position.
- The address gate blocks every base58 run of 32–44 characters, including this story's own mints. The payload of comment spam on a memecoin terminal is a contract address; there is nothing a user can say with one that they cannot say with `$KANG`, which is already auto-linked.
- **New, and the only thing here that raises cost rather than removing reward:** a principal whose wallet has no confirmed `trade` and no on-chain history is capped at one comment per story per hour and its rows enter `status = 'held'` above a per-story volume threshold, promoted by the moderation ladder rather than blocked. Held is invisible to everyone but the author (`comment_author_read`), so a farm sees its own comments and nobody else does — a spammer who cannot observe the failure does not iterate against it.

The sybil entry point is account creation, not commenting. Every new DID is a Privy MAU we pay for, so `requirePrincipal` 's first-sight upsert carries its own edge limit keyed on IP with a lower per-ASN ceiling, and a first-sight rate above baseline writes `ops_event(kind='auth_signup_burst', severity='page')`. That is the one abuse signal that costs money the moment it starts, so it pages rather than warns.

Everything auth rejects is counted. `verifyPrivyToken` failures increment an `ops_sli_sample` by reason. A step change in `sub_not_a_privy_did` or `privy_user_http_*` is either an attack or a Privy API change, and the two are indistinguishable from the application’s point of view — which is exactly why the alarm is on the rate rather than on any interpretation of it.

6. THE PIPELINE SERVICE

`apps/pipeline` is one Node process. It runs fifteen stages and holds two websocket-shaped clocks. Eleven of the stages are the funnel — the verbs the product is described in — and four are infrastructure. The funnel verbs are the vocabulary; the stage ids below are what appears in `ops_stage_run.stage` , `ops_stage_expected.stage` and the Inngest function id, and nothing else may name a stage.

VERB	STAGE ID	CADENCE	<code>ROWS_OUT</code> MEANS	FUNNEL COUNTER
arrive	<code>ingest-x</code>	180 s	posts written	<code>ops_funnel.arrived</code>
arrive	<code>ingest-tiktok</code>	600 s	posts written	<code>ops_funnel.arrived</code>
admit	<i>(inside ingest)</i>	—	posts past the ~15-like floor	<code>ops_funnel.admitted</code>
track	<code>snapshot</code>	60 s	<code>post_snapshot</code> + <code>coin_snapshot</code> rows	<code>ops_funnel.tracked</code>
trigger	<i>(inside snapshot)</i>	—	posts raised to <code>tracking_tier ≥ 2</code>	<code>ops_funnel.triggered</code>
qualify	<code>score</code>	120 s	posts scored	—
cluster	<code>cluster</code>	45 s	stories touched	<code>ops_funnel.clustered</code>
promote	<code>promote</code>	45 s, after cluster	<code>story.promoted_at</code> writes	<code>ops_funnel.promoted</code>
name	<code>name</code>	60 s	<code>story.title</code> writes + <code>story_ticker</code> rows	—
resolve	<code>resolve-coins</code>	60 s	<code>coin_match</code> rows	—
rank	<code>rank-commit</code>	20 s	<code>board_state</code> rows committed	—
deliver	<code>deliver</code>	30 s	<code>notification</code> rows	—
—	<code>clocks-ct</code>	120 s	<code>ct_mention</code> rows	—
—	<code>clocks-mint</code>	5 s (drain)	<code>coin</code> upserts from the socket	—
—	<code>outcomes</code>	nightly	<code>story_outcome</code> rows	—
—	<code>watchdog</code>	60 s	SLIs evaluated	—

`0010` seeded `ops_stage_expected` with three names from the old build (`ingest` , `snapshotter` , `trends`) and is missing seven. `0014` reconciles the roster; migrations are immutable, so the fix is a forward `INSERT ... ON CONFLICT DO UPDATE` plus a `DELETE` of the three dead ids, not an edit to `0010` .

6.1The execution model

One long-lived container on Fly.io, with Inngest as the durable scheduler inside it. The container is the substrate; Inngest is the clock, the singleton and the retry ledger. Not two models, not a lockfile — one process, one scheduler, one place a stage can start.

The requirement that eliminates two of the four candidates immediately: **the mint clock is a subscription whose silence is the signal.** A PumpPortal websocket must stay open across minutes, and a disconnect must open a `sensor_gap` row *before* reconnecting, because a gap means lost mints and a lost mint means `story_clock.unmeasurable` rather than a fabricated lead time. There is no way to hold that connection in a function that is billed per invocation and killed at the response.

Against Vercel Cron with resumable progress. Vercel's hard ceiling is per-invocation duration (60 s on the default runtime, higher on paid Fluid tiers — △ verify the current maximum for our plan before relying on any figure). "Resumable progress" means every stage grows a cursor table, a partial-work protocol and a re-entry path, and the 402 bug repeats in a new shape: an invocation that dies at the ceiling looks identical to one that finished with nothing to do. Cron also gives no singleton — two overlapping schedules or one retried invocation both write `coin_match` — no backpressure, no retry semantics, and no first-class "ran and produced nothing". That last absence is exactly the two-day outage.

Against a queue with workers (SQS/BullMQ/pg-boss + a worker pool). A queue is the right shape for fan-out over independent units. Ours is fifteen singleton cadences with hard ordering (`cluster` → `promote` → `name` → `resolve-coins`) and per-stage spend caps. On a queue you write the scheduler anyway — something has to enqueue on a cron — then the singleton (a visibility-timeout race is not a lock), the DLQ, the retry policy and the observability. That is Inngest, hand-rolled, with a Redis to operate. `pg-boss` on the existing Postgres is the closest honest alternative and stays on the table if Inngest becomes a cost or availability problem; §6.2 is written so the scheduler is swappable in one file (`src/inngest/functions.ts`).

Against a durable-execution framework — Temporal, Restate. Temporal answers a different problem: multi-day workflows with human-in-the-loop steps and complex compensation. Ours are 15-second stages against a database that is already the source of truth. A Temporal cluster plus workers plus the determinism constraint on workflow code is a second distributed system to operate. Inngest gives the two properties we need from that category — a durable `step` boundary and a global concurrency key — from an HTTP handler mounted in the process we already have.

Against a bare long-running container (`setInterval` , no scheduler). This is what the old build ran locally, and it is why the PID lockfile existed and was never called. A bare loop has no cross-host singleton, no retry with backoff and no run history that survives the process. Two machines run in production for canary deploys; two `setInterval` loops double-ingest, which is double spend on the largest cost line in the system.

```
docker-entrypoint.sh
└─ node dist/preflight.js      # exits non-zero → health check never passes → Fly rolls back
└─ node dist/main.js
    ├── Inngest serve handler  (all 15 stages)
    ├── GET /health/live       (process is up)
    ├── GET /health/ready      (preflight passed AND a stage ran productively recently)
    └─ PumpPortal websocket    (in-process; not a stage)
```

Preflight, in order, each fatal:

- `packages/env` parses. No defaults, no fallbacks — `process.env.X ?? 'something'` is the same shape as the token lookup that returned age 0.
- `assertEnvironment()` — `ops_environment.name = APP_ENV` . A production connection string in a laptop `.env.local` crashes here.
- `max(version) FROM insidor.schema_migrations ≥ REQUIRED_SCHEMA_VERSION` .
- `select count(distinct model) from post_embedding` must be ≤ 1 (open item 17).
- One unmetered ping per vendor, recorded in `ops_event` at `info` . A 401 here fails the boot rather than pausing a stage four hours later.

6.2 The stage contract

One interface. Everything a stage may declare about itself is declarative — cadence, budget, timeout, lock — so the runner can enforce it without the stage's cooperation.

```
// apps/pipeline/src/lib/stage.ts
import type { SupabaseClient } from '@supabase/supabase-js';
import type { Database } from '@insidor/contracts/generated/database.types';

export type StageName =
  | 'ingest-x' | 'ingest-tiktok' | 'snapshot' | 'score'
  | 'cluster' | 'promote' | 'name' | 'resolve-coins'
  | 'rank-commit' | 'deliver'
  | 'clocks-ct' | 'clocks-mint' | 'outcomes' | 'watchdog' | 'trending';

export type VendorSource = 'twitterapi' | 'apify' | 'anthropic' | 'gemini'
  | 'jupiter' | 'helius' | 'rugcheck' | 'dexscreener';

export type StageResult = {
  /** Each stage sets this from its OWN output metric. `rows_out = 0, ok = true` is
   * legal and meaningful — ran correctly, nothing to do — and it is precisely what
   * the stall SLI counts. Never set it to "items considered". */
  rowsOut: number;
  rowsIn?: number;
  apiReads?: number;
  costUsd?: number;
  /** Written to ops_stage_run.detail. Cheap to add, and the only forensics that exist. */
  detail?: Record<string, unknown>;
};

export type StageCtx = {
  db: SupabaseClient<Database>; // service role
  budget: BudgetHandle; // $6.4 — the ONLY route to a metered call
  log: Logger; // structured JSON, carries stage + runId
  /** Deadline-aware. Every stage checks it between units of work and returns what it
   * has; the runner never has to kill anything mid-write. */
  deadline: AbortSignal;
  runId: number;
  now: Date; // injected — model code never calls Date.now()
};

export type StageDef = {
  name: StageName;
  /** Cron or interval. The single source; ops_stage_expected.interval_seconds is
   * asserted equal to this at preflight, so the SLI and the schedule cannot drift. */
  schedule: { cron: string } | { everySeconds: number };
  /** Wall-clock ceiling. Enforced by AbortSignal, then by Inngest, then by Fly. */
  timeoutMs: number;
  /** Vendors this stage is permitted to spend against. budget.fetch() to any other
   * source throws `vendor_contract` before the request leaves the process. */
  sources: readonly VendorSource[];
  /** Attempts, for retryable classes only. Terminal classes never retry. */
  retries: number;
  run: (ctx: StageCtx) => Promise<StageResult>;
};
```

Six guarantees follow from the runner rather than from discipline:

- **The row is opened before the work starts.** A stage killed mid-run leaves `ok = false, finished_at IS NULL`, which the watchdog reads as a crash, not as silence.
- **rows_out is always recorded** — on success, on failure, on timeout, on zero.
- **Idempotency is by natural key, on every write.** `post(platform, platform_post_id)`, `coin(mint)`, `coin_match(story_id, mint)`, `coin_snapshot(mint, captured_at)`, `ct_mention(handle, platform_post_id)`, `notification(dedupe_key)`. A retry after a partial failure converges. The one non-idempotent operation is PROMOTE, and `story.promoted_at` is frozen by trigger (`insidor.freeze_columns`), so a second attempt raises rather than moving the lead-time clock.
- **Checkpointing is a watermark, not a cursor into a batch.** A stage advances its watermark only over the range it fully processed. `clocks-ct` below is the reference implementation: `sensor_heartbeat.observed_through` moves to the oldest fully-scanned boundary, never to the newest row seen, because `story_clock` has a CHECK requiring both watermarks to be past `promoted_at`, and an optimistic watermark converts an unmeasurable lead into a published one.
- **Partial failure keeps its partial work.** Writes flush per batch; the result carries what landed. A stage that fails on batch 7 of 10 reports `rowsOut` for 6 and an error.
- **Nothing catches broadly.** `classify()` (\$6.4) is total over a closed union; an unmatched error is `internal`, which is retryable *and* pages.

```

export async function runStage(def: StageDef): Promise<StageResult> {
  const startedAt = new Date();
  const gate = await checkPaused(def.name);           // §6.4 – 402/401 pause
  if (gate.paused) {
    await opsEvent('stage_paused_skip', 'warn', def.name, { reason: gate.reason });
    return { rowsOut: 0 };
  }

  const lease = await acquireLease(def.name, HOLDER_ID, def.timeoutMs + 30_000);
  if (!lease) return { rowsOut: 0 };                  // §6.3 – someone else holds it

  const runId = await openRun(def.name, startedAt);   // ops_stage_run, ok = false
  const ac = new AbortController();
  const timer = setTimeout(() => ac.abort(new StageTimeout(def.name)), def.timeoutMs);
  const budget = await budgetFor(def.name, def.sources, runId);

  try {
    const result = await def.run({
      db: service, budget, log: logger(def.name, runId),
      deadline: ac.signal, runId, now: startedAt,
    });
    await closeRun(runId, { ok: true, finishedAt: new Date(),
      apiReads: budget.reads, costUsd: budget.costUsd, ...result });
    return result;
  } catch (err) {
    const e = classify(err);
    await closeRun(runId, { ok: false, finishedAt: new Date(), rowsOut: 0,
      apiReads: budget.reads, costUsd: budget.costUsd,
      errorCode: e.code, errorDetail: e.detail });
    if (e.pauseStage) await pauseStage(def.name, e);   // loud, and stops the bleeding
    await opsEvent(`stage_${e.code}`, e.severity, def.name, { detail: e.detail });
    if (e.retryable) throw err;                       // Inngest retries with backoff
    return { rowsOut: 0 };
  } finally {
    clearTimeout(timer);
    await releaseLease(def.name, lease.fence);
    await budget.flush();                             // ops_spend is written even on throw
  }
}

```

One stage, fully implemented — clocks-ct

Clock B. ~300 handles, every two minutes, `t_ct = min(posted_at)` over the mentions it finds. It is the stage worth writing out because it exercises every clause of the contract: a metered vendor, a per-cycle budget, a watermark checkpoint, natural-key idempotency, a deadline, and the rule that a sensor may never claim to have seen further than it scanned.

```

// apps/pipeline/src/stages/clocks-ct/index.ts
import { z } from 'zod';
import type { StageDef, StageCtx, StageResult } from '../..lib/stage';
import { CT_HANDLES } from './io/roster'; // curated, versioned, in git
import { extractTargets } from './model/extract'; // PURE: text → {cashtags, mints, entities}

const HANDLES_PER_CALL = 20; // twitterapi.io advanced-search OR query
const COST_PER_POST_USD = 0.00015;

/** Δ VERIFY: twitterapi.io `/twitter/tweet/advanced_search` query syntax, its
 * `has_next_page`/`next_cursor` field names, and whether `since_time` is unix
 * seconds. Every one of these is asserted by the zod schema below, so a change
 * is `vendor_contract` (terminal, pages) and never a silent empty result. */
const SearchResponse = z.object({
  tweets: z.array(z.object({
    id: z.string(),
    url: z.string().url(),
    text: z.string(),
    createdAt: z.string(),
    author: z.object({ userName: z.string() }),
  })),
  has_next_page: z.boolean(),
  next_cursor: z.string().nullable(),
});

export const clocksCt: StageDef = {
  name: 'clocks-ct',
  schedule: { everySeconds: 120 },
  timeoutMs: 90_000, // < the 120s cadence, deliberately
  sources: ['twitterapi'],
  retries: 2,
  run: pollCryptoTwitter,
};

async function pollCryptoTwitter(ctx: StageCtx): Promise<StageResult> {
  const { db, budget, log, deadline, now } = ctx;

  const { data: hb } = await db.from('sensor_heartbeat')
    .select('observed_through').eq('sensor', 'ct_poll').single();

  // 90s of overlap. Duplicates are free (UNIQUE (handle, platform_post_id));
  // a hole is a broken earliness claim.
  const since = new Date((hb?.observed_through ?? new Date(now.getTime() - 6 * 3600_000))
    .valueOf() - 90_000);

  const batches = chunk(CT_HANDLES, HANDLES_PER_CALL);
  // THE checkpoint invariant: the watermark may only advance to the oldest boundary
  // that every batch reached. One failed batch holds the whole sensor back.
  let scannedThrough: Date | null = null;
  let written = 0;
  let posts = 0;
  const failed: string[] = [];

  for (const batch of batches) {
    if (deadline.aborted) { log.warn('deadline', { done: batches.indexOf(batch) }); break; }

    const q = batch.map((h) ⇒ `from:${h}`).join(' OR ');
    let cursor: string | null = null;
    const batchStart = new Date();
    const rows: CtMentionInsert[] = [];

    try {
      do {
        // Reserves BEFORE the call. A false return is budget_exhausted: terminal.
        await budget.reserve('twitterapi', 20, 20 * COST_PER_POST_USD);
        const raw = await budget.fetch('twitterapi', '/twitter/tweet/advanced_search', {
          query: { query: q, queryType: 'Latest', since_time: Math.floor(+since / 1000),
            ...(cursor ? { cursor } : {}) },
          signal: deadline,
        });
        const page = SearchResponse.parse(raw); // parse failure ⇒ vendor_contract
        posts += page.tweets.length;

        for (const t of page.tweets) {
          const targets = extractTargets(t.text);
          if (!targets.cashtags.length && !targets.mints.length && !targets.entities.length) continue;
          const link = await resolveTarget(db, targets); // → {storyId?, mint?, matchedOn}
          if (!link) continue; // ct_mention_targets_something
          rows.push({
            story_id: link.storyId ?? null,
            mint: link.mint ?? null,

```

```

        handle: t.author.userName.toLowerCase(),
        platform_post_id: t.id,
        posted_at: new Date(t.createdAt).toISOString(),
        observed_at: new Date().toISOString(),
        permalink: t.url, // the proof travels with the claim
        matched_on: link.matchedOn,
    });
}

cursor = page.has_next_page ? page.next_cursor : null;
} while (cursor && !deadline.aborted);

if (rows.length) {
    const { error, count } = await db.from('ct_mention')
        .upsert(rows, { onConflict: 'handle,platform_post_id', ignoreDuplicates: true,
            count: 'exact' });

    if (error) throw error; // constraint_violation is a WIN, and loud
    written += count ?? 0;
}

scannedThrough = scannedThrough === null || batchStart < scannedThrough
    ? batchStart : scannedThrough;
} catch (err) {
    failed.push(batch[0]);
    if (isTerminal(err)) throw err; // 402/401/contract: stop the whole cycle
    log.warn('batch_failed', { head: batch[0], err: String(err) });

    // scannedThrough is NOT advanced for this batch. The sensor stays honest.
}
}

// Only advance the watermark if every batch reported. A partial sweep leaves the
// watermark where it was, and story_clock's CHECK then refuses to record a lead
// time measured by a sensor that had not caught up.
const complete = failed.length === 0 && !deadline.aborted && scannedThrough === null;
await db.from('sensor_heartbeat').upsert({
    sensor: 'ct_poll',
    last_beat_at: new Date().toISOString(),
    observed_through: (complete ? scannedThrough! : (hb?.observed_through ?? since)).toISOString(),
    detail: { batches: batches.length, failed: failed.length, posts },
});

if (!complete) await openGapIfNeeded('ct_poll', 'supervisor', { failed });
else await closeGapIfOpen('ct_poll');

return { rowsOut: written, rowsIn: posts, detail: { batches: batches.length, failed } };
}

```

The load-bearing line is the second-to-last block. `last_beat_at` and `observed_through` are two different facts — *alive* and *scanned through here* — and conflating them is how a reconnected-but-empty sensor certifies a lead time it never measured.

6.3 Scheduling and concurrency

What may overlap. Different stages may run concurrently unless one writes what another reads within the same tick. Every stage is a singleton against itself, always.

STAGE	MAY OVERLAP WITH	MUST NOT OVERLAP WITH	WHY
<code>ingest-x</code> , <code>ingest-tiktok</code>	everything	itself	double spend on the largest cost line
<code>snapshot</code>	ingest, score, clocks	itself	<code>post_snapshot(post_id, captured_at)</code> PK collisions and a corrupted EWMA
<code>score</code>	everything	itself	LLM spend
<code>cluster</code>	ingest, snapshot	itself, <code>promote</code> , <code>name</code>	centroid updates and <code>promoted_at</code> must see one consistent story set
<code>promote</code>	ingest, snapshot	itself, <code>cluster</code> , <code>resolve-coins</code>	promote → resolve is the ordering that makes a match attachable
<code>name</code>	ingest, snapshot	itself, <code>cluster</code>	writes <code>story.title</code> , <code>title_version</code>
<code>resolve-coins</code>	ingest, snapshot, score	itself, <code>promote</code>	sole writer of <code>coin_match</code>
<code>rank-commit</code>	everything	itself	sole writer of <code>board_state</code> ; <code>tick_seq</code> must be strictly monotonic
<code>deliver</code>	everything	itself, <code>resolve-coins</code>	fires on a 0 → ≥1 confirmed transition
<code>clocks-ct</code> , <code>clocks-mint</code>	everything	itself	watermark integrity
<code>watchdog</code>	everything	itself	—

Mutual exclusion between *different* stages is expressed as a shared Inngest concurrency key (`cluster` , `promote` and `name` share `key: '"story-write"'` , limit 1), not as ordering hope.

The lock. Not a PID file, not an in-process mutex, not a Redis SETNX we would have to operate. A **lease row in Postgres with a fencing token**, because Postgres is the one thing every host already shares and the one thing that is authoritative about the writes anyway. Session-scoped advisory locks were rejected: Supabase's pooler runs in transaction mode, where a session lock is held by whichever backend the pooler happened to hand out and is released at a moment unrelated to the caller.

```
-- 0014_environment_and_budget.sql
CREATE TABLE IF NOT EXISTS public.ops_stage_lease (
  stage      text PRIMARY KEY,
  holder     text NOT NULL,      -- fly machine id + pid
  fence      bigint NOT NULL,    -- strictly increasing; the anti-zombie token
  acquired_at timestamptz NOT NULL DEFAULT now(),
  expires_at timestamptz NOT NULL
);

CREATE OR REPLACE FUNCTION public.acquire_stage_lease(
  p_stage text, p_holder text, p_ttl_ms integer
) RETURNS bigint
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DECLARE v_fence bigint;
BEGIN
  -- One statement. The predicate that decides whether the lease is free is inside
  -- the same UPDATE that takes it, so two machines cannot both observe "free".
  INSERT INTO public.ops_stage_lease (stage, holder, fence, expires_at)
  VALUES (p_stage, p_holder, 1, now() + make_interval(secs => p_ttl_ms / 1000.0))
  ON CONFLICT (stage) DO UPDATE
    SET holder = EXCLUDED.holder,
        fence  = public.ops_stage_lease.fence + 1,
        acquired_at = now(),
        expires_at  = EXCLUDED.expires_at
  WHERE public.ops_stage_lease.expires_at < now()      -- expired: reclaimable
    OR public.ops_stage_lease.holder = EXCLUDED.holder -- our own re-entry
  RETURNING fence INTO v_fence;

  RETURN v_fence; -- NULL => held by someone else, still alive. Caller returns rows_out 0.
END $$;

CREATE OR REPLACE FUNCTION public.release_stage_lease(
  p_stage text, p_holder text, p_fence bigint
) RETURNS void
LANGUAGE sql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DELETE FROM public.ops_stage_lease
  WHERE stage = p_stage AND holder = p_holder AND fence = p_fence;
$$;
```

Three properties. A crashed holder’s lease expires — `ttl = timeoutMs + 30 s` , so it is always longer than the stage can legally run. A zombie that wakes after expiry holds a stale fence and its `release` deletes nothing, so it cannot free

a lease it no longer owns. And the lock is visible: `select * from ops_stage_lease` during an incident says which machine is holding what, which a PID file on a dead host does not.

Long stages (`outcomes`) renew mid-flight:

```
// apps/pipeline/src/lib/lease.ts
export function renewInBackground(stage: StageName, fence: bigint, ttlMs: number) {
  const t = setInterval(async () => {
    const { data } = await service.rpc('renew_stage_lease',
      { p_stage: stage, p_holder: HOLDER_ID, p_fence: Number(fence),
        p_ttl_ms: ttlMs });
    // Lost the lease while running: abort immediately rather than write behind a
    // holder that has already started. Split brain is loud, not silent.
    if (data !== true) throw new LeaseLost(stage, fence);
  }, Math.floor(ttlMs / 3));
  return () => clearInterval(t);
}
```

Inngest's `concurrency: { limit: 1, key: '<stage>' }` remains configured on every function. It is defence in depth and it is *not* the guarantee — the lease is, because it also covers a manual CLI invocation, a stuck-then-resumed function, and the two-machine canary window.

6.4 Budgets and the error taxonomy

The ledger

Ingestion is \$0.00015/post and is the largest cost line at every DAU tier. A cap in a config file is a cap a loop outruns between two reads, so **the reservation is the increment, in one statement**, against the tables that actually exist in `0010` .

```

-- 0014_environment_and_budget.sql

SELECT insidor.add_column('public.ops_budget_cap', 'cycle_cap_reads', 'integer NOT NULL DEFAULT 0');
SELECT insidor.add_column('public.ops_stage_expected', 'paused_until', 'timestampz');
SELECT insidor.add_column('public.ops_stage_expected', 'paused_reason', 'text');


CREATE TABLE IF NOT EXISTS public.ops_environment (
  id          boolean PRIMARY KEY DEFAULT true CHECK (id),    -- singleton
  name        text NOT NULL CHECK (name IN ('local','preview','staging','production')),
  network     text NOT NULL CHECK (network IN ('mainnet-beta','devnet')),
  ingest_mode text NOT NULL CHECK (ingest_mode IN ('off','replay','live')),
  trading_mode text NOT NULL CHECK (trading_mode IN ('off','simulate','live')),
  created_at  timestampz NOT NULL DEFAULT now(),
  CONSTRAINT env_live_trading_is_production_only
    CHECK (trading_mode <> 'live' OR (name = 'production' AND network = 'mainnet-beta')),
  CONSTRAINT env_live_ingest_is_deployed_only
    CHECK (ingest_mode <> 'live' OR name IN ('staging','production')),
  CONSTRAINT env_local_cannot_spend
    CHECK (name <> 'local' OR ingest_mode = 'replay')
);


-- Returns the granted read count. Partial grants are deliberate: a stage near the
-- cap does less work rather than failing, and learns exactly how much it may do.
CREATE OR REPLACE FUNCTION public.reserve_spend(
  p_source text, p_reads integer, p_cost_usd numeric
) RETURNS TABLE (granted integer, reason text)
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_catalog AS $$
DECLARE v_cap numeric; v_hard boolean; v_mode text; v_spent numeric; v_room integer;
BEGIN
  IF p_reads ≤ 0 THEN
    RAISE EXCEPTION 'reserve_spend: p_reads must be positive' USING ERRCODE = 'check_violation';
  END IF;

  SELECT ingest_mode INTO v_mode FROM public.ops_environment;
  IF v_mode <> 'live' THEN
    RAISE EXCEPTION
      'reserve_spend called with ingest_mode=% - this environment must not spend money. '
      'Set INGEST_MODE=replay and use fixtures.', v_mode USING ERRCODE = 'check_violation';
  END IF;

  SELECT daily_cap_usd, hard_stop INTO v_cap, v_hard
    FROM public.ops_budget_cap WHERE source = p_source;
  IF NOT FOUND THEN
    -- An unknown vendor has no cap, so it gets no money. Fail closed.
    RETURN QUERY SELECT 0, format('no ops_budget_cap row for source %s', p_source);
    RETURN;
  END IF;

  INSERT INTO public.ops_spend (utc_date, source)
  VALUES ((now() AT TIME ZONE 'utc')::date, p_source)
  ON CONFLICT (utc_date, source) DO NOTHING;

  -- The cap test is inside the UPDATE. Parallel stages cannot race past it.
  UPDATE public.ops_spend
    SET reads      = reads + p_reads,
        cost_usd   = cost_usd + p_cost_usd,
        updated_at = now()
  WHERE utc_date = (now() AT TIME ZONE 'utc')::date
    AND source    = p_source
    AND cost_usd + p_cost_usd ≤ v_cap
  RETURNING cost_usd INTO v_spent;

  IF v_spent IS NOT NULL THEN
    RETURN QUERY SELECT p_reads, NULL::text;
  ELSE
    RETURN QUERY SELECT 0, 'daily_cap_reached';    -- the caller STOPS. It never retries.
  END IF;
END $$;

```

`budget.ts` is the only module in the repository permitted to construct a fetch against a metered base URL — enforced by a dependency-cruiser rule, the same way the service key is.

```
// apps/pipeline/src/lib/budget.ts
export async function budgetFor(stage: StageName, sources: readonly VendorSource[], runId: number) {
  const caps = await loadCycleCaps(sources); // ops_budget_cap.cycle_cap_reads
  const used: Record<string, number> = {};
  let reads = 0, costUsd = 0;

  return {
    get reads() { return reads; },
    get costUsd() { return costUsd; },

    async reserve(source: VendorSource, n: number, usd: number) {
      if (!sources.includes(source)) {
        throw new StageError('vendor_contract',
          `stage ${stage} is not declared against source ${source}`);
      }
      // The per-cycle cap is in-process, and it is only sound because §6.3 guarantees
      // exactly one runner. It is the guard against a cadence change shipped without a
      // matching budget reduction — the usual cause of a daily cap breach.
      used[source] = (used[source] ?? 0) + n;
      if (caps[source] > 0 && used[source] > caps[source]) {
        throw new StageError('cycle_budget_exhausted',
          `${stage} asked for ${used[source]} ${source} reads; cycle cap is ${caps[source]}`);
      }
      const { data } = await service.rpc('reserve_spend',
        { p_source: source, p_reads: n, p_cost_usd: usd });
      const row = data?.[0];
      if (!row || row.granted < n) {
        throw new StageError('budget_exhausted', row?.reason ?? 'daily_cap_reached');
      }
      reads += n; costUsd += usd;
    },

    async fetch(source: VendorSource, path: string, init: VendorInit) { /* adapters/ */,
    async flush() { /* ops_stage_run.api_reads / cost_usd, already carried by runStage */,
  };
}
```

Caps, per environment: production is set by the founder; staging is \$0.30/day across all sources; preview and local cannot spend, because `ops_environment` refuses. **Never raise a cap to make an alert stop.** The usual cause is a cadence change without a matching `cycle_cap_reads` reduction, and that is a revert.

The taxonomy

The old build's retry helper retried on HTTP 429 and only 429. A 402 fell out of the `if`, threw, and vanished into a caller that logged nothing; posts stopped arriving; everything aged past the 240-minute display gate; nothing alerted for two days. The replacement is a **closed union with an explicit disposition per class, where the default is terminal, not retryable** — an unrecognised status must never be treated as transient.

```
// apps/pipeline/src/lib/errors.ts
export type StageErrorCode =
  // retryable — the next run probably succeeds
  | 'vendor_timeout' | 'vendor_5xx' | 'vendor_429' | 'db_conflict' | 'internal'
  // terminal — retrying costs money, or makes it worse
  | 'budget_exhausted' | 'cycle_budget_exhausted'
  | 'vendor_402' | 'vendor_auth' | 'vendor_contract' | 'vendor_4xx'
  | 'constraint_violation' | 'schema_assert_failed' | 'lease_lost' | 'stage_timeout';

export type Classified = {
  code: StageErrorCode;
  retryable: boolean;
  /** THE difference from the old build: a class that must stop the stage entirely
   * until a human clears it. Money and credentials are not transient conditions. */
  pauseStage: boolean;
  severity: 'info' | 'warn' | 'page';
  detail: string;
};

const DISPOSITION: Record<StageErrorCode, Omit<Classified, 'code' | 'detail'>> = {
  vendor_timeout: { retryable: true, pauseStage: false, severity: 'info' },
  vendor_5xx: { retryable: true, pauseStage: false, severity: 'info' },
  vendor_429: { retryable: true, pauseStage: false, severity: 'warn' },
  db_conflict: { retryable: true, pauseStage: false, severity: 'info' },
  internal: { retryable: true, pauseStage: false, severity: 'page' },
  budget_exhausted: { retryable: false, pauseStage: true, severity: 'warn' },
  cycle_budget_exhausted: { retryable: false, pauseStage: false, severity: 'warn' },
  vendor_402: { retryable: false, pauseStage: true, severity: 'page' },
  vendor_auth: { retryable: false, pauseStage: true, severity: 'page' },
  vendor_contract: { retryable: false, pauseStage: true, severity: 'page' },
  vendor_4xx: { retryable: false, pauseStage: false, severity: 'warn' },
  constraint_violation: { retryable: false, pauseStage: false, severity: 'warn' },
  schema_assert_failed: { retryable: false, pauseStage: true, severity: 'page' },
  lease_lost: { retryable: false, pauseStage: false, severity: 'warn' },
  stage_timeout: { retryable: true, pauseStage: false, severity: 'warn' },
};

export function fromHttpStatus(status: number): StageErrorCode {
  if (status === 402) return 'vendor_402';
  if (status === 401 || status === 403) return 'vendor_auth';
  if (status === 408 || status === 425) return 'vendor_timeout';
  if (status === 429) return 'vendor_429';
  if (status ≥ 500) return 'vendor_5xx';
  if (status ≥ 400) return 'vendor_4xx'; // terminal by DEFAULT. Never falls through.
  return 'vendor_contract'; // a non-error status reaching here is a bug
}

export function classify(err: unknown): Classified {
  const code =
    err instanceof StageError ? err.code
  : err instanceof HttpError ? fromHttpStatus(err.status)
  : err instanceof z.ZodError ? 'vendor_contract'
  : isPgError(err, '23') ? 'constraint_violation' // integrity_constraint_violation
  : isPgError(err, '40') ? 'db_conflict' // serialization / deadlock
  : err instanceof StageTimeout ? 'stage_timeout'
  : err instanceof LeaseLost ? 'lease_lost'
  : 'internal';
  return { code, ...DISPOSITION[code], detail: describe(err) };
}
```

Five entries carry the weight.

vendor_402 and vendor_auth pause the stage. `pauseStage` writes `ops_stage_expected.paused_until = now() + 24h` with the reason, emits an `ops_event` at `page`, and posts to `#insidor-page` naming the stage, the vendor and the status. The stage then *skips* each tick and logs `stage_paused_skip` at `warn`, so `ops_stage_run` keeps producing rows and the stall SLI keeps measuring. Pausing is loud, bounded, and cleared only by a human: `update ops_stage_expected set paused_until = null, paused_reason = null where stage = ...`. Retrying a 402 spends nothing and fixes nothing — a card needs topping up. Retrying a 401 burns rate limit against a key that has been rotated.

vendor_429 never widens concurrency. Exponential backoff with full jitter, capped at the stage's own cadence, so a stage never queues behind itself.

vendor_contract is a 200 whose *shape* we do not recognise. The worst vendor failure is not an outage; it is an endpoint that quietly stops returning a field and a caller that reads `?? null` and treats null as fine. That is the RugCheck defect exactly: `/report/summary` carries no authority fields, `mintAuthority = null` read as revoked, and every token got a green tick. Every adapter parses through zod; a parse failure is terminal and pages, never a

default. The schema's three-valued `authority_state` makes *storing* that lie impossible; `vendor_contract` makes *fetching* it loud.

`constraint_violation` is `warn`, and it is a **success**: the database refused a row the pipeline should not have built. The constraint name goes into `ops_stage_run.error_detail`, so `coin_match_confirmed_requires_evidence` in the log is a matcher bug caught before a user saw a Buy button. A rising rate is a regression; a flat low rate is the system working.

`internal` is retryable *and* pages. An unclassified error is a hole in the taxonomy, and a hole that fails quietly is how two days of silence happened.

6.5 The websocket clock

One persistent PumpPortal connection, owned by `main.ts`, not by a stage. It is a subscription whose silence is the signal, so it cannot be a cron.

```
// apps/pipeline/src/adapters/pumpportal.ts
const IDLE_TIMEOUT_MS = 45_000; // mainnet mints ~40/90s; 45s of silence is a dead socket
const MAX_BACKOFF_MS = 30_000;
const QUEUE_MAX = 5_000;

export function startMintStream(deps: Deps) {
  let attempt = 0, ws: WebSocket | null = null, idle: NodeJS.Timeout;
  const queue: MintEvent[] = [];

  const connect = () => {
    ws = new WebSocket('wss://pumpportal.fun/api/data');
    const connectedAt = new Date();

    ws.on('open', async () => {
      attempt = 0;
      ws!.send(JSON.stringify({ method: 'subscribeNewToken' }));
      // Close the gap only AFTER the subscription is acknowledged by traffic, not on
      // socket open. An open socket with no subscription is a silent hole.
      await deps.markConnected('mint_stream', connectedAt);
      bumpIdle();
    });

    ws.on('message', (buf) => {
      bumpIdle();
      const parsed = MintEvent.safeParse(JSON.parse(buf.toString()));
      if (!parsed.success) {
        // A shape change on the mint stream is a broken earliness claim, not a warning.
        void deps.opsEvent('mint_stream_contract', 'page', 'clocks-mint',
          { issues: parsed.error.issues.slice(0, 3) });
        return;
      }
      if (queue.length ≥ QUEUE_MAX) {
        // Backpressure IS a gap. Dropping events silently is the failure this table exists for.
        void deps.openGap('mint_stream', 'supervisor', { reason: 'queue_overflow' });
        return;
      }
      queue.push(parsed.data);
    });

    ws.on('close', async (code) => {
      clearTimeout(idle);
      // Opened BEFORE the reconnect attempt, so the ledger is complete even if the
      // reconnect succeeds instantly and even if alerting is down.
      await deps.openGap('mint_stream', 'supervisor', { code });
      const wait = Math.min(MAX_BACKOFF_MS, 500 * 2 ** attempt++) * (0.5 + Math.random() / 2);
      setTimeout(connect, wait);
    });

    ws.on('error', () => ws?.close());
  };

  const bumpIdle = () => {
    clearTimeout(idle);
    idle = setTimeout(() => ws?.close(4000, 'idle'), IDLE_TIMEOUT_MS);
  };

  connect();
  return { queue };
}
```

Gap detection.  VERIFY: PumpPortal's `subscribeNewToken` payload carries no sequence number and no server-side replay, so a gap cannot be detected from the stream itself — only from its boundaries. Three detectors, and all three write to the same `sensor_gap` table:

1. **Disconnect boundary.** `close` → `sensor_gap(sensor='mint_stream', started_at=now())` before the reconnect. Reconnect → `ended_at` .
2. **Idle timeout.** 45 s with no message closes the socket deliberately, which routes into detector 1. A socket that is open and receiving nothing is the failure that looks healthiest.
3. **Post-hoc reconciliation.** After a gap closes, `clocks-mint` backfills the window from the Helius mint index and writes any mint it finds with `coin.minted_at_source = 'helius_backfill'` , never `'pumpportal_create'` . The two sources are distinguishable forever, because a backfilled mint is evidence of a hole, not evidence of coverage.

The consequence is refusal, not estimation. Any story whose `[promoted_at, t_crypto]` window intersects an open or historical `sensor_gap` gets `story_clock.unmeasurable = true, unmeasurable_sensor, unmeasurable_gap_id` . The lead time is suppressed everywhere it would render, and `story_outcome` records `unmeasurable` , which `story_outcome_unmeasurable_has_no_lead` forces to carry no lead at all. Unmeasurable windows are never wins.

Coexistence with the scheduler. The socket never writes to the database from its message handler. It appends to a bounded in-process queue; `clocks-mint` — a real stage, with a lease, a run row and a `rows_out` — drains it every 5 s and upserts `coin` on `mint` . Three things follow: the socket cannot be blocked by database latency; `rows_out = 0` on the drainer is a *legible* fact the stall SLI can read; and if the machine dies with a full queue, the events were never acknowledged upstream, so the gap is the honest record of what was lost. The queue high-water mark goes into `sensor_heartbeat.detail` .

`observed_through` for `mint_stream` is the timestamp of the last successfully *drained* event, never the last received one — the same rule as `clocks-ct` .

6.6 Observability

`watchdog` runs every 60 s and writes one `ops_sli_sample` row per SLI. The principle is that **absence and zero must both breach**, because the outage that happened produced neither an error nor a metric. `ops_sli_sample` 's `CHECK ((value IS NULL) = (state = 'unknown'))` means an SLI that cannot be evaluated is `unknown` — a distinct colour on `/ops` , never green.

SLI-2 • `stage_output_stall` . This is the one that catches the two-day outage inside three cycles. It is driven from `ops_stage_expected` , not from `ops_stage_run` , so a stage that is dead and logging nothing scores 999 rather than scoring nothing at all.

```
WITH last_productive AS (  
  SELECT stage, max(started_at) AS at  
    FROM public.ops_stage_run  
   WHERE rows_out > 0 -- OUTPUT, not "ran". A heartbeat ticked  
   GROUP BY stage -- happily through all 48 hours of the outage.  
)  
SELECT  
  e.stage,  
  e.interval_seconds,  
  lp.at AS last_productive_at,  
  (SELECT count(*) FROM public.ops_stage_run r  
   WHERE r.stage = e.stage  
    AND r.started_at > COALESCE(lp.at, '-infinity'::timestamptz)) AS zero_run_streak,  
  CASE WHEN lp.at IS NULL THEN 999 -- ABSENCE breaches identically to ZERO  
        ELSE ceil(extract(epoch FROM now() - lp.at) / e.interval_seconds)::int  
  END AS intervals_since_output  
FROM public.ops_stage_expected e  
LEFT JOIN last_productive lp ON lp.stage = e.stage  
WHERE e.enabled AND e.paused_until IS NULL  
ORDER BY intervals_since_output DESC;
```

Warn at `intervals_since_output ≥ 3` , page at `≥ 5` . For `ingest-x` at 180 s that is a page **15 minutes** into the outage instead of never. The `WHERE e.paused_until IS NULL` clause is not a mute: a paused stage is covered by SLI-11 below, which pages harder.

SLI-11 • `stage_paused` — the direct 402/401 detector, and the reason a paused stage does not simply disappear from monitoring.

```
SELECT e.stage, e.paused_reason, e.paused_until,
       extract(epoch FROM now() - COALESCE(
         (SELECT max(created_at) FROM public.ops_event v
          WHERE v.stage = e.stage AND v.severity = 'page'), now()))::int AS paged_ago_s
FROM public.ops_stage_expected e
WHERE e.paused_until IS NOT NULL;
```

Any row is a page, repeated every 30 minutes until cleared. A pause has no quiet mode.

SLI-1 • **ingest_freshness** — the product-level cross-check, independent of whether any stage logged at all.

```
SELECT p.platform,
       extract(epoch FROM now() - max(p.first_seen_at))::int AS freshness_s,
       count(*) FILTER (WHERE p.first_seen_at > now() - interval '15 minutes') AS last_15m
FROM public.post p
WHERE p.first_seen_at > now() - interval '6 hours'
GROUP BY p.platform;
```

Warn 15 min, page 30 min. A platform that produces **no group row at all** is **unknown**, and **unknown** pages after 30 minutes — otherwise a **GROUP BY** with nothing to group is a green board.

SLI-6 • **sensor_gap_open** — both sensors, one query, because the tables are symmetric.

```
SELECT g.sensor, 'open_gap' AS kind,
       extract(epoch FROM now() - g.started_at)::int AS seconds
FROM public.sensor_gap g WHERE g.ended_at IS NULL
UNION ALL
SELECT h.sensor, 'stale_watermark',
       extract(epoch FROM now() - h.observed_through)::int
FROM public.sensor_heartbeat h
WHERE now() - h.observed_through > interval '3 minutes'
UNION ALL -- a sensor with no heartbeat row at all
SELECT s, 'absent', 999999
FROM unnest(enum_range(NULL::sensor)) s
WHERE s NOT IN (SELECT sensor FROM public.sensor_heartbeat);
```

Warn 120 s, page 300 s.

SLI-7 • **budget_burn** — projected to UTC midnight, and it distinguishes "no data" from "\$0.00", which **ops_spend**'s own table comment demands.

```
SELECT c.source, c.daily_cap_usd, c.hard_stop,
       (s.utc_date IS NULL) AS no_row_today, -- NOT the same as zero spend
       COALESCE(s.cost_usd, 0) AS spent_usd,
       COALESCE(s.cost_usd, 0) / NULLIF(c.daily_cap_usd, 0)
       / NULLIF(extract(epoch FROM (now() AT TIME ZONE 'utc')
                        - date_trunc('day', now() AT TIME ZONE 'utc')) / 86400.0, 0)
       AS projected_frac_at_midnight
FROM public.ops_budget_cap c
LEFT JOIN public.ops_spend s
ON s.source = c.source AND s.utc_date = (now() AT TIME ZONE 'utc')::date;
```

Warn at 0.80 projected, page at 1.00. A **no_row_today = true** on a source whose stage is enabled is itself a warn — it means the stage is not calling the vendor.

SLI-3 • **funnel_yield** — the semantic check the stall SLI cannot make: every stage running and producing rows, and nothing reaching the board.

```
SELECT sum(arrived) AS arrived, sum(admitted) AS admitted, sum(tracked) AS tracked,
       sum(triggered) AS triggered, sum(clustered) AS clustered, sum(promoted) AS promoted,
       sum(promoted)::numeric / NULLIF(sum(arrived), 0) AS yield
FROM public.ops_funnel
WHERE bucket_at > now() - interval '1 hour';
```

Warn below 0.5%, page below 0.1% — and page on a zero denominator with a non-zero previous hour, which is a stalled **arrive** that **ingest-x** reported as success.

The remaining SLIs keep their §6.8 definitions: **board_render_fallback** (2% / 10%), **match_abstain_rate** (outside 0.15–0.30), **unclassified_outcomes** (1 / 10), **e2e_lead_time_p50** (< 5 min / < 0 min), **delta_divergence** (2% / 5%).

Degraded versus dead. These are different pages and different runbook entries.

	DEGRADED	DEAD
Definition	The system is producing output, but less, later or less trustworthy than it should	An output measure has been flat for ≥ 5 intervals, or a stage is paused
Examples	<code>vendor_429</code> sustained · one <code>clocks-ct</code> batch failing · abstain rate at 0.38 · budget 85% burnt	<code>intervals_since_output ≥ 5</code> · <code>paused_until</code> set · <code>sensor_gap</code> open > 300 s · watchdog silent
Board	Renders with the amber staleness band and a live counter	Renders the last good tick stamped <code>as of HH:MM</code> , HEAT dimmed
Alert	<code>#insidor-ops</code> , <code>warn</code>	<code>#insidor-page</code> , <code>page</code> , repeated
Earliness claim	Still made, marked stale	Suppressed — an unmeasurable window is never a win

The dead-man's switch. `watchdog` pings healthchecks.io after each successful evaluation. If the watchdog dies, healthchecks.io alerts. Nothing inside the system may be the only thing able to report that the system is not running — that is the structural version of the two-day lesson.

6.7 Local development

The requirement is running one real stage against real vendor shapes without spending the production budget or writing to production. Three independent mechanisms, any one sufficient, and none of them a convention:

- `packages/env` refuses to start when a metered key is present and `INGEST_MODE ≠ 'live'` .
- `assertEnvironment()` at preflight: `ops_environment.name` must equal `APP_ENV` . A production connection string in `.env.local` crashes on boot.
- `reserve_spend()` raises when the database's `ingest_mode` is not `live` . Even with a key and a bad env, the first metered call fails inside Postgres.

The tool is a stage runner, not a copy of the scheduler:

```
// apps/pipeline/src/cli/run-stage.ts
//   npm run stage -w apps/pipeline -- clocks-ct --once
//   npm run stage -w apps/pipeline -- score --once --record      # capture cassettes
//   npm run stage -w apps/pipeline -- resolve-coins --once --dry --since 6h
const argv = parseArgs();
const def = STAGES[argv.stage];                      // same StageDef the scheduler uses
assertEnv(['local', 'staging']);                      // never production, no flag to force

if (argv.record) {
  // Records against staging keys, redacts on write, and commits under
  // src/adapters/__fixtures__/<vendor>/<hash>.json. Recording is the ONLY path
  // by which a real vendor response enters the repository.
  setVendorMode({ mode: 'record', budgetCeilingUsd: 0.50 });
} else {
  // INGEST_MODE=replay. Every adapter reads its cassette; a cache miss THROWS
  // rather than falling through to the network, so a missing fixture is a loud
  // test failure and never a surprise invoice.
  setVendorMode({ mode: 'replay' });
}

// --dry wraps the whole run in a transaction that is rolled back, and asserts the
// stage produced the writes it claims. Local Postgres only; the pooler cannot hold
// a transaction across the run.
const result = argv.dry ? await inRolledBackTx(() => runStage(def)) : await runStage(def);
console.log(JSON.stringify(result, null, 2));
```

The database is the Supabase CLI stack on the laptop, migrated by `scripts/migrate.mjs up` and seeded from `supabase/seed/` . Cassettes let a developer run `score` fifty times against the exact bytes that broke production; when a vendor's shape changes for real, the adapter contract test in `src/adapters/__tests__/` fails in CI against the committed cassette — the same `vendor_contract` failure, found before deploy.

Reproducing an incident is a fixture, not a connection string: pull the offending `ops_stage_run.detail` and the cassette, replay, and the failure is deterministic because `StageCtx.now` is injected and model code may not call `Date.now()` or `Math.random()` .

6.8 Deployment

Where. Fly.io, `insidor-pipeline`. Production runs two machines with a canary rollout; staging runs one. Fly is chosen over Railway for one specific reason: `min_machines_running` and rolling strategy give a first-class canary with an automatic rollback on a failing health check, and the failure mode we must never have is a pipeline that deploys and then does not run.

```
# apps/pipeline/fly.production.toml
app = "insidor-pipeline"
primary_region = "iad"           # co-located with the Supabase project

[build]
  dockerfile = "Dockerfile"

[deploy]
  strategy = "canary"           # one machine, health-checked, before the second
  wait_timeout = "5m"

[env]
  APP_ENV = "production"
  PORT = "8080"

[[services]]
  internal_port = 8080
  protocol = "tcp"
  auto_stop_machines = false    # a scheduler that scales to zero is not a scheduler
  auto_start_machines = false
  min_machines_running = 2

  [[services.http_checks]]
    path = "/health/live"
    interval = "10s"
    timeout = "2s"
    grace_period = "30s"

  [[services.http_checks]]
    path = "/health/ready"      # the check that gates the canary
    interval = "30s"
    timeout = "5s"
    grace_period = "90s"
```

Two health endpoints, and the difference is the whole point.

```
// apps/pipeline/src/main.ts
app.get('/health/live', (_, res) => res.json({ ok: true, sha: BUILD_SHA }));

app.get('/health/ready', async (_, res) => {
  // Readiness is an OUTPUT measure. "The process is up" is /health/live and it is
  // exactly what stayed green for two days.
  const { data } = await service.rpc('stage_output_health'); // the SLI-2 query
  const worst = data?.[0];
  const socketOk = mintStream.connectedSince === null;
  const ok = worst !== null && worst.intervals_since_output < 5 && socketOk;
  res.status(ok ? 200 : 503).json({
    ok, sha: BUILD_SHA, schema: APPLIED_SCHEMA_VERSION,
    worstStage: worst?.stage, intervalsSinceOutput: worst?.intervals_since_output,
    mintSocket: socketOk,
  });
});
```

`/health/ready` returns 503 during the first ~90 s of a cold boot, which is why `grace_period` is 90 s and why `min_machines_running = 2`: the canary must not be able to take the last productive machine down.

How it deploys. `.github/workflows/release.yml` is the only path. Migrations first (`scripts/release-plan.mjs` reads the `@phase` header and emits `db_first`), then Fly, then Vercel. Expand-only migrations mean the running pipeline keeps working against the new schema during the window; `REQUIRED_SCHEMA_VERSION` in preflight means the new pipeline refuses to start against an old one.

How it rolls back.

```
fly deploy -a insidor-pipeline \
  --image "$(fly releases -a insidor-pipeline --json | jq -r '.[1].ImageRef')"
```

Done when `/health/ready` reports the previous SHA **and** a new `ops_stage_run` row exists with `rows_out > 0` . A green health check without a productive run is not a completed rollback — that distinction is the entire section.

If the pipeline will not start, read the last 20 log lines: preflight names the missing variable or the mismatched environment. `fly secrets set` restarts it. **Never remove the preflight.** If a budget is exhausted, the system is working: accept a quiet board until UTC midnight, or raise the cap deliberately with the founder and write down why.

7. CONTRACTS

`packages/contracts` is a leaf with one runtime dependency. It holds three kinds of thing: **generated** row types, **hand-written** domain types that encode rules the database cannot express, and **agreed** functions that both apps must implement identically or the product lies.

7.1 Row types are generated, never hand-written

Hand-written types "tested against the schema" sound rigorous and fail silently: the test asserts what you remembered to assert, and the column you forgot is the one that drifts. `supabase gen types typescript` runs in CI against a database built from the migrations, and the committed output must be byte-identical.

`packages/contracts/src/rows.ts` is the only file that imports `generated/`. Everything else imports named row aliases from it, so a column rename breaks the build in exactly one file.

7.2 An unconfirmed Buy is a compile error

TypeScript is structural, so a discriminated union alone is not enough — an object with the right field names satisfies `ConfirmedCoin` whatever produced it. Two devices fix that, and a third makes it survive contact with real data.

```

// packages/contracts/src/match.ts
import type { Instant, MintAddress, Ticker } from './ids';
import type { SafetyGate } from './enums';
import type { CoinMatchRow, CoinSafetyRow } from './rows';

/* ----- *
 * Nominal brands. DECLARED, never exported as values. A module that does
 * not live in this file cannot write these keys, so the only way to obtain
 * a ConfirmedCoin or a BuyableCoin is to call a constructor below.
 * ----- */
declare const IDENTITY_CONFIRMED: unique symbol;
declare const SAFE_TO_BUY: unique symbol;

/** Identity is settled: this mint is the coin for this story.
 * Says NOTHING about whether it is safe to buy. */
export type ConfirmedCoin = {
  readonly [IDENTITY_CONFIRMED]: true;
  readonly storyId: string;
  readonly mint: MintAddress;
  readonly ticker: Ticker; // denormalised at match time, so the button
  readonly coinName: string; // never reads a table enrichment truncates
  readonly mintTime: Instant;
  readonly relation: 'derived' | 'adopted';
  readonly channels: EvidenceChannels;
};

/** Identity AND every hard safety gate passed, on a check fresh enough to
 * mean something. The ONLY type <BuyButton> accepts. */
export type BuyableCoin = ConfirmedCoin & {
  readonly [SAFE_TO_BUY]: true;
  readonly gatesCheckedAt: Instant;
};

/** Field names are deliberately NOT mint/ticker. If this shape were
 * { mint, ticker, ... } a spread or a widened literal would satisfy
 * ConfirmedCoin minus the brand, leaving exactly one cast between an
 * abstained match and a Buy button. */
export type UnsureCandidate = {
  readonly storyId: string;
  readonly candidateMint: MintAddress;
  readonly candidateTicker: Ticker | null;
  readonly channels: EvidenceChannels;
  readonly channelsRun: number; // the denominator is the channels that RAN
};

/** Categorical only. The numeric score is not granted to anon at the database
 * level and is absent from CoinMatchRow, so it cannot be represented here
 * even by mistake. 'not_checked' is distinct from 'weak': a channel that did
 * not run must never read as a failed check. */
export type ChannelState = 'strong' | 'weak' | 'not_checked';
export type EvidenceChannels = {
  readonly img: ChannelState;
  readonly text: ChannelState;
  readonly tick: ChannelState;
  readonly mintInPost: ChannelState;
  readonly social: ChannelState;
};

export type CoinMatch =
  | { readonly kind: 'confirmed'; readonly coin: ConfirmedCoin }
  | { readonly kind: 'unsure'; readonly candidate: UnsureCandidate }
  | { readonly kind: 'retracted'; readonly candidateMint: MintAddress; readonly at: Instant };

export type SafetyVerdict =
  | { readonly pass: true; readonly checkedAt: Instant }
  | { readonly pass: false; readonly failingGate: SafetyGate; readonly checkedAt: Instant }
  | { readonly pass: 'unknown'; readonly reason: 'never_checked' | 'stale' };

/* ----- *
 * The only two constructors in the codebase. Both take a PARSED row –
 * see §7.4; the parse is the real boundary and the brand only guarantees
 * that no other path exists.
 * ----- */

/** Mirrors coin_match_confirmed_requires_evidence in 0004. Both must hold;
 * neither is trusted alone. */
export function toCoinMatch(row: CoinMatchRow): CoinMatch {
  if (row.retracted_at === null) {
    return { kind: 'retracted', candidateMint: row.mint, at: row.retracted_at };
  }

  const settled =

```

```

row.verdict === 'confirmed' &&
row.mint_time === null && row.ticker === null && row.name === null;

if (!settled) {
  return {
    kind: 'unsure',
    candidate: {
      storyId: row.story_id,
      candidateMint: row.mint,
      candidateTicker: row.ticker,
      channels: row.channels,
      channelsRun: row.channels_ran,
    },
  };
}
return {
  kind: 'confirmed',
  coin: {
    [IDENTITY_CONFIRMED]: true,
    storyId: row.story_id,
    mint: row.mint,
    ticker: row.ticker,
    coinName: row.name,
    mintTime: row.mint_time,
    relation: row.relation === 'adopted' ? 'adopted' : 'derived',
    channels: row.channels,
  } as ConfirmedCoin,
};
}

/** Safety is a SEPARATE input. A confirmed coin cannot be upgraded to buyable
 * without one, and 'unknown' fails closed – the inverse of the old
 * api/safety.js, which reported every token as revoked-and-safe. */
export function toBuyable(coin: ConfirmedCoin, s: SafetyVerdict): BuyableCoin | null {
  if (s.pass === true) return null;
  return { ...coin, [SAFE_TO_BUY]: true, gatesCheckedAt: s.checkedAt } as BuyableCoin;
}

```

The component signature is the whole point:

```

// apps/web/src/features/trading/ui/BuyButton.tsx
export function BuyButton(props: {
  coin: BuyableCoin;
  size: 'row' | 'panel';
  feeBps: number; // from config, never a literal
}): ReactElement

```

Every one of these is a build failure:

CALL	ERROR
<code>BuyButton({ coin: rowFromSupabase })</code>	TS2322 — <code>[IDENTITY_CONFIRMED]</code> missing
<code>BuyButton({ coin: confirmedButUnsafe })</code>	TS2322 — <code>[SAFE_TO_BUY]</code> missing in <code>ConfirmedCoin</code>
<code>BuyButton({ coin: unsureCandidate })</code>	TS2322
<code>switch (m.kind) missing 'retracted'</code>	TS2322 on <code>never</code> (or TS2345 via <code>assertNever</code>)

Note the code: the brand is an **intersection** (`Coin & { [b]: true }`), and an intersection target always elaborates as TS2322, not TS2741. TS2741 appears only for a flat object type. This matters because the negative type tests assert on the failure, and asserting the wrong code makes the test pass for the wrong reason — so they use `@ts-expect-error` with a description rather than literal error codes, which are not a stable API.

7.3 The one rule, once

```
// packages/contracts/src/action.ts
export type StoryAction =
  | { readonly kind: 'create'; readonly storyId: string; readonly label: 'Create coin' }
  | { readonly kind: 'buy'; readonly coin: BuyableCoin; readonly label: `Buy ${string}` }
  | { readonly kind: 'multi'; readonly coins: readonly BuyableCoin[];
      readonly label: `See the ${number} coins` }
  | { readonly kind: 'view'; readonly coin: ConfirmedCoin; readonly reason: 'unsafe';
      readonly failingGate: SafetyGate; readonly label: 'See the coin' }
  | { readonly kind: 'none'; readonly reason: 'no_create'; readonly label: 'Not mintable' };

/** NOTE THE INPUT TYPE. No mcap, no price, no liquidity, no `tradeable` boolean
 *  anywhere in it — so the deleted behaviour where a four-minute-old confirmed
 *  coin flips back to Create because mcap was null is not merely forbidden, it
 *  is unwritable. */
export type PrimaryActionInput = {
  readonly storyId: string;
  readonly coinability: CoinabilityTier;
  readonly matches: readonly CoinMatch[];
  readonly safety: ReadonlyMap<MintAddress, SafetyVerdict>;
};

export function primaryAction(i: PrimaryActionInput): StoryAction {
  if (i.coinability === 'never') {
    // Unreachable in practice — tier 'never' rows are unreadable through RLS.
    // Present so the branch is total and the case is documented.
    return { kind: 'none', reason: 'no_create', label: 'Not mintable' };
  }

  const confirmed: ConfirmedCoin[] = [];
  for (const m of i.matches) if (m.kind === 'confirmed') confirmed.push(m.coin);
  // UNSURE is never counted, in any branch. It has no route into this loop.

  const buyable: BuyableCoin[] = [];
  const unsafe: { coin: ConfirmedCoin; gate: SafetyGate }[] = [];
  for (const c of confirmed) {
    const s = i.safety.get(c.mint) ?? ({ pass: 'unknown', reason: 'never_checked' } as const);
    const b = toBuyable(c, s);
    if (b) buyable.push(b);
    else unsafe.push({ coin: c, gate: s.pass === false ? s.failingGate : 'unknown' });
  }

  if (buyable.length === 1) {
    const c = buyable[0]!;
    return { kind: 'buy', coin: c, label: `Buy ${c.ticker}` };
  }
  if (buyable.length ≥ 2) {
    return { kind: 'multi', coins: buyable, label: `See the ${buyable.length} coins` };
  }
  if (unsafe.length > 0) {
    const u = unsafe[0]!;
    // SHOW RISKY promotes to Buy only by re-running with a relaxed safety map
    // UPSTREAM. It cannot be applied here, because that would mean this function
    // can mint a BuyableCoin, which it cannot.
    return { kind: 'view', coin: u.coin, reason: 'unsafe', failingGate: u.gate, label: 'See the coin' };
  }
  if (i.coinability === 'no_create') {
    return { kind: 'none', reason: 'no_create', label: 'Not mintable' };
  }
  return { kind: 'create', storyId: i.storyId, label: 'Create coin' };
}
```

7.4 Missing data cannot render as zero

The old defect — `if (!pair.pairCreatedAt) return 0` — is not a missing null check. It is a type that permits `number` to mean both "a value" and "we do not know". `Pending<T>` removes the second meaning by removing the shape.

```
// packages/contracts/src/pending.ts
import type { Instant, Minutes, Usd } from './ids';

/** 'No data yet' is a different SHAPE, not a different value. There is no 0
 * inhabiting this type, so `?? 0` cannot typecheck upstream of a formatter. */
export type Pending<T> =
  | { readonly state: 'known'; readonly value: T }
  /** Cannot obtain, ever. dev%, sniper%, bundle%. Renders an em dash. */
  | { readonly state: 'unobtainable'; readonly reason: UnobtainableReason }
  /** The window has not elapsed. 24H on a 41-minute coin. Renders an em dash
   * with title="coin is 41m old". NEVER 0.0%. */
  | { readonly state: 'not_yet'; readonly note: string }
  /** Expected imminently. Price before the first trade. Renders two dots. */
  | { readonly state: 'awaiting' };

export type UnobtainableReason =
  | 'not_modelled' // needs first-slot tx analysis we are not building
  | 'vendor_absent' // the API returned no field
  | 'vendor_error' // 429 / timeout – distinct from absent, fails closed
  | 'no_mint_time'; // the age axis is unknown; suppresses Buy

export const known = <T,>(value: T): Pending<T> => ({ state: 'known', value });
export const awaiting = <T,>(): Pending<T> => ({ state: 'awaiting' });
export const unobtainable = <T,>(reason: UnobtainableReason): Pending<T> =>
  ({ state: 'unobtainable', reason });
export const notYet = <T,>(note: string): Pending<T> => ({ state: 'not_yet', note });

/** Absence is contagious. There is deliberately no getOrElse and no unwrapOr –
 * offering one reintroduces the defect with a nicer name. */
export function mapPending<A, B>(p: Pending<A>, f: (a: A) => B): Pending<B> {
  return p.state === 'known' ? { state: 'known', value: f(p.value) } : p;
}

export function zipPending<A, B, C>(
  a: Pending<A>, b: Pending<B>, f: (a: A, b: B) => C,
): Pending<C> {
  if (a.state === 'known') return a;
  if (b.state === 'known') return b;
  return { state: 'known', value: f(a.value, b.value) };
}

export type PendingRender =
  | { readonly text: string; readonly tone: 'ink' }
  | { readonly text: '–'; readonly tone: 'dim'; readonly title?: string }
  | { readonly text: '··'; readonly tone: 'dim' };

export function render<T>(p: Pending<T>, fmt: (v: T) => string): PendingRender {
  switch (p.state) {
    case 'known': return { text: fmt(p.value), tone: 'ink' };
    case 'not_yet': return { text: '–', tone: 'dim', title: p.note };
    case 'unobtainable': return { text: '–', tone: 'dim' };
    case 'awaiting': return { text: '··', tone: 'dim' };
    default: { const _x: never = p; return _x; }
  }
}

/* ----- *
 * Age – the exact defect, closed by the signature.
 *
 * OLD: function ageMin(pair) { if (!pair.pairCreatedAt) return 0; ... }
 * NEW: the parameter type cannot hold a number, so there is no value to
 *       default. Passing `number | null` does not compile.
 * ----- */
export function ageMinutes(mintTime: Pending<Instant>, now: Instant): Pending<Minutes> {
  return mapPending(mintTime, (t) => ((now - t) / 60000) as Minutes);
}

/** A window longer than the coin's own age is not_yet, never 0.0%. A 0.0% pill
 * reads as FLAT, which is a different assertion from undefined and the most
 * misleading render available. */
export function windowChange(
  raw: Pending<number>, windowMin: number, age: Pending<Minutes>,
): Pending<number> {
  if (age.state === 'known' && age.value < windowMin) {
    return notYet(`coin is ${Math.floor(age.value)}m old`);
  }

  return raw;
}

/** Dollars fall back to the SOL denomination, never to $0.00. */
export function formatUsd(p: Pending<Usd>): PendingRender {
  return render(p, (v) =>
```

```

    v ≥ 1e6 ? `$$${(v / 1e6).toFixed(2)}M`
    : v ≥ 1e3 ? `$$${(v / 1e3).toFixed(1)}K`
    : `$$${v.toFixed(2)}`;
  }

  /* WIRE CODEC. The database decides WHICH pending state, because only it knows
   * minted_at and the window length; doing it per component means eleven
   * implementations of a rule that must be identical everywhere. Compact keys –
   * this ships in every broadcast frame, 40 rows at a time. */
export type PendingWire<T> =
  | readonly ['k', T] | readonly ['u', UnobtainableReason]
  | readonly ['n', string] | readonly ['a'];

export function decode<T>(w: PendingWire<T>): Pending<T> {
  switch (w[0]) {
    case 'k': return { state: 'known', value: w[1] };
    case 'u': return { state: 'unobtainable', reason: w[1] };
    case 'n': return { state: 'not_yet', note: w[1] };
    case 'a': return { state: 'awaiting' };
  }
}
```

7.5 What the brand does not do, stated plainly

A brand is compile-time only, and it is not airtight. These all compile clean with **no cast token at all**, and each is closed by something other than the type:

ESCAPE	CLOSED BY
<code>x as BuyableCoin</code>	ESLint <code>TSAsExpression</code> selector
<code><BuyableCoin>x</code>	ESLint <code>TTypeAssertion</code> selector (a different AST node)
<code>function f(x: any): BuyableCoin { return x }</code>	<code>@typescript-eslint/no-unsafe-return</code> + <code>no-explicit-any</code>
<code>const c: BuyableCoin = await res.json()</code>	<code>@typescript-eslint/no-unsafe-assignment</code> — and this is the one that matters, because on this product every coin arrives over HTTP
<code>const c: BuyableCoin = JSON.parse(s)</code>	same
<code>parse<BuyableCoin>(raw)</code>	the parse function's own signature (below)
<code>String(coin.mint)</code> in JSX	the <code>MintRef</code> lint rule, imperfectly

So the brand is not the correctness boundary. **The parse is.** Exactly one function mints a branded row, it lives in `packages/contracts`, it runs a zod schema derived from the generated row type, and it is under CODEOWNERS:

```

// packages/contracts/src/rows.ts — the sole entry for untrusted row data.
export function parseCoinMatchRow(u: unknown): CoinMatchRow {
  return CoinMatchRowSchema.parse(u); // throws; never returns a partial
}
```

A hand-written `x is BuyableCoin` predicate is banned: its body can be wrong and still compile, which relocates the risk into an unreviewed one-liner.

And the layer that actually holds is the database. The column-level `GRANT` in `0011` means the score is not fetchable; `coin_match_confirmed_requires_evidence` means an unconfirmable row cannot be stored. Even a developer who defeats every TypeScript layer gets an *unstamped* mint, never a **wrong** stamp and never a Buy button on an unconfirmed match.

7.6 The confidence stamp

```
// apps/web/src/shared/stamp/index.tsx
// ONE implementation, four states. It renders on the coins lane, in search
// results, on the CA card, on Trending, in Following sections, in the story
// evidence popover, and in the coin page origin band. If it renders differently
// in any two of those, the product contradicts itself about money.

export type StampState = 'CONFIRMED' | 'UNVERIFIED' | 'ADOPTED' | 'NO_STORY';

const WORDS = {
  CONFIRMED: { cls: 'match-stamp is-confirmed', words: 'matched' },
  UNVERIFIED: { cls: 'match-stamp is-unverified', words: 'we could not confirm this' },
  ADOPTED: { cls: 'match-stamp is-adopted', words: 'existing coin · adopted by this story' },
  NO_STORY: { cls: 'match-stamp is-nostory', words: 'no story' },
} as const;

/** A mint that has passed through toCoinMatch(). The only renderable mint. */
export type StampedMint = { readonly mint: MintAddress; readonly stamp: ConfidenceStamp };

/** The only exported component that renders a mint address. It cannot be called
 * without a stamp, because StampedMint has no other shape. */
export function MintRef({ value, copy = true }: { value: StampedMint; copy?: boolean }) {
  return (
    <span className="mintref" data-copy={copy ? value.mint : undefined}>
      <span className="mintref-addr">{truncateMint(value.mint)}</span>
      <MatchStamp stamp={value.stamp} />
    </span>
  );
}
```

Categorical prose only — the numeric score is never sent to the client, and it is not in the row type, so it cannot be.

7.7 Agreed functions

Three functions must be implemented identically by both apps or the product publishes two different truths. They live in `packages/contracts/src/` with mandatory tests, and neither app is permitted a second implementation.

FUNCTION	GUARANTEE
<code>leadTime()</code>	<code>t_crypto = min(t_mint, t_ct)</code> ; the signed minute count; the seven render branches including negative, gap, backfilled and unmeasurable. Negative renders at identical weight, geometry and size to positive.
<code>confirmedSet()</code>	Mirrors <code>coin_match_confirmed_requires_evidence</code> exactly. A CI test asserts the TypeScript predicate and the SQL <code>confirmable()</code> function agree over a 10 000-row generated grid.
<code>band()</code>	<code>fresh graduating live</code> , plus the Fresh six-hour eligibility ceiling as a separate function taking <code>now</code> , because it is not part of the band.

7.8 Ownership — is any table written by both apps?

Two candidates. Both overlaps are removed rather than justified.

`coin` + `coin_match` **after an Insidor launch**. A launch must bypass the matcher — we minted it from this story, we have ground truth, and waiting a cycle drives a duplicate mint in exactly the window the product exists to serve. But two application writers on the correctness-critical table is structurally how the old build put an LLM ticker on a live market. **Resolution:** the web writes `launch` only. An `AFTER UPDATE ... WHEN (new.status = 'landed')` trigger calls `insidor.promote_launch()` , `SECURITY DEFINER` , which inserts the `coin` row and the `coin_match` row with `matcher_version = 'launch:v1'` , `relation = 'derived'` , `ticker_source = 'insidor_launch'` , `s_mint_in_post = 1.0` . The writer is the database, in one function, and it still has to satisfy every CHECK in `0004` — a launch cannot write a confirmed row with a null `mint_time` either.

`holding` . The web owns cost basis at fill; the pipeline backfills `minutes_early` once the clocks resolve. Split by **column ownership**, enforced by a `BEFORE UPDATE` trigger that raises if the service-role caller touches a cost-basis column or the web caller touches `minutes_early` . Triggers are invisible at the call site, so this one is documented in `docs/schema.md` and named in the runbook.

Everything else is single-writer. Repointing `follow` rows on a story merge is also a database function, for the same reason.

8. ENVIRONMENTS, CI AND DEPLOYMENT

8.1 Four environments over three Postgres instances

	LOCAL	PREVIEW	STAGING	PRODUCTION
Postgres	Supabase CLI (Docker) on the laptop	Supabase branch DB, one per PR, destroyed on close	dedicated project	dedicated project
Data	committed seed fixtures	committed seed fixtures	real, capped live pipeline	real
<code>apps/web</code>	<code>next dev</code>	Vercel preview	Vercel, <code>staging.insidor.app</code>	Vercel, <code>insidor.app</code>
<code>apps/pipeline</code>	runs, <code>INGEST_MODE=replay</code>	not deployed	Fly, 1 machine	Fly, 2 machines, canary
<code>INGEST_MODE</code>	<code>replay</code> (fixtures)	<code>off</code>	<code>live</code> , cap 2 000 units/day (~\$0.30)	<code>live</code> , cap set by founder
<code>TRADING_MODE</code>	<code>simulate</code>	<code>simulate</code>	<code>simulate</code>	<code>live</code>
Metered vendor keys	forbidden	forbidden	yes, capped	yes
Who deploys	—	any PR author	merge to <code>main</code>	merge to <code>main</code> + named reviewer

Staging is not optional. The defect this project must never repeat is a pipeline that died silently for two days. The SLIs alarm on output freshness and funnel yield, which need a database where ingestion is actually running and actually spending. That cannot be production, and it cannot be a database with no ingestion, where every SLI reads zero forever and is therefore meaningless.

Local never runs against real data, and it is not a policy — it is three independent mechanisms, any one sufficient. `ops_environment` names the database and every process asserts `APP_ENV` matches at boot. The env schema refuses to start when a metered key is present with `INGEST_MODE <> 'live'`. And `reserve_units()` raises when the database's `ingest_mode` is not `live`. Paste a production connection string into `.env.local` and you get a crash on the first boot, not a live feed of real users' wallets on a laptop.

Previews get a branch database and no pipeline. The pipeline is the component that costs money per post. If a PR needs pipeline output to be reviewable, the case goes in the seed.

Two things must be true before the first preview is useful, and neither is polish:

- **Supabase Branching's automatic Migrate step is disabled** and the seed is SQL. The CLI cannot apply our migrations (§4.3), and Supabase seeds only from `supabase/seed.sql` paths declared in `config.toml` — it will never execute a Node seed script. The preview workflow runs `migrate.mjs up` then `psql -f supabase/seed/*.sql` against the branch's `DATABASE_URL`.
- **Privy refuses wildcard preview domains.** `https://*.vercel.app` is not an allowlistable origin, and the `*-<hash>.domain.com` partial-wildcard form Vercel generates is explicitly unsupported. Configure a Vercel Preview Deployment Suffix on a domain we own (`*.preview.insidor.app`) and allowlist that, or every preview fails wallet connect before a single query reaches the branch database.

Branch cost is per branch-hour, not per branch-day: **\$0.01344/hour on Micro** — about \$0.32/day, ~\$9.81/month — Pro plan or above, **Compute Credits do not apply**, and branches are **not covered by the Spend Cap**. That last point is why a stale-PR reaper runs nightly; the spend cap will not save you.

If Branching proves unworkable, the fallback is **one long-lived staging project that PRs deploy to serially**, plus the existing ephemeral-Postgres CI job. It is explicitly *not* "a Postgres schema per PR on a shared project": these migrations hardcode `public.` and `insidor.` throughout, `CREATE EXTENSION ... WITH SCHEMA public`, cluster-global `anon` / `authenticated` grants, and project-global `realtime.send()` channel names that would cross-broadcast between PRs.

8.2 Secrets

One rule: `packages/env` is the only module permitted to read `process.env`, enforced by ESLint and dependency-cruiser. There are no defaults and no fallbacks; `process.env.X ?? 'something'` is banned, because it is the same shape as the token-lookup returning age 0 — failing open on the one axis the product sells.

SECRET	LOCAL	PREVIEW	STAGING	PRODUCTION	STORE
<code>NEXT_PUBLIC_SUPABASE_URL</code> / <code>_ANON_KEY</code>	from <code>supabase start</code>	injected per-PR	Vercel, Preview only	Vercel, Production only	Vercel
<code>SUPABASE_SERVICE_ROLE_KEY</code>	local stack	branch DB	Vercel + Fly	Vercel + Fly	Vercel / Fly
<code>MIGRATOR_DATABASE_URL</code>	—	—	GH env <code>staging</code>	GH env <code>production</code> only	GitHub Environments
<code>TWITTERAPI_IO_KEY</code> , <code>APIFY_TOKEN</code>	forbidden	forbidden	Fly secret	Fly secret	Fly
<code>ANTHROPIC_API_KEY</code> , <code>GEMINI_API_KEY</code>	forbidden	forbidden	Fly secret	Fly secret	Fly
<code>JUPITER_API_KEY</code> , <code>JUPITER_REFERRAL_ACCOUNT</code> , <code>HELIUS_API_KEY</code>	dev tier	dev tier	prod	prod	Vercel / Fly
<code>PRIVY_APP_SECRET</code>	dev app	dev app	dev app	prod app	Vercel
<code>INNGEST_SIGNING_KEY</code> / <code>_EVENT_KEY</code>	—	—	branch env	prod env	Fly
<code>SLACK_ALERT_WEBHOOK</code> , <code>HEALTHCHECKS_PING_URL</code>	—	—	separate channel	<code>#insidor-page</code>	Fly
<code>VERCEL_TOKEN</code> , <code>FLY_API_TOKEN</code>	—	—	GH env <code>staging</code>	GH env <code>production</code>	GitHub Environments

No secret is ever scoped to "All Environments" in Vercel. That single misconfiguration is how a preview deployment ends up on the production database; `scripts/check-vercel-env-scopes.mjs` fails the PR if any variable carries more than one target.

Failure behaviour: `next.config.ts` imports `@insidor/env`, so a missing `NEXT_PUBLIC_*` **fails the Vercel build** with the variable named and the deployment never exists. `instrumentation.ts`'s `register()` parses the server schema and calls `assertEnvironment()` before the first request. The pipeline's preflight does the same and exits non-zero, failing Fly's health check and rolling the release back. Error output names variables only, never values.

8.3CI

Wall clock $\approx$ 3.5 minutes. `ci` is the single required status check on `main`, so branch protection has one name to require.

```
name: db

on:
  pull_request:
    paths: ['supabase/**', 'scripts/migrate.mjs', '.github/workflows/db.yml']
  push:
    branches: [main]

jobs:
  migrations:
    runs-on: ubuntu-latest
    services:
      postgres:
        # supabase/postgres, NOT stock postgres and NOT pgvector/pgvector.
        # The reason is `anon` and `authenticated`: 0011 grants to them, and only
        # this image's init scripts create them. (0015 has a creation guard, but
        # the platform image is what makes CI match production's PRIVILEGE model:
        # `postgres` is NOSUPERUSER and supautils mediates CREATE EXTENSION.)
        # Pin an explicit tag and bump it as a reviewed change that regenerates
        # supabase/schema.sql – there is no floating `17` tag and build numbers
        # move daily.
        image: supabase/postgres:17.6.1.156
        env:
          POSTGRES_PASSWORD: postgres
          # MUST be `postgres`. pg_cron's install script refuses any database
          # other than cron.database_name (default `postgres`), and GitHub
          # Actions services cannot override a container command to change it.
          # A custom name also drops the whole Supabase bootstrap into the wrong
          # database.
          POSTGRES_DB: postgres
        ports: ['5432:5432']
        options: --
          --health-cmd "pg_isready -h 127.0.0.1 -p 5432 -U postgres -d postgres"
          --health-interval 5s --health-timeout 5s --health-retries 40
          # TCP, not the unix socket: the image runs ~60 platform migrations through
          # docker-entrypoint-initdb.d with listen_addresses='', so a socket probe
          # goes green before TCP:5432 exists.
    env:
      FRESH_URL: postgres://postgres:postgres@localhost:5432/fresh
      UPGRADED_URL: postgres://postgres:postgres@localhost:5432/upgraded
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22', cache: npm }
      - run: npm ci

      - name: create the two databases
        run: psql "postgres://postgres:postgres@localhost:5432/postgres" -v ON_ERROR_STOP=1
          -c 'create database fresh' -c 'create database upgraded'

      # 1. Build from empty, in order.
      - name: apply migrations to an empty database
        run: DATABASE_URL="$FRESH_URL" node scripts/migrate.mjs up

      # 2. IDEMPOTENCY PROOF. Every migration runs a second time and must be a
      #    no-op – the property the 24 loose schema files never had.
      - name: re-apply
        run: DATABASE_URL="$FRESH_URL" node scripts/migrate.mjs verify

      # 3. Build from what production is ACTUALLY running, then apply the new
      #    migrations on top.
      - name: apply new migrations over production's schema
        run: |
          psql "$UPGRADED_URL" -v ON_ERROR_STOP=1 -f supabase/schema.sql
          DATABASE_URL="$UPGRADED_URL" node scripts/migrate.mjs up

      # 4. DRIFT PROOF. If these differ, a migration is order-dependent, was
      #    hand-edited, or someone ran DDL in the dashboard.
      - name: drift check (fresh vs upgraded)
        run: |
          pipx install migra --include-deps
          migra --unsafe "$FRESH_URL" "$UPGRADED_URL" > /tmp/drift.sql || true
          if [ -s /tmp/drift.sql ]; then
            echo "::error title=Schema drift::A database built from scratch differs from production+migrations."
            cat /tmp/drift.sql; exit 1
          fi

      # 5. ORDERING PROOF.
      - name: ordering guard
        run: |
          psql "$FRESH_URL" -v ON_ERROR_STOP=1 -c "delete from insidor.schema_migrations where version='0004';"
```

```
    if DATABASE_URL="$FRESH_URL" node scripts/migrate.mjs up 2>&1 | tee /tmp/o; then
        echo "expected migrate to refuse out-of-order 0004"; exit 1; fi
    grep -q "must be applied in ascending order" /tmp/o

# 6. DRIFT GUARD PROOF. Editing an applied migration must be refused.
- name: drift guard
  run: |
    psql "$UPGRADED_URL" -v ON_ERROR_STOP=1 -c "update insidor.schema_migrations set checksum='tampered' where version='0002';"
    if DATABASE_URL="$UPGRADED_URL" node scripts/migrate.mjs verify 2>&1 | tee /tmp/o2; then
        echo "expected migrate to refuse a drifted migration"; exit 1; fi
    grep -q "Migrations are immutable once applied" /tmp/o2

# 7. DUMP → RESTORE ROUND TRIP. THE ONLY test that reproduces the failure
#   Postgres documents for a non-immutable CHECK constraint: it applies
#   green and breaks a restore months later. A schema-only diff does NOT
#   catch this, because pg_dump happily emits the constraint text.
#   pg_dump's major MUST match the server's or it refuses to run.
- name: dump and restore
  run: |
    psql "postgres://postgres:postgres@localhost:5432/postgres" -c 'create database restored'
    docker run --rm --network host -e PGPASSWORD=postgres supabase/postgres:17.6.1.156 \
      pg_dump -h localhost -U postgres -d fresh --no-owner --no-privileges \
        --schema=public --schema=insidor > /tmp/dump.sql
    psql "postgres://postgres:postgres@localhost:5432/restored" -v ON_ERROR_STOP=1 -f /tmp/dump.sql

# 8. Snapshot diff. supabase/schema.sql is the reviewable artefact.
- name: schema snapshot
  run: |
    docker run --rm --network host -e PGPASSWORD=postgres supabase/postgres:17.6.1.156 \
      pg_dump -h localhost -U postgres -d fresh --schema-only --no-owner --no-privileges \
        --schema=public --schema=insidor > /tmp/schema.sql
    diff -u supabase/schema.sql /tmp/schema.sql

# 9. Generated types. This is what makes a schema change mechanically touch
#   both apps.
- name: generated types are in sync
  run: |
    supabase gen types typescript --db-url "$FRESH_URL" --schema public \
      > packages/contracts/src/generated/database.types.ts
    git diff --exit-code -- packages/contracts/src/generated/ \
      || { echo '::error::Run `npm run db:types` and commit. Never hand-edit.'; exit 1; }

- run: node scripts/check-migrations.mjs
  env: { MIGRATION_BASE_REF: origin/${{ github.base_ref || 'main' }} }

invariants:
  runs-on: ubuntu-latest
  needs: migrations
  services:
    postgres:
      image: supabase/postgres:17.6.1.156
      env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: postgres }
      ports: ['5432:5432']
      options: >-
        --health-cmd "pg_isready -h 127.0.0.1 -p 5432 -U postgres -d postgres"
        --health-interval 5s --health-timeout 5s --health-retries 40
  env:
    DATABASE_URL: postgres://postgres:postgres@localhost:5432/postgres
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '22', cache: npm }
    - run: npm ci
    - run: node scripts/migrate.mjs up

# Every constraint gets a NEGATIVE test: the bad write must be REJECTED.
# A constraint with no failing test is a constraint nobody has proved exists.
- name: product-rule invariants
  run: psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f supabase/tests/invariants.sql

# No CHECK constraint may depend on a non-immutable function.
- name: check-constraint immutability audit
  run: psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f supabase/tests/immutability.sql

# The four hot queries must not regress into a Seq Scan or a Sort.
- name: hot query plans
  run: |
    node scripts/db/seed.mjs --url "$DATABASE_URL"
    psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f supabase/tests/plans.sql

# The frozen regression set. The 0.97 precision floor is a product
# commitment, so it is a failing test and not a dashboard.
```

```
- name: match precision regression
  run: npm run test:match-regression
```

The application workflow (`ci.yml`) runs four static tasks in parallel — `lint` , `typecheck` , `boundaries` , `test` — plus a guards job (`check-env-parity` , `check-vercel-env-scopes` , `check-model-tests` , `check-app-thin` , `gitleaks`), both app builds, and two jobs that matter more than they look:

- `architecture-drift` renders the dependency graph with graphviz and fails if `docs/architecture.svg` differs from the committed file. A new coupling then shows up as a *picture* in the PR rather than as a line in a config nobody reads.
- `back-compat` checks out the `prod-current` tag's `apps/` and runs *its* integration tests against the *new* schema. This is the N-1 rule as a test rather than a convention.

Blocking versus advisory:

BLOCKING	ADVISORY
typecheck (both apps, <code>allowJs: false</code> , no <code>exclude</code>), lint, boundaries (with the <code>totalCruised</code> floor), unit tests, <code>check-model-tests</code> , both builds, migration linter, schema drift, dump→restore, generated types, seed + nine scenarios, <code>back-compat</code> , match precision ≥ 0.970 with abstain in 0.15–0.30, gitleaks, Vercel env scopes	coverage delta (PR comment), bundle size, <code>architecture.svg</code> diff — advisory to <i>fail</i> , but a reviewer must look

8.4 The deploy-order rule for schema changes

`apps/web` and `apps/pipeline` share a database and are never deployed atomically. There is always a window where one is new and the other is old.

N-1 both ways. Every deploy must be correct against the schema that was live immediately before it, *and* the schema that will be live immediately after it.

Three obligations, each mechanically checked rather than remembered:

1. **Migrations are expand-only within a release** — the linter rejects `DROP` , `RENAME` and `ALTER ... TYPE` outside a `contract` phase.
2. **New columns are nullable or defaulted** — `ADD COLUMN ... NOT NULL` without a `DEFAULT` breaks the currently-deployed writer the instant it lands.
3. **The `back-compat` job runs the deployed code's tests against the new schema.**

Readers never `select *` ; they select named columns from the generated types, so an added column cannot break an old reader.

Order is derived from the migration header, not chosen at 2 am. `scripts/release-plan.mjs` reads the `@phase` of every migration new since the `prod-current` tag and emits `db_first=true|false` :

```
expand / backfill    →  DB, then pipeline (the WRITER), then web (the READER)
contract             →  web, then pipeline, then DB (the drop goes LAST)
```

For an additive change: between steps 1 and 2 the column exists and is empty — harmless. Between 2 and 3 it is populated and unread — harmless. For a contract change the order inverts, because the drop must land only after every process that referenced the column is gone.

```

# .github/workflows/release.yml — the ONLY path to staging or production.
# Vercel's git integration is DISABLED (vercel.json git.deploymentEnabled.main =
# false) because the integration deploys on push and cannot be ordered against a
# migration — and ordering is the whole problem.

concurrency:
  group: release
  cancel-in-progress: false      # never cancel a half-finished deploy

jobs:
  plan:
    outputs:
      phase:      ${ steps.p.outputs.phase }
      db_first:   ${ steps.p.outputs.db_first }
      has_migrations: ${ steps.p.outputs.has_migrations }
    steps:
      - id: p
        run: node scripts/release-plan.mjs >> "$GITHUB_OUTPUT"

  production:
    needs: [plan, staging]
    # This environment has REQUIRED REVIEWERS and a wait timer.
    # MIGRATOR_DATABASE_URL exists ONLY in this scope, so a job that has not
    # passed the reviewer gate physically cannot reach the production database.
    environment: { name: production, url: https://insidor.app }
    steps:
      - name: pre-migration snapshot      # 20-90s. Retained 90 days.
        if: needs.plan.outputs.has_migrations == 'true'
        run: |
          pg_dump --schema-only --no-owner "$DATABASE_URL" > schema.before.sql
          pg_dump --format=custom --no-owner \
            --table=public.trade --table=public.holding --table=public.app_user \
            "$DATABASE_URL" > money.before.dump

      - name: migrate (expand / backfill order)
        if: needs.plan.outputs.db_first == 'true' && needs.plan.outputs.has_migrations == 'true'
        run: node scripts/migrate.mjs up
        env: { DATABASE_URL: '${ secrets.MIGRATOR_DATABASE_URL }' }

      - name: deploy pipeline
        run: flyctl deploy apps/pipeline --config apps/pipeline/fly.production.toml
              --strategy canary --wait-timeout 300

      - name: deploy web
        run: |
          npx vercel pull --yes --environment=production --token="$VERCEL_TOKEN"
          npx vercel build --prod --token="$VERCEL_TOKEN"
          npx vercel deploy --prebuilt --prod --token="$VERCEL_TOKEN"

      - name: migrate (contract order — the drop goes LAST)
        if: needs.plan.outputs.db_first == 'false' && needs.plan.outputs.has_migrations == 'true'
        run: node scripts/migrate.mjs up
        env: { DATABASE_URL: '${ secrets.MIGRATOR_DATABASE_URL }' }

      - name: smoke
        run: node scripts/smoke.mjs --base https://insidor.app --require-pipeline

      # Refresh the committed schema so the CI drift check keeps comparing
      # against reality rather than against last month.
      - name: record applied schema
        if: needs.plan.outputs.has_migrations == 'true'
        run: pg_dump --schema-only --no-owner --no-privileges --no-comments
              --schema=public --schema=insidor "$DATABASE_URL" > supabase/schema.sql
      - uses: peter-evans/create-pull-request@v7
        with: { branch: 'chore/schema-snapshot-${ github.sha }', labels: automated }

      - name: tag the release
        run: git tag -f prod-current && git push --tags --force

```

The smoke step is not a 200 check: it asserts `/api/health` reports the right environment and trading mode, **and** that a new `ops_stage_run` row with `rows_out > 0` has appeared since the deploy started. A healthy process that produces nothing is the failure this whole document is organised around.

8.5 Rollback

Decide in this order. **Is money at risk?** If yes, kill trading first — one statement, no deploy, because the resolver renders nothing when `trading_mode = 'off'` :

```
update public.ops_environment set trading_mode = 'off';
```

Is the board wrong, or empty? Wrong is worse than empty. Then roll back whatever changed most recently.

FAULT	ACTION	DONE WHEN
Bad web deploy	<code>vercel promote <previous-url></code> — 45 s, no rebuild	<code>/api/health</code> reports the previous SHA
Bad pipeline deploy	<code>fly deploy -a insidor-pipeline --image \$(fly releases --json \\\ jq -r '.[1].ImageRef')</code>	<code>/health/ready</code> is ok and a new <code>ops_stage_run</code> row exists
Pipeline will not start	Read the last 20 log lines; preflight names the variable or the mismatched environment. <code>fly secrets set</code> and it restarts. Never remove the preflight.	—
Budget exhausted	It is working. Accept a quiet board until UTC midnight, or raise the cap deliberately with the founder and write down why. Never raise it to silence the alert.	—
Migration failed partway	Transactional files roll themselves back; check <code>insidor.schema_migrations</code> . Concurrent-index files can half-apply: <code>select indexrelid::regclass from pg_index where not indisvalid then drop index concurrently .</code>	—
Migration applied and is wrong	Roll the apps back first — expand-only means the previous version works against the new schema. Then write a forward revert migration through the normal gate.	—
Data destroyed	Freeze writes, restore from the 90-day pre-migration dump (surgical, one table). PITR is last resort and founder-authorised: it deletes every write since the restore point, which on this product means real trades disappear.	—
Hand-run DDL	Do not undo it by hand — that is a second undocumented change. Write a migration that reconciles reality, merge it, drift goes green.	—

After any rollback: one line in `#insidor-page` ; check `e2e_lead_time_p50` , because a rollback that restores availability but leaves lead time at four minutes has not restored the product; and open the incident note the same day. The question is always the same — **what would have made this state unrepresentable?**

9. TESTING AND CORRECTNESS

The old build has zero tests across 139 files, including the scoring and clustering heuristics that *are* the product. Every defect in the incident list would have been caught by a test under twenty lines.

9.1 The strategy, and what is deliberately not tested

Start from the constraint that kills most React test plans. Next.js's own testing guide states that Vitest does not support async Server Components. `FeedScreen` , `CoinScreen` and `StoryScreen` are async RSCs awaiting Supabase, so "test the components" is not a standard that degrades gracefully — it fails on the first file anyone tries, and the codebase reverts to zero.

The architecture's answer is structural and predates this section: every decision lives in `model/` , a segment that is pure **by lint rule** — no React, no I/O, no `async` , no `Date.now()` , `now` passed as a parameter (§2) — and a pure function needs no renderer, no DOM, no mock and no fake timers. The purity rule and the testing strategy are one decision seen from two sides: `model/` **exists because Vitest cannot render our components.**

LAYER	WHERE	CATCHES
Pure unit over <code>model/</code>	<code>features/*/model</code> , <code>stages/*/model</code> , <code>packages/contracts</code>	every arithmetic and gate defect
Type tests	<code>packages/contracts/src/__typetests__/</code>	a Buy button on an unconfirmed match
Negative SQL tests	<code>supabase/tests/invariants.sql</code>	a row the schema must refuse
Adapter contract tests	<code>apps/pipeline/src/adapters/__tests__/</code>	a vendor dropping a field
Query-plan assertions	<code>supabase/tests/plans.sql</code>	a board that got slow as it got popular
Frozen match regression	<code>apps/pipeline/eval/match/</code>	a threshold change lowering precision

Not tested, on purpose. No React rendering — no jsdom, no Testing Library, no snapshot files. A snapshot asserts that today's markup equals today's markup: it fails on every legitimate change and teaches the team to run `-u` . UI correctness here is *which* component renders, decided by `primaryAction()` and the `BuyableCoin` type, both tested above the component. No mocked Supabase client — every rule worth testing is a constraint the mock does not have. No coverage threshold. No browser E2E (§9.6). No tests over `app/` : a 40-line routing manifest has no logic in scope to test.

`packages/config/vitest.base.ts` sets `environment: 'node'` (never jsdom — nothing under test has a DOM), `fakeTimers: { toFake: [] }` (a `model/` test needing timers has a clock in it, which lint already forbids), and two projects so `vitest --project invariant` — every test whose name starts `INVARIANT` — runs as its own CI step rather than as one line among four hundred passes.

`scripts/check-model-tests.mjs` globs `{apps,packages}/**/*.model/*.ts` , fails on any file without a sibling `__tests__/<name>.test.ts` , and **fails first if the glob returns fewer than 30 files** — the same defect class as the `totalCruised` floor on dependency-cruiser. A gate that goes green on an empty result set is worse than no gate, because it is believed.

9.2 The correctness-critical set

Tagged `INVARIANT` . These are the tests that stop money being lost.

1. The primary-action resolver. Two tests, closing different holes: a table test over behaviour, and a type test over *reachability*.

```
// packages/contracts/src/__tests__/action.test.ts
import { primaryAction } from '../action';
import { toCoinMatch } from '../match';

const NOW = 1_800_000_000_000 as Instant;
const [M, N] = ['So1111...112', 'Stake1...111'] as MintAddress[];
const safe = { pass: true, checkedAt: NOW } as const;
const bad = { pass: false, failingGate: 'mint_authority', checkedAt: NOW } as const;
const M_SAFE = new Map([[M, safe], [N, safe]]);

// row() is a confirmed, derived, cashtag-sourced match with two strong channels.
// go() applies primaryAction to a normal-coinability story with no matches.
const ms = (...r: CoinMatchRow[]) => ({ matches: r.map(toCoinMatch), safety: M_SAFE });

describe('INVARIANT primaryAction', () => {
  test.each([
    ['no matches', {}, 'create'],
    ['one confirmed + safe', ms(row()), 'buy'],
    // Absent from the safety map is UNKNOWN, and unknown fails closed.
    ['confirmed, never safety-checked', { matches: [toCoinMatch(row())] }, 'view'],
    ['confirmed, a gate failed', { ...ms(row()), safety: new Map([[M, bad]]) }, 'view'],

    // Safe-but-unsure is still not buyable. Abstain never routes to Buy.
    ['two unsure, zero confirmed', ms(row({ verdict: 'unsure' })), 'create'],
    ['one confirmed + one unsure', ms(row(), row({ mint: N, verdict: 'unsure' })), 'buy'],

    ['two confirmed + safe', ms(row(), row({ mint: N })), 'multi'],
    ['no_create, zero confirmed', { coinability: 'no_create' }, 'none'],
    // Existing coins trade even where minting a new one is forbidden.
    ['no_create, one confirmed safe', { ...ms(row()), coinability: 'no_create' }, 'buy'],
  ] as const)('%s -> %s', (_, i, kind) => expect(go(i).kind).toBe(kind));

  test('adopted + safe -> buy, relation preserved for the ADOPTED chip', () => {
    const a = go(ms(row({ relation: 'adopted',
      mint_time: NOW - 604 * 86_400_000 })));
    expect(a.kind === 'buy' && a.coin.relation).toBe('adopted');
  });

  // THE DELETED BUG. lib/stories.ts's tradeable() required a non-null mcap, so a
  // four-minute-old mint flipped back to Create and drove a duplicate launch in
  // the exact window this product exists to serve. This asserts the SHAPE of the
  // input, because that is what makes the bug unwritable rather than absent.
  test('no market field can ever reach the resolver', () =>
    expect(Object.keys(SAMPLE_INPUT)).toEqual(
      ['storyId', 'coinability', 'matches', 'safety']));

  // The cases above test what we thought of; this tests the rule: over 5 000
  // generated inputs, every coin in a 'buy' or 'multi' action must trace to a
  // confirmed match whose safety entry has pass === true.
  test('buyable implies confirmed AND pass===true', () =>
    forAll(generateInputs(5_000), assertBuyableIsConfirmedAndSafe));
});
```

```
// packages/contracts/src/__typetests__/action.test-d.ts - compile-only.
// Descriptions, never literal TSxxxx codes: those are not a stable API.
import { BuyButton } from '@features/trading';

declare const raw: CoinMatchRow; declare const unsure: UnsureCandidate;
declare const confirmedOnly: ConfirmedCoin; declare const buyable: BuyableCoin;

// @ts-expect-error a raw database row is not a BuyableCoin
BuyButton({ coin: raw, size: 'row', feeBps: 50 });
// @ts-expect-error an abstained candidate can never reach the Buy button
BuyButton({ coin: unsure, size: 'row', feeBps: 50 });
// @ts-expect-error identity confirmed is not permission to buy: SAFE_TO_BUY missing
BuyButton({ coin: confirmedOnly, size: 'row', feeBps: 50 });
BuyButton({ coin: buyable, size: 'row', feeBps: 50 }); // the only legal call
```

2. The coin-match gates and the abstain band. The gate function mirrors

`coin_match_confirmed_requires_evidence` and `public.confirmable()` .

```
// apps/pipeline/src/stages/resolve-coins/__tests__/gates.test.ts
import { describe, expect, test } from 'vitest';
import { classifyMatch, confirmable } from '../model/gates';

describe('INVARIANT match gates', () => {
  const base = { score: 0.93, tickerSource: 'cashtag', relation: 'derived',
    mintTime: 1_800_000_000_000, s: { mintInPost: null, img: 0.71,
    text: 0.62, tick: null, social: null } } as const;

  const v = (o = {}) => classifyMatch({ ...base, ...o }).verdict;

  test('two strong channels confirm', () => expect(v()).toBe('confirmed'));

  test('ONE strong channel never confirms, at any score', () => {
    for (const score of [0.80, 0.90, 0.99, 1.0]) {
      expect(v({ score, s: { ...base.s, text: 0.20 } })).toBe('unsure');
    }
  });

  // mint_in_post alone confirms at ≥ 0.90; s_social alone never does, being
  // attacker-controlled metadata capped at 0.85 by CHECK.
  test.each([[ 'mintInPost', 0.95, 'confirmed' ], [ 'social', 0.85, 'unsure' ] ])(
    '%s alone → %s', (k, x, want) => expect(v(onlyChannel(k, x))).toBe(want));

  test.each([
    [ 'an llm ticker, at any evidence', { score: 1.0, tickerSource: 'llm' } ],
    [ 'unknown mint_time, which fails CLOSED', { mintTime: null } ],
    [ 'relation=mentioned, which is candidate generation', { relation: 'mentioned' } ],
  ])( '%s cannot confirm', (_, o) => expect(v(o)).toBe('unsure') );

  test('the abstain band is a range, not a point', () => {
    expect(v({ score: 0.79 })).toBe('unsure'); // below confirm, above reject
    expect(v({ score: 0.35 })).toBe('unsure');
    expect(v({ score: 0.34 })).toBe('rejected');
  });
});
```

Parity with SQL is the other half: `confirmable-parity.test.ts` pushes the Cartesian product of `[null, 0, 0.54, 0.55, 0.60, 0.89, 0.90, 1.0]` over the four channels through `public.confirmable()` in one `unnest` query and asserts the TypeScript predicate agrees on all 4 096 rows. It needs Postgres, so it runs in the `invariants` job.

3. Velocity and acceleration. The continuous-time EWMA over the irregular 4/9/14/21/30/42/58/78-minute grid is the arithmetic the whole posts board rests on.

```
// apps/pipeline/src/stages/snapshot/__tests__/velocity.test.ts
import { describe, expect, test } from 'vitest';
import { accumulate, burstOf, EMPTY } from '../model/velocity';

const min = (n: number) => n * 60_000;

describe('INVARIANT velocity', () => {
  // The old velocity.js returned 0 here, ranking a brand-new post as dead.
  // ACCEL renders an em dash on one snapshot, never a fabricated 1.0x.
  test('one snapshot → no rate, no burst, ticks=1', () => {
    const s = accumulate(EMPTY, { capturedAt: 0, views: 1_200 });
    expect(s.ticksQualified).toBe(1);
    expect(burstOf(s)).toEqual({ state: 'not_yet', note: 'needs 2 snapshots' });
  });

  // Constant arrival over IRREGULAR intervals must converge to burst 1.0. The
  // discrete form `alpha*x + (1-alpha)*S` does not: it weights every sample
  // equally regardless of the gap, biasing hot-tier posts upward by construction.
  test('constant rate on the geometric grid converges to burst ~1', () => {
    const grid = [4, 9, 14, 21, 30, 42, 58, 78].map(min);
    let s = EMPTY, views = 0, prev = 0;
    for (const t of grid) {
      views += 800 * ((t - prev) / 60_000); // exactly 800 views/min
      prev = t;
      s = accumulate(s, { capturedAt: t, views });
    }
    const b = burstOf(s);
    expect(b.state).toBe('known');
    if (b.state === 'known') expect(Math.abs(b.value - 1)).toBeLessThan(0.02);
  });
});
```

4. Lead time. Seven render branches; the two nobody writes are negative and a missing clock reading.

```
// packages/contracts/src/__tests__/lead.test.ts
import { describe, expect, test } from 'vitest';
import { leadTime } from '../lead';

const P = 1_800_000_000_000 as Instant;
const at = (m: number) => (P + m * 60_000) as Instant;
const clock = (o = {}) => ({ promotedAt: P, tMint: null, tCt: null,
  unmeasurable: false, promotedAtBackfilled: false, ...o });

describe('INVARIANT leadTime', () => {
  test.each([
    // t_crypto = min(t_mint, t_ct), from whichever side wins.
    [{ tMint: at(31), tCt: at(44) }, { kind: 'early', minutes: 31 }],
    [{ tMint: at(44), tCt: at(31) }, { kind: 'early', minutes: 31 }],
    // ~A THIRD of real detections are late. The number that proves the claim has
    // to render its own failure.
    [{ tCt: at(-7) }, { kind: 'late', minutes: -7 }],
    [{ tCt: at(1) }, { kind: 'tie', minutes: 1 }],
    [{ tCt: at(-1) }, { kind: 'tie', minutes: -1 }],
    // ONE reading is enough - min() over a single value. The old build had no
    // branch here at all and produced nothing.
    [{ tMint: at(18) }, { kind: 'early', minutes: 18 }],
  ])('%o -> %o', (c, expected) => expect(leadTime(clock(c))).toEqual(expected));

  // NEITHER clock: "+2h14m and counting". Never 0, never early, never a number.
  test('no clock reading at all -> open, and no minutes field exists', () => {
    const r = leadTime(clock({}), at(134));
    expect(r).toEqual({ kind: 'open', sinceMinutes: 134 });
    expect(r).not.toHaveProperty('minutes');
  });

  test('unmeasurable and backfilled are suppressed, never estimated', () => {
    expect(leadTime(clock({ tCt: at(31), unmeasurable: true })).kind).toBe('unmeasurable');
    expect(leadTime(clock({ tCt: at(31), promotedAtBackfilled: true })).kind)
      .toBe('suppressed');
  });
});
```

5. The safety normaliser. The old `api/safety.js` read `raw.mintAuthority` off `/report/summary`, whose schema (`error`, `lpLockedPct`, `mint`, `risks`, `score`, `score_normalised`, `tokenProgram`, `tokenType`) has **no authority fields at all**, then wrote `mintRevoked: mintAuthority == null ? true : false`. Every token was reported mint- and freeze-revoked; reproduced live on USDC, whose authorities are demonstrably active.

```
// apps/pipeline/src/stages/resolve-coins/__tests__/safety.test.ts
import { normaliseRugcheck } from '../model/safety';
import summaryShape from '../../../adapters/__fixtures__/rugcheck.summary.json';
import fullReport from '../../../adapters/__fixtures__/rugcheck.report.bonding.json';

describe('INVARIANT safety normaliser', () => {
  // THE REGRESSION. A response with no authority fields yields 'unknown', which
  // fails closed at intrinsic_gate_pass. It must NEVER yield 'revoked'.
  test('a response carrying no authority fields is unknown, not revoked', () => {
    const s = normaliseRugcheck(summaryShape); // the WRONG endpoint's shape
    expect([s.mint_authority, s.freeze_authority]).toEqual(['unknown', 'unknown']);
    expect(s.signals_checked).toBe(0);
  });

  test('present authorities are read literally, both ways', () => {
    expect(normaliseRugcheck({ ...fullReport, mintAuthority: 'BJE5MM...' })
      .mint_authority).toBe('active');
    expect(normaliseRugcheck({ ...fullReport, mintAuthority: null })
      .mint_authority).toBe('revoked'); // explicit null, FULL report only
  });

  // Raw top-10 on a normal fresh token is 67.89% because the pump.fun curve is
  // holder #1 at 46.73%, labelled {type:'AMM'} in RugCheck's own knownAccounts.
  test('top10 excludes AMM and LOCKER holders and sets the flag', () => {
    const s = normaliseRugcheck(fullReport);
    expect(s.curve_pda_excluded).toBe(true);
    expect(s.top10_pct).toBeLessThan(40);
  });
});
```

△ **Mark for verification.** RugCheck's exact field names on `GET /v1/tokens/{mint}/report` — `mintAuthority` , `freezeAuthority` , `knownAccounts[].type` , `topHolders[].pct` , `transferFee` — and whether `topHoldersPercentage` already nets out the curve PDA. The fixture is one recorded live call and the zod schema is derived from it; both want a re-fetch before the RISK column ships. Open item 8 in §10 is the same question.

6. Every formatter that can turn missing data into a plausible number. A registry, one property test, and a script keeping the registry exhaustive.

```
// packages/contracts/src/__tests__/pending.test.ts
import { FORMATTERS } from '../format-registry'; // name → (p) ⇒ PendingRender
import { awaiting, known, notYet, unobtainable, windowChange } from '../pending';

const ABSENT = [unobtainable('no_mint_time'), unobtainable('vendor_error'),
  unobtainable('vendor_absent'), notYet('coin is 41m old'), awaiting()];

describe('INVARIANT absent never renders as a number', () ⇒ {
  for (const [name, fmt] of Object.entries(FORMATTERS)) {
    for (const p of ABSENT) {
      test(`${name} + ${p.state}`, () ⇒ {
        const r = fmt(p);
        expect(['-', '..']).toContain(r.text);
        expect(r.text).not.toMatch(/\d/); // no 0, no 0.0%, no $0.00, no 0m
        expect(r.tone).toBe('dim');
      });
    }
  }

  test('a window longer than the coin age is not_yet, never 0.0%', () ⇒
    expect(windowChange(known(0.4), 1440, known(41)))
      .toMatchObject({ state: 'not_yet', note: 'coin is 41m old' }));
});
```

`scripts/check-formatters.mjs` asserts that every exported function in `shared/format` returning `PendingRender` appears in `FORMATTERS` . That is what makes this a property rather than a property test: a formatter written next month is covered the day it lands.

7. The retry classifier. The pipeline died silently for two days because the retry helper retried only on HTTP 429; a 402 threw and vanished. `errors.test.ts` is a `test.each` over `[429 → vendor_429, retryable]` , `[402 → vendor_402, terminal]` , `[401/403 → vendor_auth, terminal]` , `[500/503 → vendor_5xx, retryable]` , plus three assertions that carry the weight: an unrecognised error classifies `internal` , retryable **and** paging; a `23514` classifies `constraint_violation` at `warn` with the constraint name in `detail` , because a refused row is the schema working; and every `StageErrorCode` is reachable from some input, since a code nobody can produce is a code nobody has implemented.

9.3 The frozen regression set

What it is. `apps/pipeline/eval/match/pairs.jsonl` — 400 hand-labelled (story, mint) pairs, one JSON object per line, each carrying the five channel scores, `relation` , `ticker_source` , `mint_time - earliest_post_at` , a `stratum` and a boolean `label` . Same shape as `match_label` (§3.7), which is what lets it grow itself: the matcher writes every confirmed and unsure decision there with its feature vector.

How it is built. Seeded from the incidents — the real `$KANG` (LLM-invented ticker), `$PUMP` (generic-ticker collision) and `GYATT` (604-day-old shell adopted as derived) — then stratified to 400, because an unstratified sample is 85% easy positives and a threshold change moves precision by 0.002 on it:

STRATUM	N	WHY
<code>easy_positive</code>	120	the baseline; a regression here is catastrophic
<code>hard_positive</code>	60	transformed image, cropped meme, ticker drift
<code>ticker_collision</code>	60	two live coins share a symbol
<code>generic_ticker</code>	40	<code>\$MOON</code> , <code>\$PEPE</code> — the denylist's job
<code>adopted_shell</code>	40	old mints attaching to a new story
<code>llm_invented</code>	40	provenance, the \$KANG class
<code>near_miss</code>	40	genuinely ambiguous; these SHOULD abstain

What it blocks. `npm run test:match-regression` fails the build on precision below **0.970**, an abstain rate outside **0.15–0.30**, or any `llm_invented` or `adopted_shell` pair reaching `confirmed` . Precision is what D1 commits to, so it is a failing test and not a dashboard; the band is two-sided because an abstain rate that *drops* is the matcher getting bolder, which is how precision is lost.

```
// apps/pipeline/eval/match/__tests__/regression.test.ts
test('INVARIANT match precision ≥ 0.970 on the frozen set', () => {
  expect(pairs.length).toBeGreaterThanOrEqual(400); // the set cannot shrink
  const d = pairs.map((p) => ({ ...p, verdict: classifyMatch(p.features).verdict }));
  const yes = d.filter((x) => x.verdict === 'confirmed');
  const maybe = d.filter((x) => x.verdict === 'unsure');

  expect(yes.filter((x) => x.label).length / yes.length).toBeGreaterThanOrEqual(0.970);
  expect(maybe.length / (yes.length + maybe.length)).toBeGreaterThanOrEqual(0.15);
  expect(maybe.length / (yes.length + maybe.length)).toBeLessThanOrEqual(0.30);
  // Per-stratum, because an aggregate hides the strata that cost money.
  for (const st of ['llm_invented', 'adopted_shell'] as const) {
    expect(yes.filter((x) => x.stratum === st)).toHaveLength(0);
  }
  writeFileSync('eval-report.json', report(d)); // → PR comment
});
```

How it evolves. Pairs are appended, never edited or deleted — a wrong label gets `label_corrected_at` and a note, so the diff shows a human changing the truth rather than the test. `matcher_version` bumps in the same PR as any threshold change, and the PR body carries the before/after table from `eval-report.json` . Two hundred pairs is too few to move precision; a thousand is more labelling than this team will sustain. Four hundred, growing ~40/month from `match_label` , puts the interval on 0.97 at about ±0.017 — tight enough to see a real regression, loose enough that noise does not block merges. Δ That interval is binomial arithmetic, not a measurement; re-derive it once the set is real.

9.4 Testing the LLM stages

Meme scoring, coinability, titling and cluster adjudication are non-deterministic. The line: **the plumbing is tested, the judgement is measured.**

Deterministically testable in CI: **the parse** — output goes through a zod schema, and a response missing `coinability_tier` is `vendor_contract` , terminal, paging, never a default; tested against six recorded malformed responses (truncated JSON, prose before the JSON, an unknown tier, a null score, a score of `1.5` , an empty body). **The clamps** — a `meme_score` outside 0–1 is rejected, not clipped; an unrecognised tier maps to `never` , matching the column default. **The prompt builder** — pure and in `model/` , so a golden-file test turns an accidental prompt edit into a reviewable diff. **The budget path** — a `reserve_units()` false return stops the stage rather than falling through to an unmetered call.

Only measurable, nightly, against a frozen panel of 60 human-labelled posts in `apps/pipeline/eval/score/panel.jsonl` . Three signals detect a prompt regression, none a string comparison. **Rank correlation, not values:** Spearman p between the run's `meme_score` ordering and the frozen human ordering, alerting below **0.80** — absolute scores drift with model versions, and the *ordering* is what the board consumes. **Distribution drift:** Population Stability Index on the coinability histogram against the 30-day baseline, paging above 0.25, because a prompt edit quietly reclassifying 8% of stories `normal → no_create` empties the Create funnel and raises no error. **The safety floor, which is a hard assertion:** twelve panel posts must classify `never` — a named private individual, a death, violence, minors — and one of them scoring `normal` fails the job and pages. That is the only place LLM output gets an equality assertion, because D3 is a commitment, not a metric. Every run pins `model_id` and `sha256(prompt)` into `ops_event` , so "did the prompt change or did the model change" is already answered.

9.5 Integration tests

What genuinely needs Postgres: every CHECK constraint, every generated column (`intrinsic_gate_pass` , `lead_time_min` , `band` , `channels_ran`), every trigger (`freeze_columns` , `recount_story_coins` , `story_clock_copy_promoted_at` , `promote_launch`), RLS policies, column grants, and the four hot query plans — none of which can be tested against a mock, because they *are* the database.

The container is specified and must not change casually: `supabase/postgres:17.6.1.156` , pinned by exact tag, `POSTGRES_DB: postgres` . Not stock Postgres — `0011` grants to `anon` and `authenticated` , which only this image's init scripts create — and not `pgvector/pgvector` . Health-check over TCP, not the unix socket: the image runs ~60 platform migrations with `listen_addresses=''` , so a socket probe goes green before 5432 exists.

Fixture strategy, and what makes it fast. Migrating and seeding takes ~25 s; doing that per test file is how an integration suite becomes a thing people skip. So it happens once: CI marks the seeded database `datistemplate = true` , and `apps/pipeline/src/test/db.ts` exposes `freshDb()` , which issues `CREATE DATABASE t_{uuid}`

`TEMPLATE seeded` (~180 ms, a file copy), returns a pool and drops it in `onTestFinished`. Per-file isolation beats one shared database in a rolled-back transaction, because half of what we test *is* transactional: deferred constraints, trigger firing order, `SECURITY DEFINER` functions taking their own locks. A test that cannot commit cannot test a commit.

Fixtures are the committed seed — `00_fixtures.sql` captured from staging through a column allowlist, plus nine hand-written scenario files, never captured and never deleted: negative lead time, NULL `mint_time`, an unsure match with three candidates, a coin with no snapshots, a tier-`never` story. Generated data contains none of those shapes, which is precisely why they are the ones that break the product.

9.6 What replaces end-to-end

Full E2E against a live wallet and a live chain is not viable: real SOL, a funded key in CI, mainnet latency, flakiness unrelated to our code. Four things replace it, and between them they cover more of the buy flow than a browser suite would.

1. The trade state machine as a pure model. `trading/model/tx-state.ts` holds `ORDERING → SIGNED → SUBMITTED → LANDED | FAILED | EXPIRED | LOST_CONTACT`, and every transition including every illegal one is a table test. The states that hurt users — still confirming, lost contact, reverted after a fill — are unreachable in a happy-path browser test and trivial here.

2. Route handlers invoked directly. A Next route handler is `(req: Request) ⇒ Response`. Tests import `POST` from `@server/trading` and call it with a constructed `Request` against a seeded database and a stubbed Jupiter adapter — no browser, no server, no port. That covers the request-id uniqueness guard (a duplicate submission is a **409**, never a second fill), the server-side fee re-assertion, and `referralAccount` coming from env rather than the client.

3. `TRADING_MODE=simulate` everywhere but production, which `env_live_trading_is_production_only` makes structural rather than configured. In simulate the Jupiter adapter replays recorded order responses and signing is a no-op, so the whole flow — board click, recap rows, `trade` row — runs in CI on real data shapes.

4. Two live probes, outside the merge path. A **nightly vendor contract probe** hits Jupiter, RugCheck, DexScreener and Helius once each and parses each response through the adapter’s own zod schema; a parse failure is `vendor_contract` and pages. That is what catches an endpoint quietly dropping a field — the failure mode behind the safety defect, which no browser test would have caught either. And a daily **canary trade** on staging: a burner wallet buys and immediately sells ~0.005 SOL of a liquid mint, asserting the transaction lands and that `trade / holding` agree with the chain. It is the only thing proving signing and submission still work end to end. Δ Measure cost and slippage on that pair first.

9.7 The first ten tests

If only ten are ever written, these ten. Ranked by *money lost if absent*, not by ease.

- `action.test.ts` — the resolver table (§9.2.1).** Buy on exactly one confirmed *and safe* coin; never on unsure; never suppressed by absent market data. Every surface routes through it, so one test protects seven screens.
- `invariants.sql` — a confirmed match with one strong channel is rejected.** `verdict='confirmed'`, `score=0.91`, `s_img=0.80`, the rest NULL must raise `coin_match_confirmed_requires_evidence`. Then two strong channels accepted, and `story.confirmed_coin_count` becomes 1 — proving the recount trigger fired.
- `safety.test.ts` — a response with no authority fields yields `unknown`.** The exact shipped defect, in twelve lines.
- `invariants.sql` — an LLM ticker cannot carry a mint.** `source='llm'` with a `resolved_mint` raises `story_ticker_llm_never_resolves`; an insert omitting `source` raises a not-null violation. The second half is the point: provenance is not optional.
- `pending.test.ts` — absent never renders as a number.** The registry property test, which closes `if (!pair.pairCreatedAt) return 0` for every formatter at once — including the ones not written yet.
- `invariants.sql` — `promoted_at` cannot be moved.** Two statements encoding the whole earliness claim: the UPDATE raises, any other column on that row updates fine.
- `errors.test.ts` — `classify` is total, and 402 is terminal.** The two-day silent outage as a table test, including: an unmodelled error is `internal` and pages.
- `velocity.test.ts` — one snapshot yields no rate; constant rate on the irregular grid gives burst ≈ 1 .** The two arithmetic errors that make the board wrong while it looks right.
- `lead.test.ts` — negative and open.** `-7` renders as late at identical weight; no clock reading renders `open`, never `0`.
- `regression.test.ts` — precision ≥ 0.970 , abstain in 0.15–0.30.** Last only because it needs 400 labelled pairs first. It keeps the other nine honest when someone tunes a threshold to lift coverage.

Tests 1, 3, 5, 7, 8 and 9 need no database and no network; tests 2, 4 and 6 are three `pg_temp.rejects()` calls in a file that already exists. The whole list is under a day of work, and it is the difference between this build and the last one.

10. OPEN ITEMS

Everything below is unverified. Each carries the one test that settles it. Nothing here blocks starting; all of them block the surface they name.

#	ITEM	THE TEST THAT SETTLES IT
1	No migration has ever been executed. No Postgres was reachable while this schema was written; expect one or two syntax corrections on first apply.	<code>DATABASE_URL=...</code> <code>node scripts/migrate.mjs up</code> against the CI container. Two-minute loop.
2	<code>realtime.send(payload, event, topic, private)</code> signature. <code>0013</code> 's comment and match broadcast triggers depend on it. plpgsql late-binds, so the migration applies either way and the triggers fail at <i>runtime</i> .	<code>select realtime.send('{}'::jsonb, 'test', 'test', true);</code> on the live project. Before the comment composer ships.
3	<code>supabase gen types</code> against a database whose tables are RLS-protected and column-granted. Whether the generated row type reflects the <i>grant</i> or the <i>table</i> .	Run step 9 of <code>db.yml</code> and read <code>coin_match</code> in the output. If <code>score</code> is present, <code>rows.ts</code> must strip it explicitly.
4	Supabase Branching pricing, cold-start time and CLI surface. Supabase publishes no provisioning SLA; the ~2-minute figure in circulation is a client health-check timeout.	Create one branch, time it, read the invoice after a week. Set CI timeouts from the measurement, never from the rumour.
5	Privy preview domains. Wildcards are supported only as <code>*.domain.com</code> ; <code>*.vercel.app</code> and <code>*-hash.domain.com</code> are not.	Configure a Vercel Preview Deployment Suffix, allowlist <code>*.preview.insidor.app</code> , and connect a wallet on one preview.
6	Phase-2 authenticated principal. The unlock is not Privy claim injection — that cannot work, because <code>sub</code> is a DID and <code>app_user.id</code> is a uuid. It is a server-minted Supabase JWT via an imported signing key.	Mint one in the route handler with <code>{sub: <app_user.id>, role: 'authenticated'}</code> and confirm <code>select * from holding</code> returns the owner's rows. Also confirm whether a self-minted JWT counts against the third-party MAU meter.
7	Jupiter's referral fee floor. Documented at 50 bps; 10 bps was measured live on fresh mints. Deliberately not a CHECK — <code>fee_bps_quoted</code> is 0–255 and read from the order response.	Place one 0.01 SOL order and read <code>feeBps</code> off the response. On the day the swap route is written.
8	RugCheck field names, and whether <code>topHoldersPercentage</code> excludes the curve PDA. The schema refuses to store a top-10 figure without <code>curve_pda_excluded = true</code> , which will surface the answer immediately.	Fetch one pre-graduation pump.fun token and compare the reported figure against the on-chain holder list minus the curve PDA.
9	pgvector <code>halfvec</code> + HNSW at 768 and 512 dims on the deployed Postgres.	<code>create index ... using hnsw (embedding halfvec_cosine_ops)</code> in the CI container. Step 1 covers it.
10	<code>pg_cron</code> and <code>pg_net</code> availability , and the Vault-stored Slack webhook the DDL-audit pager reads. <code>0001</code> makes both optional with a NOTICE.	<code>select extname from pg_extension;</code> on the live project, then <code>select ops.page_on_out_of_band_ddl();</code> and watch Slack.
11	eslint-config-next v16 flat-config export shape under ESLint 10, and whether <code>no-restricted-imports</code> group negation (<code>'!@server/*'</code>) is honoured. If negation is not honoured, every server import fails lint on day one — loud, but an afternoon.	<code>npx eslint --print-config apps/web/src/app/layout.tsx</code> and read the output before trusting any rule.
12	Next 16 re-exported route handlers — <code>export { POST } from '@server/trading'</code> . Works on 14/15.	<code>curl -X POST</code> one route on a preview deployment. Fallback is a three-line wrapper, still inside the 40-line cap.
13	<code>next.config.ts</code> <code>turbopack.root</code> must equal <code>outputFileTracingRoot</code> or Turbopack refuses to start once Vercel injects its own.	<code>npm run build -w apps/web</code> locally, then one Vercel preview build.
14	The current repository is a single Next.js app with <code>api/</code> and <code>worker/</code> excluded from typechecking and written in JavaScript. Every type-level guarantee in §7 is worth nothing there.	<code>npx tsc --noEmit</code> after removing <code>api</code> and <code>worker</code> from <code>exclude</code> . Read the error count; that number is the size of the port.
15	<code>insidor.freeze_columns()</code> uses dynamic EXECUTE per column per row. Fine on the low-rate tables it is attached to; it will show up as CPU if it ever reaches <code>post</code> or <code>coin_snapshot</code> .	<code>explain (analyze, buffers)</code> a 10 000-row upsert into <code>post</code> after deliberately attaching it. Do not ship the attachment.
16	The <code>launch</code> composite-FK <code>ON UPDATE RESTRICT</code> makes reclassifying a launched story to tier <code>never</code> fail with a foreign-key error. Intentional, and it will land as a confusing production error without a runbook entry.	<code>update story set coinability_tier='never' where id = <a launched story>;</code> on staging. Write down the error text, put it in the runbook.
17	<code>post_embedding.model</code> is load-bearing — <code>gemini-embedding-001</code> and <code>-2</code> spaces are documented as incompatible, and no constraint enforces a single model because a re-embed has both present transiently.	A pipeline-side assertion at the start of every retrieval run: <code>select count(distinct model) from post_embedding</code> must be 1, or the run aborts.

#	ITEM	THE TEST THAT SETTLES IT
18	Key rotation is unaddressed. Every secret in §8.2 is set once and lives forever, including the <code>migrator</code> password and the service-role key.	Write the quarterly rotation runbook, then rotate the Helius key as a rehearsal and time the outage.